

Project Analysis Explanation

Today 12:46 PM



Assignment - Python_Django Developer _V1 (1)(1).docx



Document

now this project analysis and explain to me in simple to understand

Note : This project is for my interview so make perfect for me

Yes. I analyzed the assignment file. The main goal is to build a **Django-based AI-Assisted Box Selection System** for an e-commerce warehouse. The assignment specifically says that each product has dimensions/weight and each box has internal dimensions, maximum weight, and cost; your system must recommend the most suitable box for an order.

Assignment - Python_Django Deve...

Because this is for your **Python/Django interview**, I'll explain it in a way that helps you both **build it and explain it confidently to the interviewer**.

1. What is this project?

Imagine an online shopping website. A

customer orders:

Product: Laptop

Length = 30 cm

Width = 20 cm

Height = 3 cm Weight =

2 kg

The warehouse has different boxes:

Small	20	20	10	2 kg	₹10
Medium	35	25	10	5 kg	₹20
Large	50	40	20	10 kg	₹40

The system should automatically say: **Recommended Box:**

Medium

Why?

Because the laptop can fit inside the Medium box and its weight is within the box's capacity. The main purpose is:

Order → Check products → Check available boxes → Find suitable boxes → Select the best/cheapest suitable box → Show recommendation

2. What exactly does the assignment ask?

The assignment gives you several deliverables. You need to submit:

1. **GitHub repository**
2. **README.md**
3. **AI_USAGE.md**
4. **Chat transcript**
5. **What you learned**
6. **Test cases**
7. **Test output / GitHub Actions output**

Assignment - Python_Django Deve...

There is also an important rule:

The company allows AI tools such as ChatGPT, Claude, Cursor, and GitHub Copilot, but they want to see **your own thinking**.

Assignment - Python_Django Deve...

So don't blindly copy AI-generated code.

3. The most important part — Box Selection Logic

This is the **core of your interview assignment**. You need to decide:

Which box can contain all products in an order?

There are two main things to check.

Check 1 — Dimensions

Suppose the product is:

```
Length = 30 cm
Width  = 20 cm
Height = 5 cm
```

Box:

```
Length = 35 cm
Width  = 25 cm
Height = 10 cm
```

Then:

```
30 <= 35
20 <= 25
5  <= 10
```

So the product fits.

Check 2 — Weight

Product:

```
2 kg
```

Box capacity:

```
5 kg
```

Check:

```
2 <= 5
```

So weight is also valid.

Therefore:

This box is suitable.

4. What if the order has multiple products?

This is where your project becomes more interesting. Suppose an order contains:

```
Product A
Length = 20
Width = 10
Height = 5
Weight = 1 kg

Product B
Length = 15
Width = 10
Height = 5
Weight = 2 kg
```

Total weight:

```
1 + 2 = 3 kg
```

You need to determine whether the products can be packed into a box.

For an interview project, I recommend that you **clearly define your packing assumption** in your README.

For example:

"For this assignment, the system uses a simple packing strategy. Products are checked against box capacity and dimensions. Advanced 3D bin-packing/orientation optimization is outside the scope."

This is important.

Don't pretend that your system solves the complete real-world 3D packing problem if it doesn't.

Printed using [ChatGPT to PDF](#), powered by PDFCrowd [HTML to PDF API](#).

5. Recommended project architecture

I recommend a clean Django structure like this:

```
box-selection-system/
├── manage.py
├── config/
│   ├── __init__.py
│   ├── settings.py
│   ├── urls.py
│   ├── asgi.py
│   └── wsgi.py
├── box_selector/
│   ├── migrations/
│   ├── admin.py
│   ├── apps.py
│   ├── models.py
│   ├── views.py
│   ├── urls.py
│   ├── services.py
│   ├── tests.py
│   └── __init__.py
├── templates/
│   └── box_selector/
│       ├── home.html
│       ├── products.html
│       ├── boxes.html
│       └── recommendation.html
├── static/
├── requirements.txt
├── README.md
├── AI_USAGE.md
├── TEST_OUTPUT.md
└── .gitignore
```

Why services.py?

This is something I would strongly recommend for your interview.

Don't put all the box-selection logic directly inside `views.py`. Instead:

```
View
┆
┆ Service
┆
┆ Business Logic
┆
┆ Models / Database
```

For example:

```
Python
recommend_box(order)
```

can live in:

```
services.py
```

This makes the project easier to:

test

- maintain
- understand
- extend
- And you can tell the interviewer:

"I separated the business logic from the Django views so the box-selection algorithm can be tested independently."

That's a good interview answer.

6. Database design

You need at least three important concepts.

Product

Example:

```
Product
-----
name
length
width
height
weight
```

Example:

```
Laptop
30
20
3
2
```

Box

Example:

```
Box
-----
name
length
width
height
max_weight
cost
```

Example:

```
Medium Box
35
25
10
5
20
```

Order

Example:

```
Order
-----created_at
```

Then an order needs products.

You can create an intermediate/order-item model:

```
OrderItem
-----
order
product
quantity
```

This is better than directly putting products into the order because one order can contain:

```
Laptop x 2
Mouse x 1
Keyboard x 1
```

7. Database relationship

The structure becomes:

```
Order
|
| 1-to-many
↓
OrderItem
|
| many-to-one
↓
Product
```

For example:

```
Order #1001
|
├─ Laptop x 1
├─ Mouse x 2
└─ Keyboard x 1
```

This is a good example to explain Django relationships during your interview.

8. Django Models

Conceptually:

Python

```
class Product(models.Model):
    name = models.CharField(max_length=200)
    length = models.DecimalField(...)
    width = models.DecimalField(...)
    height = models.DecimalField(...)
    weight = models.DecimalField(...)
```

Box:

Python

```
class Box(models.Model):
    name = models.CharField(max_length=200)
    length = models.DecimalField(...)
    width = models.DecimalField(...)
    height = models.DecimalField(...)
    max_weight = models.DecimalField(...)
    cost = models.DecimalField(...)
```

Order:

```
Python class Order(models.Model):
    created_at = models.DateTimeField(auto_now_add=True)
```

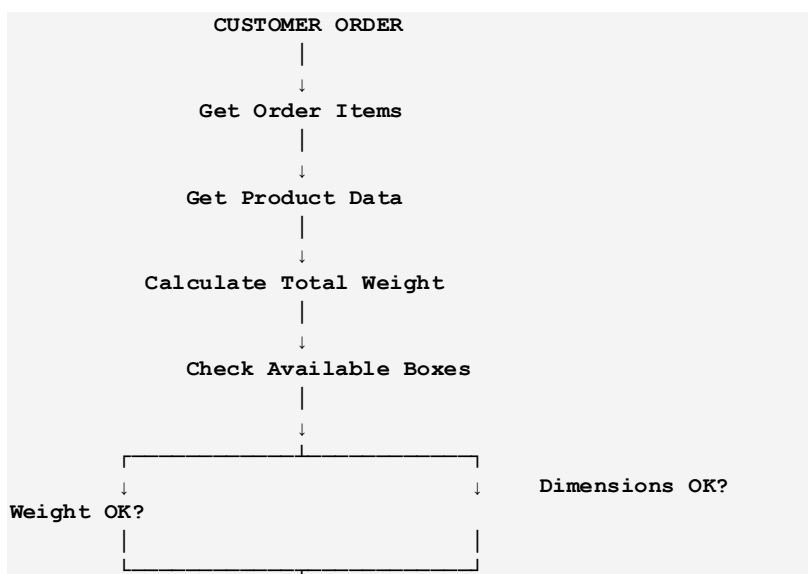
Order item:

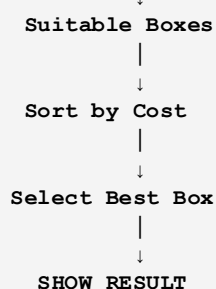
```
Python class OrderItem(models.Model):
    order = models.ForeignKey(Order, ...)
    product = models.ForeignKey(Product, ...)
    quantity = models.PositiveIntegerField(...)
```

The exact implementation should be yours, and you should understand every line before submitting.

9. How the complete system works

Think about the application as a pipeline:





That's the entire project in simple terms.

10. How should the "best box" be selected?

Suppose three boxes can contain the order:

Small	₹10
Medium	₹20
Large	₹40

If all three are suitable:

```

Small = suitable
Medium = suitable
Large = suitable
  
```

The system should choose:

Small — ₹10 because it is the

cheapest suitable box.

However, you should define what "best" means. For this assignment, I recommend:

Primary rule

Choose the lowest-cost box that can safely contain the order. If two boxes have the same cost:

```

Choose the smaller-volume box
  
```

You can explain this clearly in your README.

11. Important edge cases

This is where you can make your assignment look professional. You should test situations like: **Case 1 — Product fits**

```

Product:
30 x 20 x 5
2 kg
  
```



```
Box :
35 x 25 x 10
5 kg

Result:
Suitable
```

Case 2 — Product too large

```
Product:
50 x 40 x 30

Box:
35 x 25 x 10
```

Result:

```
Not suitable
```

Case 3 — Product too heavy

```
Product weight = 8 kg
Box capacity = 5 kg
```

Result:

```
Not suitable
```

Case 4 — Exact dimensions

```
Product:
30 x 20 x 10

Box:
30 x 20 x 10
```

This should normally be:

```
Suitable
```

because:

```
30 <= 30
20 <= 20
10 <= 10
```

Case 5 — No suitable box

If every box is too small or has insufficient weight capacity:

```
No suitable box found
```

Your application should handle this cleanly instead of crashing.

Case 6 — Multiple quantities Example:

Printed using [ChatGPT to PDF](#), powered by PDFCrowd [HTML to PDF API](#).

Laptop x 3

Your algorithm needs to correctly account for quantity.

12. One important interview-level issue: dimensions/orientation

Suppose:

Product:
10 x 20 x 30

Box:

30 x 20 x 10

The same dimensions exist but in a different order.

A more advanced system can rotate the product. For

example:

10 x 20 x 30

can potentially become:

30 x 20 x 10

depending on packing rules.

This is a good interview discussion.

You can say:

"For a production-level system, I would support dimension rotation and a proper 3D binpacking algorithm. For this assignment, I would keep the packing logic simple and clearly document the assumption."

That demonstrates that you understand the limitation rather than hiding it.

13. Where does "AI-Assisted" come in?

This is important.

The assignment title says:

AI-Assisted Box Selection System

But the assignment does **not** explicitly require you to build an AI/ML model.

Instead, it explicitly allows you to use AI tools during development. It asks you to document which AI tools you used, your prompts, accepted/rejected output, mistakes, and how you verified your final code.

Assignment - Python_Django Deve...

So don't unnecessarily create an ML model. Your actual application can use:

```
Django
+
Python
+
Database
+
Box selection algorithm
```

And you can document your development assistance in:

```
AI_USAGE.md
```

14. AI_USAGE.md

This file is **mandatory**.

The assignment asks you to document:

- AI tools used prompts
- accepted output
- rejected/modified output AI
- mistakes verification steps
- Assignment - Python_Django Deve...
- Example structure:

Markdown

```
# AI Usage

## Tools Used

- ChatGPT
- GitHub Copilot

## Purpose

I used AI tools to understand Django concepts,
review code structure, and generate ideas for
test cases. ## Prompts Used

1. Explain Django model relationships.
2. Suggest test cases for box selection.
3. Review my box selection algorithm.

## Accepted Output

I accepted suggestions related to:
- project structure
- test case ideas
- Django concepts

## Modified Output

I modified some generated code to match
my business rules. ## Rejected Output

I rejected code that did not correctly handle
product quantity. ## AI Mistakes

The AI initially suggested a solution that did
not properly handle multiple products. I
manually corrected it.

## Verification

I verified the final implementation using:
- unit tests
- manual testing
- Django test command
```

But don't copy this as your actual AI_USAGE.md. The assignment specifically wants your real AI usage, including your real prompts and mistakes.

Assignment - Python_Django Deve...

15. Very important — Chat transcript

The assignment says:

Export the chat transcript and attach/include it in your repository.

And it specifically says **not to generate this using AI**, otherwise the assignment can be rejected.

Assignment - Python_Django Deve...

So this part you should do yourself using the actual chat history/tool export provided by your environment.

Don't create a fake transcript.

16. README.md

Your README should explain the project. A professional structure:

```
# AI-Assisted Box Selection System

## Overview

## Features

## Technologies

## Project Structure

## Database Design

## Box Selection Logic

## Installation

## Running the Project ##

API / Application Usage

## Test Cases

## Limitations ##

Future Improvements

## AI Usage

## Author
```

The README is explicitly required by the assignment.

Assignment - Python_Django Deve...

17. Testing

Testing is another important part of this assignment.

The company specifically asks for **test cases** and **test run output**.

Assignment - Python_Django Deve...

You should create tests for:

```
✓ Product fits
✓ Product doesn't fit
✓ Weight exceeds capacity
✓ Exact dimensions
✓ Multiple products
✓ Product quantity
✓ No suitable box
✓ Cheapest suitable box selected
✓ Empty order
✓ Invalid data
```

Then run:

```
Bash
python manage.py test
```

You want something like:

```
Ran 10 tests
```

```
OK
```

You can either provide a GitHub Actions link or put the terminal output in:

```
TEST_OUTPUT.md
```

The assignment explicitly accepts either option.

Assignment - Python_Django Deve...

18. Recommended technology stack

Keep it simple.

```
Python
↓
Django
↓
SQLite
↓
HTML/CSS
```

For testing:

```
Django TestCase
```

For version control:

```
Git
GitHub
```

For CI:

```
GitHub Actions
```

You don't need to add unnecessary technologies just to make the project look complicated.

19. What I expect the final application to have

For an interview assignment, I would make a simple web interface.

Dashboard

```
-----
BOX SELECTION SYSTEM
-----Products    Boxes
Orders
Create Order
```

Product page

```
Products
```

Laptop	30 x 20 x 3	2 kg
Keyboard	40 x 15 x 5	1 kg
Mouse	10 x 6 x 4	0.2 kg

Box page

Boxes			
Small	20 x 20 x 10	2 kg	₹10
Medium	35 x 25 x 10	5 kg	₹20
Large	50 x 40 x 20	10 kg	₹40

Order page

```
Order #1001

Laptop x 1
Mouse x 1

[ Recommend Box ]
```

Result:

```
-----
Recommended Box
-----

Medium Box

Dimensions:
35 x 25 x 10 cm

Maximum Weight:
5 kg

Cost:
₹20
-----
```

That's enough to demonstrate Django properly.

20. What the interviewer may ask you

Be ready for these questions.

Q1. Why did you use Django?

Simple answer:

"I used Django because it provides a strong structure for web applications, ORM for database operations, URL routing, views, models, admin panel and testing."

Q2. How does your box selection algorithm work?

"First, I get all products in the order. Then I calculate the required weight and check the box dimensions and weight capacity. I remove boxes that cannot fit the order and select the lowest-cost suitable box."

Q3. Why did you create a service layer?

"I kept the business logic separate from the Django views so it is easier to test and maintain."

Q4. What happens if no box fits?

"The system returns a clear no-suitable-box result instead of failing."

Q5. What about multiple products?

"I use OrderItem to store each product and its quantity. The recommendation logic considers all items in the order."

Q6. What are the limitations?

A good answer:

"The current version uses a simple packing strategy. A real warehouse system would need more advanced 3D packing, product rotation, fragile-product rules, and possibly multiple boxes."

Q7. Did you use AI?

Be honest.

"Yes. I used AI as a development assistant to understand concepts, review ideas and generate test-case suggestions. I did not blindly accept the output. I verified the implementation using my own tests and documented the AI usage."

The assignment explicitly expects this kind of documentation.

Assignment - Python_Django Deve...

21. Your complete project roadmap

I recommend that we build it in this exact order:

```
STEP 1
Understand requirements
↓
STEP 2
Create GitHub repository
↓
STEP 3
Create Python virtual environment
↓
STEP 4
```



```

Install Django
↓
STEP 5
Create Django project
↓
STEP 6
Create box_selector app
↓
STEP 7
Create database models
↓
STEP 8
Run migrations
↓
STEP 9
Create Django admin
↓
STEP 10
Add Products
↓
STEP 11
Add Boxes
↓
STEP 12
Create Orders
↓
STEP 13
Implement box-selection service
↓
STEP 14
Create views/API
↓
STEP 15
Create frontend
↓
STEP 16
Handle errors/edge cases
↓
STEP 17
Write unit tests
↓
STEP 18
Run tests
↓
STEP 19
GitHub Actions
↓
STEP 20
README.md
↓
STEP 21
AI_USAGE.md
↓
STEP 22
Final testing
↓
STEP 23
Interview preparation

```

My recommendation for you

Since this is an **interview assignment**, don't try to build everything at once.

We should build it **step by step**, and for every step I'll explain:

What we are doing → Why we are doing it → Exact command/code → What each important line means → How to test it → What interviewer may ask.

That will make you understand the project instead of just copying code.

Next step should be **Step 1: finalize the requirements and business rules**, especially how we handle multiple products, quantities, dimensions, weight, and box selection. This is the most important part before writing Django code.



step 1

Step 1 — Finalize the Requirements

Before writing **any Django code**, we need to clearly understand what the company wants.

The assignment says the system is for an e-commerce warehouse. When an order is placed, the warehouse needs to know **which shipping box should be used**. Products have dimensions and weight, while boxes have internal dimensions, maximum weight capacity, and cost. The system must recommend the most suitable box.

Assignment - Python_Django Deve...

1. Understand the problem in simple English

Imagine this:

A customer buys:

Laptop

The laptop has:

```
Length = 30 cm
Width  = 20 cm
Height = 5 cm
Weight = 2 kg
```

The warehouse has:

```
Small Box
20 x 20 x 10 cm
Max weight = 2 kg
Cost = ₹10
```

```
Medium Box
35 x 25 x 10 cm
Max weight = 5 kg
Cost = ₹20
```

```
Large Box
50 x 40 x 20 cm
Max weight = 10 kg
Cost = ₹40
```

Our application should check the boxes and recommend the best one. In this example:

Small → ✗ Too small

```
Medium → ☒ Fits
Large  → ☒ Fits
```

So:

```
Recommended Box → Medium
```

2. What information does our system need?

We need three main things.

Product

A product needs:

```
Name
Length
Width
Height
Weight
```

Example:

```
Laptop
30 cm
20 cm
5 cm
2 kg
```

Box

A box needs:

```
Name
Internal Length
Internal Width
Internal Height
Maximum Weight
Cost
```

Notice the assignment specifically says **internal dimensions** for boxes.

Assignment - Python_Django Deve...

Example:

```
Medium Box

Length      = 35 cm
Width       = 25 cm
Height      = 10 cm
Max Weight  = 5 kg
Cost        = ₹20
```

Order

An order contains one or more products.

For example:

```
Order #1001
Laptop x 1
Mouse x 2
Keyboard x 1
```

So we also need to know:

```
Which product?
How many?
Which order?
```

3. Our basic database structure

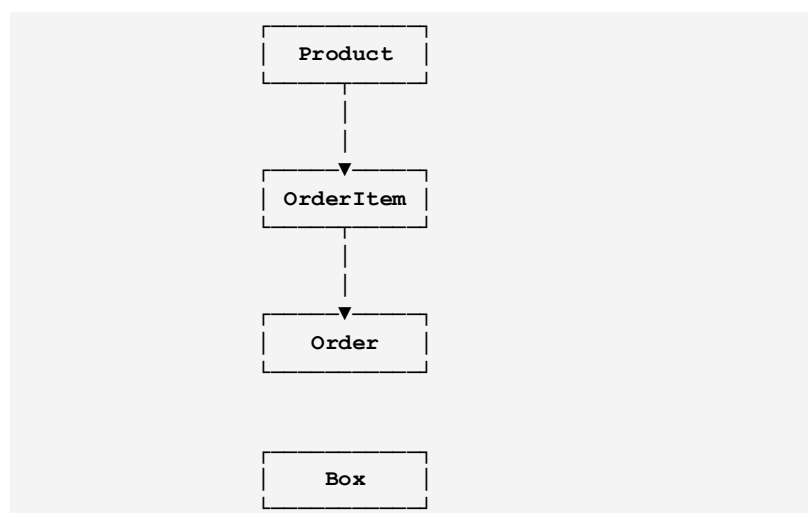
We will use four models:

```
Product
  ↑
  |
OrderItem
  |
  ↓
Order
```

And separately:

```
Box
```

More visually:



This will later become Django models.

4. Define our first business rule

The first rule is:

A product can fit into a box only if its dimensions fit inside the box dimensions.

Simple example:

```
Product:
30 x 20 x 5

Box:
35 x 25 x 10
```

Check:

```
30 <= 35  ☒
20 <= 25  ☒
5  <= 10  ☒
```

Therefore:

```
Product fits → YES
```

5. Second business rule — Weight

The product's weight must not exceed the box's maximum weight capacity. Example:

```
Product weight = 2 kg
Box capacity   = 5 kg
```

Check:

```
2 <= 5
```

Result:

```
Weight fits → YES
```

But:

```
Product weight = 7 kg
Box capacity   = 5 kg
```

Result:

```
Weight fits → NO
```

6. Third business rule — Find suitable boxes

Suppose we have:

```
Small  ☒
Medium ☒
Large  ☒
```

We don't want to simply choose the first box.

We need to compare the suitable boxes.

For example:

```
Medium → ₹20
Large → ₹40
```

The logical choice is:

```
Medium → ₹20
```

So our initial rule will be:

Among suitable boxes, select the lowest-cost box.

This is a **design decision we are making for the implementation**; the assignment itself says the box has a cost and asks for the "most suitable" box, but it does not explicitly define a cost-optimization rule.

Assignment - Python_Django Deve...

That distinction is important in an interview. If the

interviewer asks:

"Why did you choose the cheapest box?" You can

answer:

"The assignment provides a cost for each box but does not define an exact selection formula, so I chose the lowest-cost valid box as my business rule and documented that assumption." Excellent answer.

7. What about multiple products?

This is an important requirement that the assignment **doesn't fully specify**. For example:

```
Order #1
Laptop × 1
Mouse × 1
Keyboard × 1
```

The real-world problem is a 3D packing problem.

We need to decide how our assignment handles it.

For our interview project

I recommend we explicitly use a **simple packing assumption**, rather than pretending to implement a complete warehouse packing algorithm. We'll document something like:

"The assignment implementation uses a simplified packing approach. The system checks whether the order can be accommodated based on the defined dimensions and total weight.

Advanced 3D packing optimization and complex product rotation are outside the current scope."

Why?

Because proper 3D packing can become a very large algorithmic problem. This keeps your project:

- understandable
- testable
- realistic for an interview assignment
- easy to explain

8. Product quantity

We should support:

```
Laptop x 2
```

instead of creating two separate product records. So:

```
OrderItem
Product = Laptop
Quantity = 2
```

This is why `OrderItem` is useful. Example:

```
Order #1001
Laptop      x 2
Mouse       x 1
Keyboard    x 1
```

9. What happens when no box fits?

We must handle this properly. Example:

```
Product: 100 x 100 x 100 cm
20 kg
```

Available boxes:

```
Small
Medium
Large
```

If none can contain it:

```
✗ No suitable box found
```

The application should show a clear message instead of crashing. For example:

```
No suitable box is available for this order.
```

10. What if dimensions are exactly equal?

Example:

```
Product:
30 x 20 x 10

Box:
30 x 20 x 10
```

We should allow this:

```
30 <= 30 ☒
20 <= 20 ☒
10 <= 10 ☒
```

So:

```
Suitable → YES
```

11. What about rotating products?

This is a good interview question. Suppose:

```
Product:
10 x 20 x 30
```

Box:

```
30 x 20 x 10
```

Technically, rotation might allow the product to fit.

But the assignment does **not specify how product orientation/rotation should be handled**. Therefore, we should not silently claim that we support every possible rotation.

For version 1, we can keep the dimension rule simple and document our assumption. If the interviewer asks:

"Can your system rotate products?" Say:

"The current version uses a simplified dimension check. If this were a production system, I would add dimension rotation and a proper 3D bin-packing algorithm."

12. What does "AI-Assisted" mean here?

This is another important point.

The assignment allows AI tools such as ChatGPT, Claude, Cursor, and GitHub Copilot. It asks you to document your AI usage.

Assignment - Python_Django Deve...

It does **not** say that we need to create an AI/ML model to choose the box.

So we should **not unnecessarily build machine learning**. Our core application can be:

```
Python
↓
Django
↓
Database
↓
Business Logic
↓
Box Recommendation
```

AI is mainly part of the **development process**, which we document in `AI_USAGE.md`.

13. Our final requirements

Let's freeze the requirements before coding.

Functional requirements Our system should:

1. Create products
2. Create boxes
3. Create orders
4. Add products to orders
5. Support product quantity
6. Check product dimensions
7. Check weight capacity
8. Find suitable boxes
9. Select the lowest-cost suitable box
10. Show the recommendation
11. Handle no suitable box
12. Provide automated tests

14. Non-functional requirements

We should also make the project:

Simple

Don't add unnecessary technologies.

Maintainable

Keep business logic separate from views.

Testable

The recommendation algorithm should be easy to test.

Documented

We need:

```
README.md
```

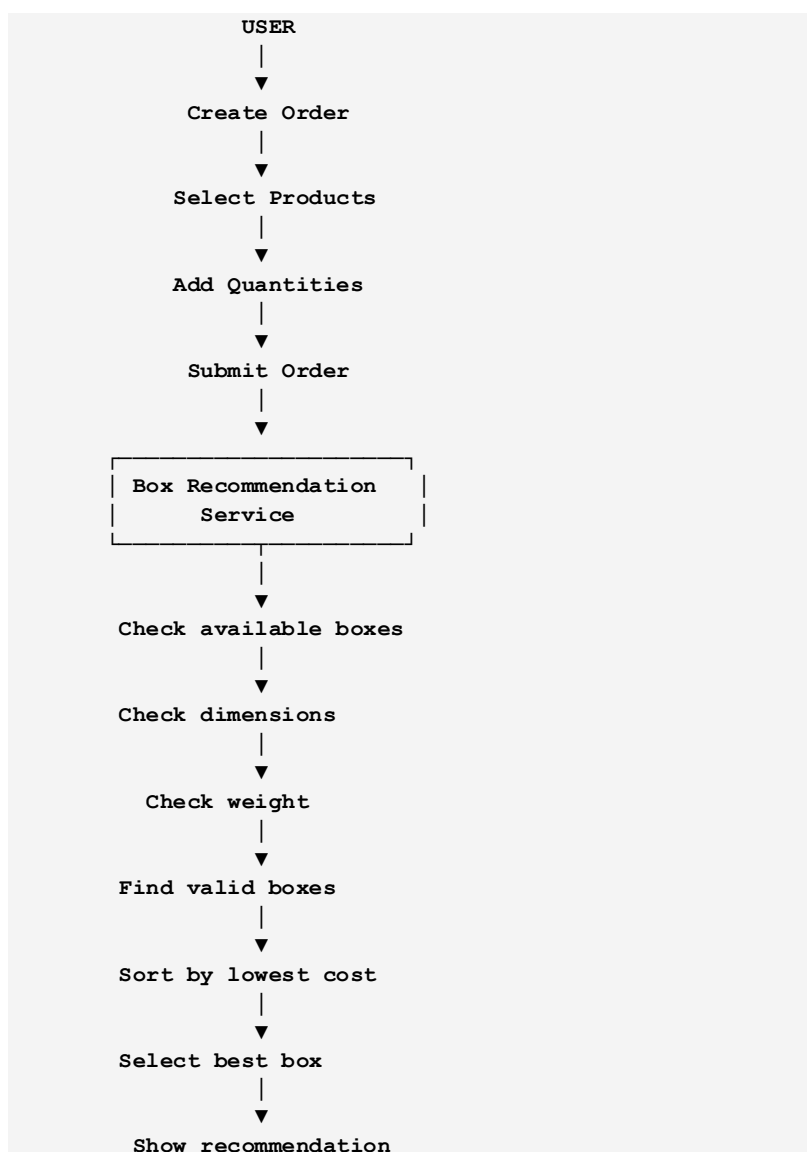
AI_USAGE.md
TEST_OUTPUT.md

The assignment explicitly requires the README, AI usage documentation, test cases, and test output.

Assignment - Python_Django Deve...

15. Final flow of our application

This is the most important diagram to remember for your interview:



16. Interview answer — "Explain your project"

You should eventually be able to say this naturally:

"I built a Django-based box selection system for an e-commerce warehouse. The system stores products with their dimensions and weight, and boxes with their internal dimensions, maximum weight capacity and cost. When an order is created, the system checks the available boxes against the order requirements and finds suitable boxes. From the suitable boxes, it selects the lowest-cost box and shows the

recommendation. I also added validation, edge-case handling and automated tests." That's a very good fresher-level interview explanation.

Step 1 is now complete ☒

Our project requirements are:

```
Product
├── Name
├── Length
├── Width
├── Height
└── Weight

Box
├── Name
├── Length
├── Width
├── Height
├── Max Weight
└── Cost

Order
└── OrderItems
    ├── Product
    └── Quantity
```

And the core logic is:

```
Order
↓
Check dimensions
↓
Check weight
↓
Find suitable boxes
↓
Choose lowest-cost suitable box
↓
Return recommendation
```

Next: Step 2 — Set up the Django project properly (Python virtual environment, Django installation, project/app creation, folder structure, and why each command is used).



step 2

Step 2 — Set Up the Django Project

Now we will actually start building the project.

In Step 1, we decided that our project will be:

E-commerce Order → Box Selection → Best Box Recommendation The

assignment asks for a **Django-based system**.

Assignment - Python_Django Deve...

We will do this step carefully because you should understand every command for your interview.

2.1 Check Python

First, open **Command Prompt / PowerShell / VS Code Terminal**. Run:

```
Bash python --  
version
```

You should get something like:

```
Python 3.12.5
```

Your Python version can be different. Also try:

```
Bash pip --  
version
```

You should see something like:

```
pip 24.x.x
```

What is Python?

Python is the programming language we will use to build the application.

What is pip?

`pip` is Python's package manager.

We will use it to install Django and other packages.

2.2 Create the Project Folder

Create a folder for the project. For

example:

```
box-selection-system
```

Then open that folder in VS Code.

Your folder should initially look like:

```
box-selection-system/
```

2.3 Create a Virtual Environment

This is an **important interview concept**. Run:

```
Bash python -m venv
venv
```

This creates:

```
box-selection-system/
├──
└── venv/
```

What is a virtual environment?

A virtual environment creates an isolated Python environment for your project. For example, suppose:

```
Project A → Django 4
Project B → Django 5
```

If both projects use the same global Python packages, versions can conflict.

A virtual environment avoids this problem. You can explain to the interviewer:

"I use a virtual environment so that the project's Python dependencies are isolated from the system and other projects." That's a good interview answer.

2.4 Activate the Virtual Environment

Because you are probably using Windows, run:

```
Bash
venv\Scripts\activate
```

After activation, your terminal should look something like:

```
(venv) C:\...\box-selection-system>
```

The important part is:

```
(venv)
```

That means the virtual environment is active.

2.5 If Activation Doesn't Work

If PowerShell gives an execution-policy error, you can use:

Printed using [ChatGPT to PDF](#), powered by PDFCrowd [HTML to PDF API](#).

```
Bash
venv\Scripts\activate.bat
```

or open **Command Prompt** and run:

```
Bash
venv\Scripts\activate
```

Don't worry if the first command doesn't work in PowerShell.

2.6 Upgrade pip

Now run:

```
Bash python -m pip install --
upgrade pip
```

This updates pip to the latest available version.

2.7 Install Django

Now install Django:

```
Bash pip install
django
```

After installation, check:

```
Bash
django-admin --version
```

You should see a Django version. For example:

```
5.x.x
```

The exact version isn't important for this step.

2.8 Save Dependencies

Now run:

```
Bash pip freeze >
requirements.txt
```

You will get:

```
box-selection-system/
```

```
|
├── venv/
└── requirements.txt
```

Why do we need `requirements.txt`?

Suppose another developer downloads your GitHub project. They can install the required packages using:

```
Bash pip install -r
requirements.txt
```

So `requirements.txt` tells them which Python packages your project needs.

Interview question

Q: Why do you use `requirements.txt`? Answer:

"It stores the project's Python dependencies and their versions so the same environment can be recreated easily."

2.9 Create the Django Project

Now we create the actual Django project. Run:

```
Bash
django-admin startproject config .
```

Important: Keep the `.` at the end. The

folder will now look like:

```
box-selection-system/
├── venv/
├── requirements.txt
├── manage.py
└── config/
    ├── __init__.py
    ├── settings.py
    ├── urls.py
    ├── asgi.py
    └── wsgi.py
```

2.10 Understand `manage.py`

You will see:

```
manage.py
```

This is one of the most important Django files.

We use it to execute Django commands. For example:

```
Bash python manage.py  
runserver
```

```
Bash python manage.py  
migrate
```

```
Bash  
  
python manage.py test
```

```
Bash python manage.py  
createsuperuser
```

Interview answer

Q: What is manage.py?

"manage.py is a command-line utility that helps us perform administrative tasks for a Django project, such as running the server, migrations and tests."

2.11 Understand the `config` Folder

We created:

```
config/
```

This is our Django **project configuration package**. It contains:

```
config/  
|  
├─ settings.py  
├─ urls.py  
├─ asgi.py  
└─ wsgi.py
```

Let's understand them.

settings.py

This contains project configuration. For example:

```
Database
```



```
Installed apps
Middleware
Templates
Static files
Security settings
```

We will spend more time on this later.

`urls.py`

This controls URL routing. For

example:

```
/products/
```

can eventually point to our product page. Later:

```
/orders/
```

can point to our order functionality.

`asgi.py`

ASGI is used for deploying Django applications and supporting asynchronous server interfaces.

You don't need to deeply understand it for this assignment.

`wsgi.py`

WSGI is another standard interface used to deploy Django applications. For a

fresher interview, knowing its purpose is enough.

2.12 Run the Django Server

Now run:

```
Bash python manage.py
runserver
```

You should see something similar to:

```
Starting development server at http://127.0.0.1:8000/
```

Open the address in your browser:

```
http://127.0.0.1:8000/
```

You should see the Django welcome page.

 **Your Django project is working.**

2.13 What just happened?

The flow was:

```
Python
↓
Virtual Environment
↓
Install Django
↓
django-admin startproject
↓
Django Project
↓
manage.py
↓
runserver
↓
Browser
```

2.14 Create Our Django App

Now we need an application inside our Django project. Run:

```
Bash
python manage.py startapp box_selector
```

Now your project becomes:

```
box-selection-system/
├── venv/
├── manage.py
├── requirements.txt
├── config/
├── box_selector/
│   ├── migrations/
│   ├── __init__.py
│   ├── admin.py
│   ├── apps.py
│   ├── models.py
│   ├── tests.py
│   └── views.py
```

2.15 Project vs App — Very Important

This is a common Django interview question.

Project

The **project** is the complete Django website/application configuration. In our case:

```
config
```

App

An **app** is a specific part of the project that handles a particular functionality. Our app:

```
box_selector
```

You can remember it like this:

```
Django Project
|
├─ Product functionality
├─ Order functionality
├─ Box functionality
└─ Recommendation functionality
```

For our assignment, we can keep these related functionalities inside one main app:

```
box_selector
```

Interview answer

"A Django project is the complete application configuration, while a Django app is a reusable module that handles a particular functionality."

2.16 Register the App

Open:

```
config/settings.py
```

Find:

```
Python
INSTALLED_APPS = [
```

You'll see Django's default apps. Add:

```
Python
'box_selector',
```

So it will look approximately like:

```
Python
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
```

```
'django.contrib.contenttypes',
'django.contrib.sessions',
'django.contrib.messages',
'django.contrib.staticfiles',

'box_selector',

]
```

Why?

Django needs to know:

"This application is part of my project."

Without registering it, Django won't properly load our app's models and other components.

2.17 Check the Project

Run:

```
Bash python manage.py
check
```

If everything is correct, you should get:

```
System check identified no issues (0 silenced).
```

This is a very useful command.

It checks for common Django configuration problems without starting the server.

2.18 Run the Server Again

Run:

```
Bash python manage.py
runserver
```

Open:

```
http://127.0.0.1:8000/
```

If the Django page appears, our basic setup is working.

2.19 Our Current Folder Structure

At the end of Step 2, we should have:

```
box-selection-system/
|
└─ venv/
```

```

├── box_selector/
│   ├── migrations/
│   ├── __init__.py
│   ├── admin.py
│   ├── apps.py
│   ├── models.py
│   ├── tests.py
│   └── views.py
├── config/
│   ├── __init__.py
│   ├── asgi.py
│   ├── settings.py
│   ├── urls.py
│   └── wsgi.py
├── manage.py
└── requirements.txt

```

2.20 What each important file does

<code>manage.py</code>	Run Django commands
<code>settings.py</code>	Project configuration
<code>urls.py</code>	URL routing
<code>models.py</code>	Database models
<code>views.py</code>	Request/response logic
<code>admin.py</code>	Django admin configuration
<code>tests.py</code>	Tests
<code>requirements.txt</code>	Python dependencies
<code>migrations/</code>	Database schema changes

2.21 Don't Commit `venv`

We should **not upload** `venv/` to GitHub. Create a:

```
.gitignore
```

with:

```

venv/
__pycache__/
*.pyc db.sqlite3 .env

```

Later we'll improve this `.gitignore`.

Why? `venv/` can contain thousands of files and doesn't need to be stored in Git.

Another developer can simply create their own environment and run:

```
Bash pip install -r
requirements.txt
```

2.22 Step 2 Interview Questions

You should understand these five questions.

1. Why virtual environment?

"To isolate project dependencies from the system and other Python projects."

2. What is Django?

"Django is a Python web framework used to build web applications quickly with features like ORM, URL routing, authentication, admin and testing."

3. Project vs App?

"A project is the complete Django application configuration, while an app handles a specific functionality."

4. What is `manage.py`?

"It is a command-line utility used to perform Django project tasks."

5. Why `requirements.txt`?

"It stores the project's dependencies so the environment can be recreated."

Step 2 Checklist ☒

Before moving to Step 3, make sure you can do all of these:

- ☒ Python installed
- ☒ Project folder created
- ☒ Virtual environment created
- ☒ Virtual environment activated
- ☒ Django installed
- ☒ `requirements.txt` created
- ☒ Django project created
- ☒ Django server runs
- ☒ `box_selector` app created
- ☒ App added to `INSTALLED_APPS`
- ☒ `python manage.py` check works
- ☒ `.gitignore` created

The most important thing

Don't just copy the commands. Run them yourself and make sure you understand what happened.

Next, **Step 3 will be Database Design + Django Models**. We'll create the `Product`, `Box`, `Order`, and `OrderItem` models and I'll explain **ForeignKey, relationships, fields, migrations, and why we choose each field**.



step 3

Step 3 — Database Design + Django Models

Now we move to one of the **most important parts of the project**: designing the database. Our assignment tells us that:

- Products have **dimensions and weight**.
- Boxes have **internal dimensions, maximum weight capacity, and cost**.
- An order contains products.
- The system uses this information to recommend a suitable box.

Assignment - Python_Django Deve...

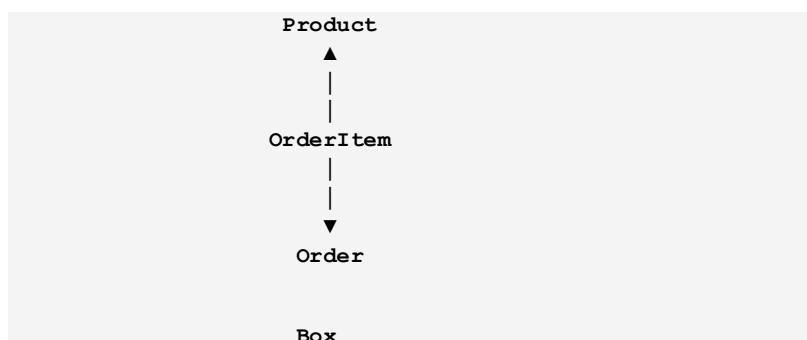
We will convert these requirements into Django models.

3.1 First understand our database

We decided to use **4 models**:

```
Product
Box
Order
OrderItem
```

The relationship is:



Why do we need `orderItem`? Suppose

an order is:

```
Order #1001
Laptop x 2
Mouse x 1
```

We don't want to duplicate the Laptop record. Instead:

```
OrderItem
-----
Order    → 1001
Product  → Laptop
Quantity → 2
```

And:

```
OrderItem
-----
Order    → 1001
Product  → Mouse
Quantity → 1
```

This is a standard and clean database design.

3.2 Product Model

Let's start with `Product`. A

product needs:

```
Name
Length
Width
Height
Weight
```

Open:

```
box_selector/models.py
```

For now, create:

```
Python
from django.db import models

class Product(models.Model):
    name = models.CharField(max_length=200)
    length = models.DecimalField(max_digits=10, decimal_places=2)
    width = models.DecimalField(max_digits=10, decimal_places=2)
    height = models.DecimalField(max_digits=10, decimal_places=2)
    weight = models.DecimalField(max_digits=10, decimal_places=2)

    def __str__(self):
        return self.name
```

Now let's understand it.

3.3 `models.Model`

```
Python class
Product(models.Model):
```


This tells Django:

"Product is a database model."

Django will create a database table for it. Conceptually:

```
Product table id | name | length | width |
height | weight
```

3.4 name

```
Python name =
models.CharField(max_length=200)
```

This stores the product name. Example:

```
Laptop
Keyboard
Mouse
Mobile
```

CharField is used for short text.

3.5 Dimensions

We use:

```
Python length = models.DecimalField(
max_digits=10,    decimal_places=2
)
```

Same for:

```
width
height
```

Why DecimalField?

Because dimensions can have decimal values. For example:

```
30.50 cm
```

rather than only:

```
30 cm
```

3.6 Weight

```
Python weight = models.DecimalField(
max_digits=10,    decimal_places=2
)
```

Example:

```
2.50 kg
```

Again, decimal values are useful for weight.

3.7 __str__

```
Python def
__str__(self):
return self.name
```

This makes the Django admin display:

```
Laptop
Mouse
Keyboard
```

instead of something like:

```
Product object (1)
Product object (2)
```

This is a small thing, but it makes your admin interface much cleaner.

3.8 Box Model

Now create the `Box` model.

Add below `Product`:

```
Python class Box(models.Model):    name =
models.CharField(max_length=200)
length = models.DecimalField(max_digits=10, decimal_places=2)
width = models.DecimalField(max_digits=10, decimal_places=2)    height
= models.DecimalField(max_digits=10, decimal_places=2)    max_weight =
models.DecimalField(max_digits=10, decimal_places=2)    cost =
models.DecimalField(max_digits=10, decimal_places=2)

def __str__(self):
return self.name
```

3.9 Why these fields?

Our box has:

```
name
length
width
height
```

```
max_weight
cost
```

This directly represents the assignment requirements. The assignment states that each box has internal dimensions, maximum weight capacity, and cost.

Assignment - Python_Django Deve...

Example database record:

```
Medium Box
35
25
10
5
20
```

Meaning:

```
Name      = Medium Box
Length    = 35 cm
Width     = 25 cm
Height    = 10 cm
Max Weight = 5 kg
Cost      = ₹20
```

3.10 Order Model

Now create:

```
Python class Order(models.Model):      created_at =
models.DateTimeField(auto_now_add=True)

    def __str__(self):
    return f"Order #{self.id}"
```

This stores the order. For

example:

```
Order #1
Created: 2026-09-03
```

3.11 Why created_at?

We want to know when the order was created.

```
Python
created_at = models.DateTimeField(auto_now_add=True)
```

`auto_now_add=True` means Django automatically sets the date/time when the object is first created. So we don't have to manually enter:

```
2026-09-03 12:30
```

3.12 OrderItem Model

Now comes the most important relationship.

Create:

```
Python class
OrderItem(models.Model):
    order = models.ForeignKey(
        Order,
        on_delete=models.CASCADE,
        related_name="items"
    )
    product = models.ForeignKey(
        Product,
        on_delete=models.CASCADE
    )
    quantity = models.PositiveIntegerField()

    def __str__(self):
        return f"{self.product.name} x {self.quantity}"
```

3.13 What is ForeignKey?

This is a very important Django interview question.

```
Python
order = models.ForeignKey(Order,
...)
```

means:

An OrderItem belongs to an Order.

And:

```
Python
product = models.ForeignKey(Product, ...)
```

means:

An OrderItem belongs to a Product.

So:

```
Order
|
├── OrderItem
├── OrderItem
└── OrderItem
```

And each OrderItem points to one Product.

3.14 Why ForeignKey?

Imagine:

```
Order #1001
Laptop x 2
Mouse x 1
```

```
Keyboard x 1
```

Database:

```
Order
-----
id
1001
```

OrderItem:

```
id | order | product | quantity -----
1  | 1001  | Laptop  | 2
2  | 1001  | Mouse   | 1
3  | 1001  | Keyboard| 1
```

This is much cleaner than storing everything directly inside `Order`.

3.15 What is `on_delete=models.CASCADE`?

You'll see:

```
Python
on_delete=models.CASCADE
```

This tells Django what to do if the referenced object is deleted. For example:

```
Order
  ↓
OrderItem
```

If an `Order` is deleted, its related `OrderItems` should also be deleted.

That's what `CASCADE` means.

You should be able to explain:

"`CASCADE` deletes related child records when the parent record is deleted."

3.16 What is `related_name="items"`?

We have:

```
Python
related_name="items"
```

This lets us easily access the order's items. For

example:

```
Python
order.items.all()
```

This means:

Give me all OrderItems belonging to this order. Without it,

Django uses a default reverse relationship name.

`related_name="items"` simply gives us a clean name.

3.17 What is `PositiveIntegerField`?

We have:

```
Python quantity =  
models.PositiveIntegerField()
```

Quantity should not normally be:

```
-5
```

So `PositiveIntegerField` makes the intention clear:

```
1  
2  
3  
10 ...
```

3.18 Complete `models.py`

At this point, your file should look approximately like:

Python

```
from django.db import models

class Product(models.Model):
    name = models.CharField(max_length=200)
    length = models.DecimalField(max_digits=10, decimal_places=2)
    width = models.DecimalField(max_digits=10, decimal_places=2)
    height = models.DecimalField(max_digits=10, decimal_places=2)
    weight = models.DecimalField(max_digits=10, decimal_places=2)

    def __str__(self):
    return self.name

class Box(models.Model):
    name = models.CharField(max_length=200)
    length = models.DecimalField(max_digits=10, decimal_places=2)
    width = models.DecimalField(max_digits=10, decimal_places=2)
    height = models.DecimalField(max_digits=10, decimal_places=2)
    max_weight = models.DecimalField(max_digits=10, decimal_places=2)
    cost = models.DecimalField(max_digits=10, decimal_places=2)

    def __str__(self):
    return self.name

class Order(models.Model):
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
    return f"Order #{self.id}"

class OrderItem(models.Model):
    order = models.ForeignKey(
        Order,
        on_delete=models.CASCADE,
        related_name="items"
    )
    product = models.ForeignKey(
        Product,
        on_delete=models.CASCADE
    )
    quantity = models.PositiveIntegerField()

    def __str__(self):
    return f"{self.product.name} x {self.quantity}"
```

3.19 Now Create the Database Tables

We've written Python classes.

But the database doesn't know about them yet.

We need **migrations**. Run:

```
Bash python manage.py
makemigrations
```

You should see something similar to:

```
Migrations for 'box_selector':
box_selector/migrations/0001_initial.py
- Create model Box
```

```
- Create model Order
- Create model Product
- Create model OrderItem
```

3.20 What is makemigrations?

This is a common interview question.

makemigrations means:

Django looks at the changes in our models and creates migration files describing those database changes.

Think:

```
models.py
↓
makemigrations
  ↓ migration
  file
```

It **doesn't actually apply** the changes to the database yet.

3.21 Apply the Migration

Now run:

```
Bash python manage.py
migrate
```

Django will create/update the database tables.

The flow is:

```
models.py
↓
makemigrations
  ↓
  0001_initial.py
    ↓
    migrate
      ↓
      Database
```

3.22 What is SQLite?

By default, Django creates:

```
db.sqlite3
```

We can use SQLite for this assignment.

For an interview project, SQLite is perfectly reasonable because this is a small assignment. Later, if the interviewer asks:

"Can this use PostgreSQL?" Answer:

"Yes. Django's ORM allows us to switch to PostgreSQL with database configuration changes, while keeping most of the application logic the same."

3.23 Check Your Database

You should now see:

```
box-selection-system/
| |
| db.sqlite3
|
| box_selector/
|   migrations/
|     |
|     | __init__.py |
|     | 0001_initial.py
|     |
|     ...
```

3.24 Create Superuser

We need the Django admin panel to easily add products and boxes. Run:

```
Bash python manage.py
createsuperuser
```

It will ask:

```
Username:
Email address:
Password:
Password (again):
```

Use your own values. For

example:

```
Username: admin
```

Don't use a weak password in a real deployment.

3.25 Register Models in Admin

Open:

```
box_selector/admin.py
```

Add:

```
Python
from django.contrib import admin
from .models import Product, Box, Order, OrderItem

admin.site.register(Product) admin.site.register(Box)
admin.site.register(Order) admin.site.register(OrderItem)
```

Now Django knows that we want these models available in the admin panel.

3.26 Start the Server

Run:

```
Bash python manage.py
runserver
```

Then open:

```
http://127.0.0.1:8000/admin/
```

Login using your superuser. You

should see:

```
BOX SELECTOR
Products
Boxes
Orders
Order items
```

 Now your database models are connected to Django Admin.

3.27 Add Test Data

Use the admin panel to create some products.

Product 1

```
Name: Laptop
Length: 30
Width: 20
Height: 5
Weight: 2
```

Product 2

```
Name: Mouse
Length: 10
Width: 6
Height: 4
Weight: 0.2
```

Product 3

```
Name: Keyboard
Length: 40
Width: 15
Height: 5
Weight: 1
```

Add Boxes

Small Box

```
Name: Small Box
Length: 20
Width: 20
Height: 10
Max Weight: 2
Cost: 10
```

Medium Box

```
Name: Medium Box
Length: 35
Width: 25
Height: 10
Max Weight: 5
Cost: 20
```

Large Box

```
Name: Large Box
Length: 50
Width: 40
Height: 20
Max Weight: 10
Cost: 40
```

3.28 Understand Our Database

After adding data, conceptually it looks like:

PRODUCT

ID	Name	L	W	H	Weight
1	Laptop	30	20	5	2
2	Mouse	10	6	4	0.2
3	Keyboard	40	15	5	1

And:

BOX

ID	Name	L	W	H	MaxWeight	Cost
1	Small	20	20	10	2	10
2	Medium	35	25	10	5	20
3	Large	50	40	20	10	40

And eventually:

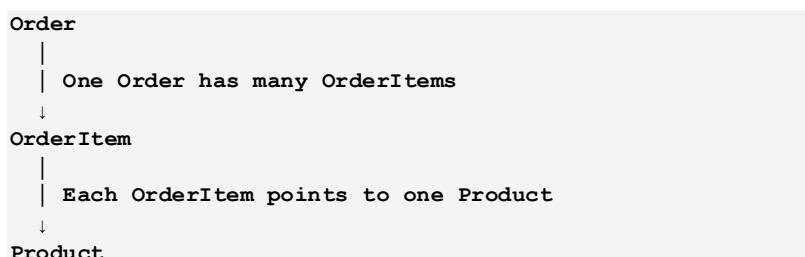
ORDER

```
ID
1
```

ORDER ITEM			
ID	Order	Product	Qty
1	1	Laptop	1
2	1	Mouse	2

3.29 Very Important: Database Relationship

Remember this for your interview:



In database terminology:

```

Order → OrderItem
One-to-Many
Product → OrderItem
One-to-Many

```

Therefore, `order` and `Product` are effectively connected through `OrderItem`.

3.30 Interview Questions From Step 3

Q1. Why did you use four models?

Answer:

"I used `Product` and `Box` for their own data, `Order` for the customer order, and `OrderItem` to connect products with orders and store quantity."

Q2. Why use `DecimalField`?

"Dimensions, weight and cost can contain decimal values, so `DecimalField` gives us accurate decimal storage."

Q3. What is a `ForeignKey`?

"A `ForeignKey` creates a relationship between two database models. In my project, `OrderItem` connects an order with a product."

Q4. Why `OrderItem`?

"Because an order can contain multiple products and different quantities. OrderItem stores the product and its quantity for a particular order."

Q5. What is migration?

"A migration is Django's way of tracking and applying database schema changes based on model changes."

Q6. Difference between makemigrations and migrate?

Very important:

```
makemigrations
  ↓
Creates migration files

migrate
  ↓
Applies migrations to database
```

Answer:

"makemigrations creates migration files from model changes, while migrate applies those migrations to the database."

3.31 Step 3 Checklist

Before moving forward, make sure you have:

- ☒ Product model
- ☒ Box model
- ☒ Order model
- ☒ OrderItem model
- ☒ ForeignKey relationships
- ☒ migrations created
- ☒ migrations applied
- ☒ SQLite database created
- ☒ Superuser created
- ☒ Models registered in admin
- ☒ Products added
- ☒ Boxes added

One important correction to keep in mind

At this point we have created the **data structure**, but we have **not yet solved the actual box-selection problem**.

That's intentional.

Our architecture is currently:





The next major step is where the project becomes interesting:

Step 4 — Build the Box Recommendation Algorithm

We'll take an order like:

```

Laptop × 1
Mouse × 2
  
```

and write the Python/Django **business logic** that:

```

Order
↓
Read products
↓
Calculate requirements
↓
Check box dimensions
↓
Check maximum weight
↓
Find valid boxes
↓
Compare costs
↓
Return recommended box
  
```

This algorithm will be the **main thing you should understand for the interview**, so we'll build it slowly and explain the Python logic line by line.



step 4

Step 4 — Build the Box Recommendation Algorithm

Now we are at the **core logic of the project**.

The assignment says the system should recommend the most suitable shipping box for an order based on product dimensions/weight and box dimensions, weight capacity, and cost.

Assignment - Python_Django Deve...

Our goal is to build this flow:

```

Order
↓
Get products
↓
Check dimensions
↓
Check total weight
↓
  
```

```
Find suitable boxes
↓
Compare cost
↓
Return recommended box
```

4.1 First understand the algorithm

Let's take a simple example.

Product

```
Laptop
Length = 30 cm
Width  = 20 cm
Height = 5 cm
Weight = 2 kg
```

Available boxes

```
Small
20 × 20 × 10
Max weight = 2 kg
Cost = ₹10

Medium
35 × 25 × 10
Max weight = 5 kg
Cost = ₹20

Large
50 × 40 × 20
Max weight = 10 kg
Cost = ₹40
```

Now check each box. **Small**

```
30 <= 20 ✗
```

Doesn't fit.

Medium

```
30 <= 35 ✓
20 <= 25 ✓
5  <= 10 ✓
2  <= 5  ✓
```

Fits.

Large

```
30 <= 50 ✓
20 <= 40 ✓
5  <= 20 ✓
2  <= 10 ✓
```

Fits.

Suitable boxes:

```
Medium → ₹20
Large  → ₹40
```

Therefore:

```
Recommended → Medium
```

because our defined rule is:

Choose the lowest-cost box among the boxes that satisfy the requirements.

Remember: this cost-selection rule is **our implementation assumption**. The assignment provides box cost but does not explicitly define the exact optimization rule.

Assignment - Python_Django Deve...

4.2 Where should this logic go?

We should **not** put all this logic inside `views.py`.

Instead, create:

```
box_selector/
├── models.py
├── views.py
├── tests.py
└── services.py ← business logic
```

Why?

Because `services.py` will contain the actual recommendation logic. This gives

us:

```
View
↓
Service
↓
Models
↓
Database
```

This separation makes the algorithm easier to test.

4.3 Create `services.py`

Inside:

```
box_selector/
```


create:

```
services.py
```

For now, we will create a simple function:

```
Python def
recommend_box(order):
pass
```

The idea is:

```
recommend_box(order)
↓
Box
```

or if nothing fits:

```
recommend_box(order)
↓
None
```

4.4 First get all products in the order

Remember that we created:

```
Python
related_name="items"
```

in `OrderItem`. Therefore:

```
Python
order.items.all()
```

gives us all products in the order. For

example:

```
Order #1
|
├─ Laptop × 1
├─ Mouse × 2
└─ Keyboard × 1
```

Then:

```
Python
order.items.all()
```

returns the three `OrderItem` objects.

4.5 Calculate total weight

Suppose:

```
Laptop × 1
Weight = 2 kg

Mouse × 2
Weight = 0.2 kg each
```

Total:

```
2 + (0.2 × 2)
= 2.4 kg
```

In Python:

```
Python total_weight = sum(
    item.product.weight * item.quantity    for item
    in order.items.all()
)
```

Let's understand this carefully.

`item.product.weight`

Gets the product's weight. Example:

```
Laptop → 2 kg
```

`item.quantity`

Gets quantity. Example:

```
Laptop × 2
quantity = 2
```

Therefore:

```
Python
item.product.weight * item.quantity
```

means:

```
weight × quantity
```

4.6 Why do we need total weight?

Because the box has:

```
max_weight
```

Example:

```
Order weight = 6 kg
```

```
Box capacity = 5 kg
```

Then:

```
6 <= 5
```

is false.

Therefore:

```
✗ Box cannot be used
```

4.7 Check dimensions

Now we need to check whether the order's products can fit within the box dimensions.

For our simplified assignment implementation, we'll use the **largest required dimension in each direction**.

For example:

```
Product A:
30 x 20 x 5

Product B:
10 x 15 x 4
```

Required dimensions become:

```
Length = max(30, 10) = 30
Width  = max(20, 15) = 20
Height = max(5, 4) = 5
```

So we need approximately:

```
30 x 20 x 5
```

This is a **simplified packing assumption**, not a full 3D bin-packing solution.

That's important because the assignment does not specify a detailed multi-item packing algorithm.

4.8 Calculate required dimensions

We can do:

```
Python
items = order.items.all()

required_length = max(
    item.product.length for item in items
)

required_width = max(
    item.product.width for item in items
)

required_height = max(
```

```
item.product.height for item in items )
```

Now suppose the order contains:

```
Laptop:
30 x 20 x 5

Mouse:
10 x 6 x 4
```

Then:

```
required_length = 30
required_width  = 20
required_height = 5
```

4.9 Check whether a box fits

Now we can check:

```
Python
box.length >=
required_length box.width
>= required_width
box.height >=
required_height
```

And:

```
Python box.max_weight >=
total_weight
```

So a box is suitable when:

```
Length sufficient
AND Width
sufficient
AND Height
sufficient
AND
Weight capacity sufficient
```

4.10 Write the first complete version

Put this in:

```
box_selector/services.py
```

Python

```
from .models import Box

def recommend_box(order):
    items = order.items.all()

    if not items.exists():
        return None

    total_weight = sum(
        item.product.weight * item.quantity
        for item in items
    )

    required_length = max(
        item.product.length
        for item in items
    )

    required_width = max(
        item.product.width
        for item in items
    )

    required_height = max(
        item.product.height
        for item in items
    )

    suitable_boxes = Box.objects.filter(
        length__gte=required_length,
        width__gte=required_width,
        height__gte=required_height,
        max_weight__gte=total_weight,
    ).order_by("cost")
    return suitable_boxes.first()
```

Now let's understand what this code does.

4.11 Line-by-line explanation

Import Box

```
Python from .models
import Box
```

We need the `Box` model because we need to search the available boxes.

Get order items

```
Python items =
order.items.all()
```

This gets all products in the order.

Empty order

```
Python if not
items.exists():
    return None
```

If there are no products:

```
No items
  ↓
No box recommendation
  ↓
None
```

This is good edge-case handling.

4.12 Total weight

```
Python total_weight = sum(
    item.product.weight * item.quantity
    for item in items
)
```

Example:

```
Laptop 2 kg × 1
= 2 kg

Mouse
0.2 kg × 2 = 0.4 kg
```

Total:

```
2 + 0.4 = 2.4 kg
```

4.13 Required dimensions

We use:

```
Python
max(...)
```

because we need the largest dimension requirement in our simplified approach. Example:

```
Laptop = 30 cm
Mouse = 10 cm
```

Then:

```
Python
max(30, 10)
```

returns:

```
30
```

So:

```
Python
required_length = 30
```

The same idea applies to width and height.

4.14 Django QuerySet filtering

This part is very important:

```
Python
suitable_boxes = Box.objects.filter(
    length__gte=required_length,
    width__gte=required_width,
    height__gte=required_height,
    max_weight__gte=total_weight,
)
```

Django's:

```
__gte
```

means: **greater than or equal to**

So:

```
Python
length__gte=30
```

means:

```
box.length >= 30
```

Similarly:

```
width >= required_width height
>= required_height max_weight
>= total_weight
```

4.15 Why use Django ORM?

Instead of writing SQL manually like:

```
SQL
SELECT *
FROM box
WHERE length >= ...
```

we can use Django ORM:

```
Python
Box.objects.filter(...)
```

This is one of Django's major features.

Interview answer

If they ask:

"What is Django ORM?" Say:

"Django ORM allows me to work with database records using Python objects and query methods instead of writing SQL for every database operation."

4.16 Sort boxes by cost

We have:

```
Python
.order_by("cost")
```

Suppose suitable boxes are:

```
Large    ₹40
Medium   ₹20
Small    ₹10
```

After:

```
Python
.order_by("cost")
```

they become:

```
Small    ₹10
Medium   ₹20
Large    ₹40
```

4.17 Get the first box

Finally:

```
Python return
suitable_boxes.first()
```

The first box is the cheapest suitable box. So:

```
Suitable boxes
↓
Sort by cost
↓
First box
↓
Recommendation
```

4.18 Example execution

Suppose:


```
Order:
Laptop x 1
```

Laptop:

```
30 x 20 x 5
2 kg
```

Boxes:

```
Small:
20 x 20 x 10
2 kg
₹10
```

```
Medium:
35 x 25 x 10
5 kg
₹20
```

```
Large:
50 x 40 x 20
10 kg
₹40
```

The algorithm checks:

```
Small
30 <= 20 ✗

Medium
30 <= 35 ✓
20 <= 25 ✓
5 <= 10 ✓
2 <= 5 ✓

Large
30 <= 50 ✓
20 <= 40 ✓
5 <= 20 ✓
2 <= 10 ✓
```

Suitable:

```
Medium ₹20
Large ₹40
```

Sort:

```
Medium ₹20
Large ₹40
```

Return:

```
Medium
```

4.19 What if no box fits?

Suppose:

```
Product:
100 x 100 x 100
20 kg
```

And every box is smaller. Then:

```
Python
suitable_boxes.first()
```

returns:

```
Python
None
```

So:

```
recommend_box(order)
↓
None
```

Later, our view can show:

```
"No suitable box found for this order."
```

4.20 Important limitation of our current algorithm

You should understand this very well before your interview. Our

simplified algorithm:

```
max product length
max product width
max product height
total weight
```

does **not** calculate real 3D packing. For

example:

```
Product A
30 x 20 x 5

Product B
30 x 20 x 5
```

Our algorithm may say:

```
Required:
30 x 20 x 5
```

But two products obviously require more physical space than one product if they cannot be stacked. A real warehouse system would need a more sophisticated packing algorithm.

Interview answer

"For this assignment I used a simplified packing assumption. I understand that real-world multi-item packing is a 3D bin-packing problem. A production implementation could use item rotation, volume constraints, stacking rules, and a proper packing algorithm."

This is a **much better answer than claiming your current algorithm is a perfect packing solution**.

4.21 Another limitation — product rotation

Our current check assumes the dimensions are compared in the same orientation. For example:

```
Product:
10 x 20 x 30

Box:
30 x 20 x 10
```

Our current check may reject it.

A future improvement could check possible orientations. You can mention:

"Dimension rotation is a future improvement."

4.22 The algorithm in simple English

Memorize this:

"First, I get all items from the order. Then I calculate the total weight and required dimensions. I query the boxes whose dimensions and maximum weight can handle the order. Finally, I sort the suitable boxes by cost and return the cheapest one. If there is no suitable box, I return None."

This is probably the **most important explanation for your interview**.

4.23 One more important thing: testing

We should **not immediately move to the frontend**.

First, we need to prove that this algorithm works. We'll create tests such as:

```
Test 1 → Product fits
Test 2 → Product too large
Test 3 → Product too heavy
Test 4 → Exact dimensions
Test 5 → Cheapest suitable box
Test 6 → No suitable box
Test 7 → Multiple products
Test 8 → Product quantity
Test 9 → Empty order
```

This is especially important because the assignment explicitly requires **test cases and test-run output**.

Assignment - Python_Django Deve...

Step 4 Checklist

At this point you should have:

- ☒ `services.py` created
- ☒ `recommend_box()` created
- ☒ `OrderItems` retrieved
- ☒ Total weight calculated ☒
- Required dimensions calculated
- ☒ Boxes filtered
- ☒ Suitable boxes sorted by cost
- ☒ Cheapest suitable box returned
- ☒ No-box case handled
- ☒ Algorithm limitations understood

Our project now looks like:



Next is **Step 5 — Build the API/View layer**. We'll connect `recommend_box()` to Django so a user can create an order and actually request a box recommendation. We'll also learn **views, URLs, GET vs POST, JSON responses, and how Django connects the browser/API to our business logic**.



step 5

Step 5 — Connect the Algorithm to Django Views & URLs

Now our project has the **database models** and the **box recommendation logic**. But there is a problem:

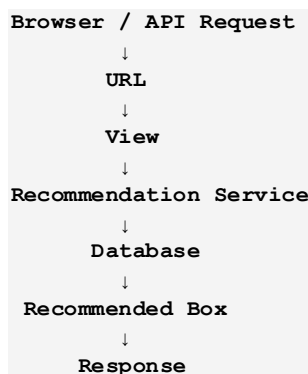
How will a user actually use our algorithm?

Right now we have:

```
recommend_box(order)
```

but there is no URL or webpage/API calling it. So Step

5 is about connecting everything:



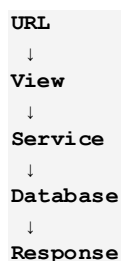
This is an important Django concept for your interview.

5.1 First understand Django's request flow

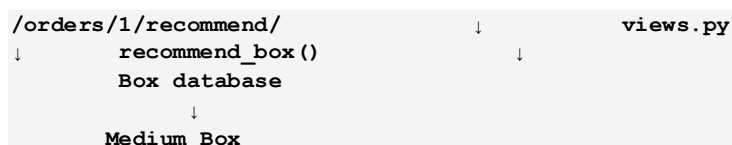
Imagine the user opens:

```
/orders/1/recommend/
```

Django receives the request. Then:



For our project:



5.2 What is a View?

A Django **view** handles a request and returns a response. Very simple example:

Python

```
def hello(request):    return
HttpResponse("Hello")
```

When someone visits the URL connected to this view, Django executes the function.

For our project, our view will:

1. Receive the order ID.
2. Find the order.
3. Call `recommend_box()`.
4. Return the result.

5.3 Create the first view

Open:

`box_selector/views.py`

Add:

Python

```
from django.http import JsonResponse from
django.shortcuts import get_object_or_404
```

```
from .models import Order from
.services import recommend_box
```

```
def recommend_box_view(request, order_id):
    order = get_object_or_404(Order, id=order_id)
    box = recommend_box(order)

    if box is None:
        return JsonResponse({
            "success": False,
            "message": "No suitable box found for this order."
        })

    return JsonResponse({
        "success": True,
        "order_id": order.id,
        "recommended_box": {
            "id": box.id,
            "name": box.name,
            "length": str(box.length),
            "width": str(box.width),
            "height": str(box.height),
            "max_weight": str(box.max_weight),
            "cost": str(box.cost),
        }
    })
```

Don't worry if this looks like a lot.

We'll understand it piece by piece.

5.4 Import JsonResponse

```
Python from django.http import
JsonResponse
```

This lets Django return JSON. For

example:

```
JSON
{
  "success": true,
  "recommended_box": {
    "name": "Medium Box",
    "cost": "20.00"
  }
}
```

JSON is useful when building APIs.

5.5 Import get_object_or_404

```
Python from django.shortcuts import
get_object_or_404
```

This is a Django helper.

Suppose the user requests:

```
/orders/100/recommend/
```

but Order 100 doesn't exist. Instead of

manually writing:

```
Python
order =
Order.objects.ge
t(id=100)

if not order:
...
```

we can use:

```
Python order = get_object_or_404(Order,
id=order_id)
```

If the order exists:

```
Order → returned
```

If it doesn't:

```
404 Not Found
```

This is clean Django code.

5.6 Import our service

```
Python from .services import  
recommend_box
```

This is extremely important.

Our view should **not contain the box-selection algorithm**. Instead:

```
views.py  
↓  
services.py
```

The view calls:

```
Python box =  
recommend_box(order)
```

This separation is good architecture.

5.7 What does `order_id` mean?

Our view is:

```
Python def recommend_box_view(request,  
order_id):
```

Suppose the URL is:

```
/orders/5/recommend/
```

Then:

```
order_id = 5
```

So Django gives the ID from the URL to the view.

5.8 Get the Order

```
Python order = get_object_or_404(Order,  
id=order_id)
```

Suppose:

```
order_id = 5
```

Django searches:


```
Order.objects.get(id=5)
```

If found:

```
Order #5
```

If not:

```
404
```

5.9 Call our algorithm Now:

```
Python box =  
recommend_box(order)
```

This is where Step 4 connects to Step 5.

Remember:

```
recommend_box()
```

does:

```
Get OrderItems  
↓  
Calculate weight  
↓  
Calculate dimensions  
↓  
Find suitable boxes  
↓  
Sort by cost  
↓  
Return cheapest box
```

So our view doesn't need to know the internal algorithm.

5.10 Handle no suitable box

We wrote:

```
Python if box  
is None:
```

If nothing fits, our service returns:

```
Python  
None
```

Then we return:

```
JSON
```

```
{
  "success": false,
  "message": "No suitable box found for this order." }
```

This is much better than allowing the application to crash.

5.11 Successful response

If a box is found:

```
Python return
JsonResponse({
  "success": True,
  ...
})
```

For example:

```
JSON
{
  "success": true,
  "order_id": 1,
  "recommended_box": {
    "id": 2,
    "name": "Medium Box",
    "length": "35.00",
    "width": "25.00",
    "height": "10.00",
    "max_weight": "5.00",
    "cost": "20.00"
  }
}
```

This gives the frontend/API client all the information it needs.

5.12 Why convert Decimal to string?

Our model uses:

```
Python
models.DecimalField(...)
```

Django's `DecimalField` gives us Python `Decimal` values. For a simple JSON response, we convert them:

```
Python
str(box.cost)
```

So instead of worrying about JSON serialization of `Decimal`, our response contains:

```
JSON
"cost": "20.00"
```

This is a small but useful implementation detail.

5.13 Now create URLs

We need to tell Django:

When someone visits this URL, execute `recommend_box_view`.

Create:

```
box_selector/urls.py
```

Add:

```
Python
from django.urls import path from
.views import recommend_box_view

urlpatterns = [
    path(
        "orders/<int:order_id>/recommend/",
        recommend_box_view,      name="recommend-box",
    ),
]
```

5.14 Understand <int:order_id>

This is important.

```
Python
<int:order_id>
```

means:

Take an integer from the URL and store it in a variable called `order_id`.

For example:

```
/orders/5/recommend/
```

gives:

```
order_id = 5
```

But:

```
/orders/abc/recommend/
```

doesn't match this URL because `abc` isn't an integer.

5.15 Connect App URLs to Project URLs

Open:

```
config/urls.py
```

You'll probably have:

```
Python
from django.contrib
import admin from
django.urls import path
```

Change it to:

```
Python
from django.contrib import admin from
django.urls import include, path

urlpatterns = [
    path("admin/", admin.site.urls),
    path("", include("box_selector.urls")), ]
```

5.16 What does include() do?

This:

```
Python
path("", include("box_selector.urls"))
```

means:

"Django, look inside `box_selector/urls.py` for the application's URLs." So now:

```
config/urls.py
↓
box_selector/urls.py
↓
views.py
```

The complete flow is:

```
Browser ↓
config/urls.py
↓
box_selector/urls.py
↓
recommend_box_view()

↓
recommend_box()
```

5.17 Test the API

First make sure you have at least one order. You can create an order from:

```
http://127.0.0.1:8000/admin/
```

Suppose the order ID is:

1

Then open:

```
http://127.0.0.1:8000/orders/1/recommend/
```

You should get JSON. For

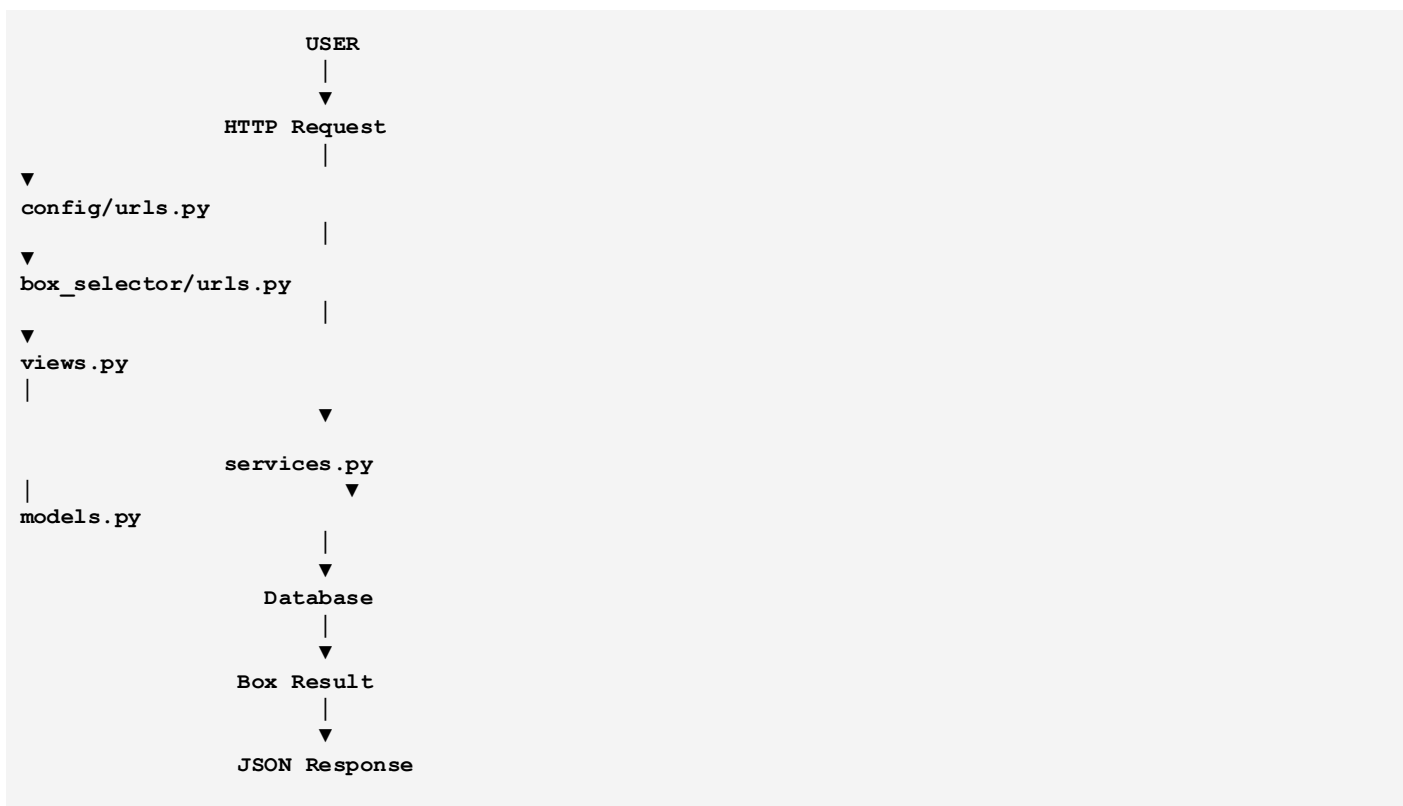
example:

```
JSON
{
  "success": true,
  "order_id": 1,
  "recommended_box": {
    "id": 2,
    "name": "Medium Box",
    "length": "35.00",
    "width": "25.00",
    "height": "10.00",
    "max_weight": "5.00",
    "cost": "20.00"
  }
}
```

Now our algorithm is actually accessible through Django.

5.18 Understand the complete architecture

We now have:



This is a very important diagram for your interview.

5.19 Why don't we put the algorithm in `views.py`?

Imagine we put 100 lines of box-selection code directly inside:

```
Python
recommend_box_view()
```

Then our view becomes responsible for:

```
HTTP request
+
database
+
calculations
+
business rules
+
response
```

That's difficult to maintain. Instead:

```
View
↓
Service
↓
Models
```

Each part has a clear responsibility.

Interview answer

"I keep the business logic in a service layer instead of putting it directly in the view. This makes the code easier to test, maintain and reuse."

5.20 What is an API?

Our endpoint:

```
/orders/1/recommend/
```

returns JSON.

This is essentially an API endpoint.

An API allows another application/frontend to communicate with our backend. For example:

```
Frontend
|
| GET /orders/1/recommend/
▼
Django
|
▼
JSON
```

5.21 GET vs POST

This is another common interview question.

GET

Usually used to **retrieve information**. Example:

```
GET /orders/1/recommend/
```

We're asking:

"Give me the recommendation for order 1." So GET

is reasonable here.

POST

Usually used to **create something**. For

example:

```
POST /orders/
```

could create a new order. You

should remember:

```
GET → retrieve  
POST → create  
PUT/PATCH → update  
DELETE → delete
```

5.22 Our current API

Currently we have:

```
GET  
/orders/<order_id>/recommend/
```

Purpose:

Get the recommended box for an existing order.

Example:

```
GET /orders/1/recommend/
```

5.23 Error cases

We should consider:

Order doesn't exist

Request:

```
/orders/999/recommend/
```

If Order 999 doesn't exist:

404 Not Found

Good.

Order has no items

Our service returns:

```
Python  
None
```

So response:

```
JSON  
{  
  "success": false,  
  "message": "No suitable box found for this order."  
}
```

Good.

No box fits

Same response:

```
JSON  
{  
  "success": false,  
  "message": "No suitable box found for this order." }  
}
```

Good.

5.24 One improvement we should make

At this stage, the response is technically working, but we haven't created the **user-friendly interface** yet.

Right now the user needs to manually open:

```
/orders/1/recommend/
```

That's not a good final application. Eventually we want:

```
BOX SELECTION SYSTEM  
  
Order #1  
Laptop x 1  
Mouse x 2
```



```
[ Recommend Box ]
```

Clicking:

```
[ Recommend Box ]
```

should show:

```
Recommended Box: Medium Box
Dimensions: 35 x 25 x 10 cm Maximum Weight: 5 kg
Cost: ₹20
```

We'll build that frontend later.

5.25 Step 5 Interview Questions

Q1. What is a Django view?

"A view handles an HTTP request and returns an HTTP response." **Q2. Why**

use a service layer?

"To keep business logic separate from HTTP and presentation logic."

Q3. What is URL routing?

"URL routing maps a URL pattern to a Django view."

Q4. What does `get_object_or_404()` do?

"It retrieves an object if it exists, otherwise it automatically returns a 404 response."

Q5. What is `JsonResponse`?

"It allows a Django view to return data in JSON format."

Q6. Why use GET here?

"Because the recommendation endpoint is retrieving information rather than creating or modifying an order."

5.26 Step 5 Checklist ☒

You should now have:

```
☒ views.py updated
☒ recommend_box_view() created
☒ services.py connected to view
```

```

❑ box_selector/urls.py created
❑ Project URLs connected using include() ❑ GET
  recommendation endpoint created
❑ JSON response created
❑ 404 handling added
❑ No-box handling added

```

Your project architecture is now:

```

box-selection-system/
├── config/
│   ├── settings.py
│   └── urls.py
├── box_selector/
│   ├── models.py           ← Database
│   ├── services.py        ← Box algorithm
│   ├── views.py           ← Request/response
│   ├── urls.py            ← Routes
│   ├── admin.py
│   └── tests.py
├── manage.py
├── db.sqlite3
└── requirements.txt

```

One important thing before Step 6

We now have enough backend logic to start testing it properly. **Don't skip testing.** The assignment explicitly requires test cases and test-run output.

Assignment - Python_Django Deve...

Next: Step 6 — Automated Testing

We'll create Django tests for:

```

Product fits
Product too large
Product too heavy
Exact dimensions
Cheapest box selected
No suitable box
Multiple products
Product quantity
Empty order

```

And I'll show you **how to write each test**, what `TestCase` means, `setUp()`, assertions, and how to read the test output.



step 6

Step 6 — Automated Testing

Now we need to **prove that our box recommendation logic works correctly**.

This is especially important because the assignment explicitly asks you to provide **test cases** and **testrun output**.

Assignment - Python_Django Deve...

For your interview, don't just say:

Printed using [ChatGPT to PDF](#), powered by PDFCrowd [HTML to PDF API](#).

"I tested my project."

You should be able to explain **what you tested and why**.

6.1 What are we testing?

Our main function is:

```
Python
recommend_box(order)
```

It should correctly handle:

```
Order
↓
Products
↓
Dimensions + Weight
↓
Suitable boxes
↓
Cheapest suitable box
```

So we'll test the business logic first.

6.2 Test cases we need

We'll create these tests:

Product fits in one box	Correct box returned
Product too large	No box
Product too heavy	No box
Exact dimensions	Box accepted
Multiple suitable boxes	Cheapest selected
No suitable box	None
Multiple products	Correct recommendation
Product quantity	Weight considers quantity
Empty order	None

These tests cover both normal and edge cases.

6.3 Where do we write tests?

Open:

```
box_selector/tests.py
```

Django already created this file when we ran:

```
Bash python manage.py startapp
box_selector
```

We'll use Django's built-in testing framework.

6.4 What is TestCase?

We'll use:

```
Python
from django.test import TestCase
```

`TestCase` provides a test environment for Django. It lets

us:

- create database records
- run our functions check
- results • automatically

clean up test data You can

explain:

"I use Django's `TestCase` to test my models and business logic in an isolated test database."

6.5 Import our models and service

At the top of `tests.py`:

```
Python
from django.test import TestCase

from .models import Box, Order, OrderItem, Product from
.services import recommend_box
```

Now we can use:

```
Product
Box
Order
OrderItem recommend_box()
```

inside our tests.

6.6 Create common test data

We'll use `setUp()`.

```
Python class
BoxRecommendationTests(TestCase):

    def setUp(self):
        self.small_box = Box.objects.create(
            name="Small Box",
            length=20,
            width=20,
            height=10,
            max_weight=2,
            cost=10,
        )

        self.medium_box = Box.objects.create(
            name="Medium Box",
            length=35,
            width=25,
            height=10,
            max_weight=5,
            cost=20,
        )

        self.large_box = Box.objects.create(
            name="Large Box",
            length=50,
            width=40,
            height=20,
            max_weight=10,
            cost=40,
        )

        self.laptop = Product.objects.create(
            name="Laptop",
            length=30,
            width=20,
            height=5,
            weight=2,
        )
```

6.7 What is `setUp()` ?

This is an important interview question.

`setUp()` runs **before each test method**. So

instead of writing:

```
Python
Box.objects.create(...)
```

again and again in every test, we create common data once in `setUp()`. Conceptually:

```
Test 1  ↓ setUp() ↓
Test

Test 2  ↓ setUp() ↓
Test

Test 3  ↓ setUp() ↓
Test
```

Each test gets a clean test database.

6.8 Test 1 — Product Fits

Let's create our first test.

```
Python def
test_product_fits_in_medium_box(self):
    order = Order.objects.create()

    OrderItem.objects.create(
        order=order,
        product=self.laptop,          quantity=1,
    )    result =
    recommend_box(order)

    self.assertEqual(result, self.medium_box)
```

6.9 Understand this test

We create:

```
Laptop
30 x 20 x 5
2 kg
```

Then:

```
Small
20 x 20 x 10
```

doesn't fit.

Medium:

```
35 x 25 x 10
5 kg
```

fits.

Large also fits.

But Medium is cheaper. Therefore:

```
Expected → Medium Box
```

And:

```
Python
self.assertEqual(result,
self.medium_box)
```

checks that.

6.10 What is `assertEqual()`?

This:

```
Python
self.assertEqual(actual, expected)
```

checks:

Are these two values equal?

Our test:

```
Python
self.assertEqual(result,
self.medium_box)
```

means:

"Did the recommendation return the Medium Box?"

If yes:

```
PASS
```

If no:

```
FAIL
```

6.11 Test 2 — Product Too Large

Now test a product that cannot fit. Add:

```
Python def
test_product_too_large(self):
    product = Product.objects.create(
        name="Large Product",
        length=100,          width=100,
        height=100,          weight=2,
    )    order =

    Order.objects.create()

    OrderItem.objects.create(
        order=order,
        product=product,      quantity=1,
    )    result =
    recommend_box(order)
    self.assertIsNone(result)
```

The product is:

```
100 x 100 x 100
```

Our largest box is:

```
50 x 40 x 20
```

So no box fits. Expected:

```
None
```

6.12 What is `assertIsNone()` ?

```
Python
self.assertIsNone(result)
```

checks that:

```
Python result
is None
```

This is perfect for our:

"No suitable box found." case.

6.13 Test 3 — Product Too Heavy

Create:

```
Python def
test_product_too_heavy(self):
    product = Product.objects.create(
name="Heavy Product",
length=30,          width=20,
height=5,           weight=20,
    )    order =

Order.objects.create()

    OrderItem.objects.create(
order=order,
product=product,    quantity=1,
    )    result =
recommend_box(order)
self.assertIsNone(result)
```

Dimensions fit inside the Large box. But:

```
Product = 20 kg
Largest box capacity = 10 kg
```

Therefore:

```
No suitable box
```


6.14 Test 4 — Exact Dimensions

This is an important edge case. Create:

```
Python def
test_exact_dimensions_are_allowed(self):
    product = Product.objects.create(
        name="Exact Product",          length=35,
        width=25,          height=10,
        weight=5,
    )    order =

    Order.objects.create(

        OrderItem.objects.create(
            order=order,
            product=product,          quantity=1,
        )    result = recommend_box(order)
    self.assertEqual(result, self.medium_box)
```

Why Medium? Because:

```
Product:
35 x 25 x 10
5 kg

Medium:
35 x 25 x 10
5 kg
```

Our comparison uses:

```
>=
```

so equality is allowed.

6.15 Test 5 — Cheapest Suitable Box

This test checks our main business rule. We

already have:

```
Medium → ₹20
Large  → ₹40
```

Both can fit the laptop. So:

```
Expected → Medium
```

Our first test already checks this, but we can make the purpose explicit:

```
Python def
test_cheapest_suitable_box_is_selected(self):
    order = Order.objects.create()

    OrderItem.objects.create(
        order=order,
        product=self.laptop,          quantity=1,
    )    result =

    recommend_box(order)

    self.assertEqual(result.cost, 20)
    self.assertEqual(result.name, "Medium Box")
```

This checks the actual business requirement we've defined.

6.16 Test 6 — Empty Order

An order may exist but contain no products. Our service contains:

```
Python if not
items.exists():
    return None
```

So we should test it.

```
Python def
test_empty_order_returns_none(self):
    order = Order.objects.create()    result
= recommend_box(order)
    self.assertIsNone(result)
```

Expected:

```
No items
↓ None
```

6.17 Test 7 — Product Quantity

This is important. Suppose:

```
Mouse weight = 0.2 kg
Quantity = 10
```

Total weight:

```
0.2 × 10 = 2 kg
```

Let's test quantity.

```
Python def
test_quantity_is_considered_for_weight(self):
    product = Product.objects.create(
        name="Mouse",          length=10,          width=6,
        height=4,              weight=0.2,
    )    order =

    Order.objects.create()

    OrderItem.objects.create(
        order=order,          product=product,
        quantity=10,
    )    result = recommend_box(order)
    self.assertEqual(result, self.small_box)
```

Why Small?

Total weight:

```
0.2 x 10 = 2 kg
```

Small capacity:

```
2 kg
```

Dimensions:

```
10 x 6 x 4
```

fit inside Small. So:

```
Small → valid
Medium → valid
Large → valid
```

Cheapest:

```
Small → ₹10
```

Therefore:

```
Small Box
```

6.18 Test 8 — Multiple Products

Now let's test an order containing multiple products.

Create another product:

```
Python def
test_multiple_products(self):
    mouse = Product.objects.create(
        name="Mouse",          length=10,
        width=6,               height=4,
        weight=0.2,
    )    order =

    Order.objects.create()

        OrderItem.objects.create(
            order=order,
            product=self.laptop,          quantity=1,
        )

        OrderItem.objects.create(
            order=order,
            product=mouse,               quantity=2,
        )    result = recommend_box(order)
    self.assertEqual(result, self.medium_box)
```

Total weight:

```
Laptop = 2 kg
Mouse  = 0.2 × 2 = 0.4 kg

Total = 2.4 kg
```

Required dimensions under our simplified approach:

```
Length = max(30, 10) = 30
Width  = max(20, 6)  = 20
Height = max(5, 4)   = 5
```

Medium:

```
35 × 25 × 10
5 kg
```

fits.

Small fails because:

```
30 > 20
```

Therefore:

```
Medium
```

6.19 Complete tests.py

At this stage, your test file can look like this:

Python

```
from django.test import TestCase

from .models import Box, Order, OrderItem, Product from
.services import recommend_box

class BoxRecommendationTests(TestCase):

    def setUp(self):
        self.small_box = Box.objects.create(
            name="Small Box", length=20,
            width=20, height=10,
            max_weight=2, cost=10,
        )

        self.medium_box = Box.objects.create(
            name="Medium Box", length=35,
            width=25, height=10,
            max_weight=5, cost=20,
        )

        self.large_box = Box.objects.create(
            name="Large Box", length=50,
            width=40, height=20,
            max_weight=10, cost=40,
        )

        self.laptop = Product.objects.create(
            name="Laptop", length=30,
            width=20, height=5,
            weight=2,
        )

    def test_product_fits_in_medium_box(self):
        order = Order.objects.create()

        OrderItem.objects.create(
            order=order, product=self.laptop,
            quantity=1,
        )
        result = recommend_box(order)

        self.assertEqual(result, self.medium_box)

    def test_product_too_large(self):
        product = Product.objects.create(
            name="Large Product",
            length=100, width=100,
            height=100, weight=2,
        )
        order =
        Order.objects.create()

        OrderItem.objects.create(
            order=order,
            product=product,
            quantity=1,
```

```

        )
        result =

recommend_box(order)

self.assertIsNone(result)

    def test_product_too_heavy(self):
        product = Product.objects.create(
name="Heavy Product",
length=30,
width=20,
height=5,
weight=20,
)
        order =

Order.objects.create()

        OrderItem.objects.create(
order=order,
product=product,
quantity=1,
)
        result =

recommend_box(order)

self.assertIsNone(result)

    def test_exact_dimensions_are_allowed(self):
        product = Product.objects.create(
name="Exact Product",
length=35,
width=25,
height=10,
weight=5,
)
        order =

Order.objects.create()

        OrderItem.objects.create(
order=order,
product=product,
quantity=1,
)
        result = recommend_box(order)

self.assertEqual(result, self.medium_box)

    def test_cheapest_suitable_box_is_selected(self):
order = Order.objects.create()

        OrderItem.objects.create(
order=order,
product=self.laptop,
quantity=1,
)
        result =

recommend_box(order)

        self.assertEqual(result.cost, 20)
self.assertEqual(result.name, "Medium Box")

    def test_empty_order_returns_none(self):
order = Order.objects.create()
        result

= recommend_box(order)

self.assertIsNone(result)

    def test_quantity_is_considered_for_weight(self):
        product = Product.objects.create(
name="Mouse",
length=10,
width=6,
height=4,

```

```

        weight=0.2,
    )
    order =

Order.objects.create()

    OrderItem.objects.create(
order=order,
    product=product,
quantity=10,
    )
    result = recommend_box(order)

self.assertEqual(result, self.small_box)

    def test_multiple_products(self):
mouse = Product.objects.create(
name="Mouse",
    length=10,
width=6,
    height=4,
weight=0.2,
    )
    order =

Order.objects.create()

    OrderItem.objects.create(
order=order,
    product=self.laptop,
quantity=1,
    )

    OrderItem.objects.create(
order=order,
product=mouse,
    quantity=2,
    )
    result = recommend_box(order)
self.assertEqual(result, self.medium_box)

```

6.20 Run the Tests

Now run:

```

Bash python manage.py
test

```

You should get something similar to:

```

Found 8 test(s).

.....
-----
-----
Ran 8 tests in 0.xxxx

OK

```

The exact timing and formatting may differ. The important part is:

```

Ran 8 tests

OK

```

6.21 What does `.` mean?

When Django runs tests:

```
.....
```

Each:

```
.
```

usually means one test passed. For example:

```
.....
```

means:

```
8 tests passed
```

If you see:

```
F
```

that means a test failed. If you see:

```
E
```

that means an error occurred while running a test.

6.22 What if a test fails?

Suppose you see:

```
.F.....
```

The `F` means one test failed.

Django will give you information such as:

```
FAIL: test_product_too_large
```

and explain:

```
Expected: None
Actual: <Box ...>
```

You then investigate the problem.

Don't simply remove the failing test.

Find out why it failed.

This is exactly the kind of thing you should be able to explain in an interview.

6.23 Why testing is important

Without tests:

```
Code
↓
"It looks correct"
```

With tests:

```
Code
↓
Automated tests
↓
PASS / FAIL
```

If later you change:

```
Python
recommend_box()
```

you can run:

```
Bash
python manage.py test
```

and make sure your old functionality still works.

6.24 Interview Question: Why use automated tests?

Good answer:

"Automated tests help me verify that the recommendation logic works correctly and prevent existing functionality from breaking when I change the code."

6.25 Interview Question: What did you test?

Don't just say:

"I tested the application." Say:

"I tested normal cases like a product fitting in a box, and edge cases like oversized products, excessive weight, exact dimensions, empty orders, product quantities, multiple products, and selecting the cheapest suitable box."

That's much stronger.

6.26 Interview Question: What is unit testing?

Simple answer:

"Unit testing means testing a small part of the application independently to make sure it behaves correctly."

In our project, an important unit is:

```
Python
recommend_box(order)
```

6.27 Important point about our tests

Our tests are currently focused mainly on the **recommendation service**. Later, we should also test our API/view:

```
GET /orders/1/recommend/
```

For example:

```
Valid order → 200
Invalid order → 404
No box → correct JSON response
```

We'll do that after our basic frontend/API structure is complete.

6.28 Test cases for your documentation

The assignment asks you to provide test cases.

Assignment - Python_Django Deve...

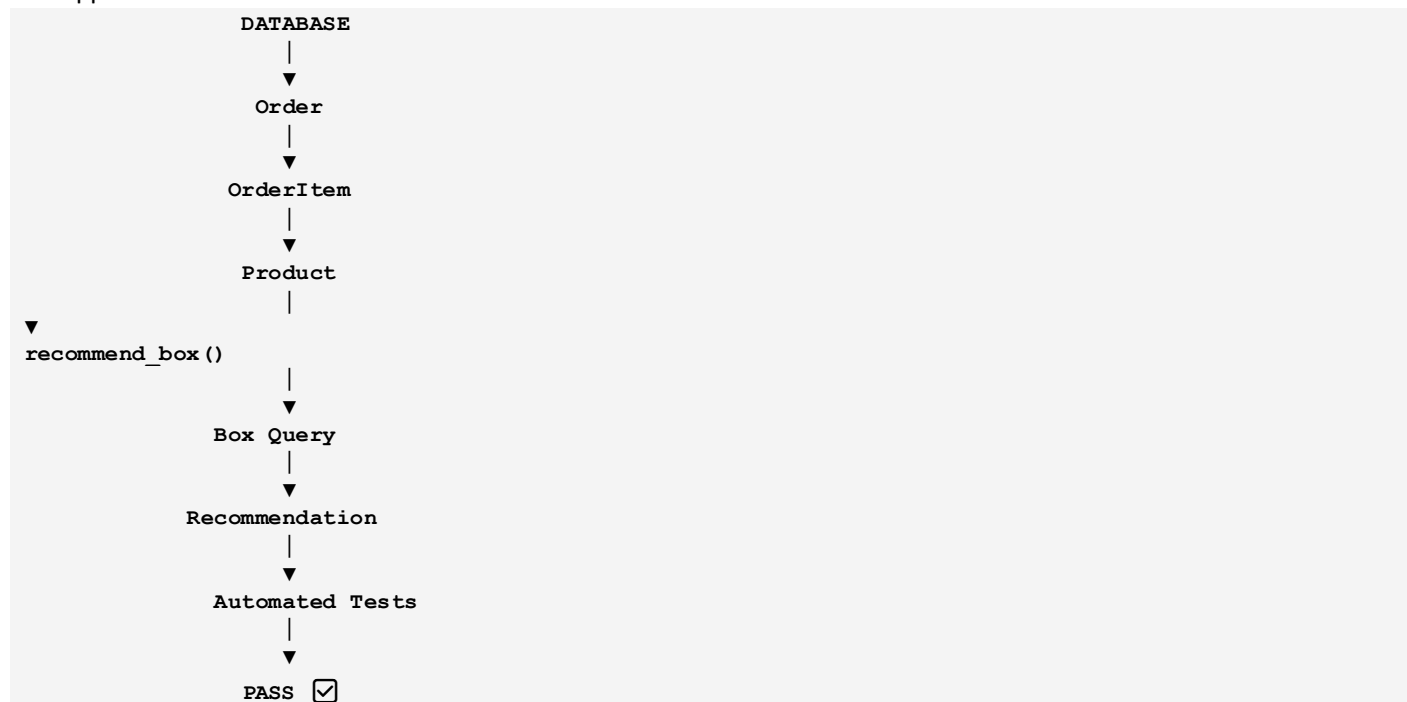
You can eventually document them in a table like:

TC01	Product fits	Correct box
TC02	Product too large	No box
TC03	Product too heavy	No box
TC04	Exact dimensions	Accepted
TC05	Multiple suitable boxes	Cheapest selected
TC06	Empty order	No box
TC07	Quantity considered	Correct weight
TC08	Multiple products	Correct recommendation

But remember: **your final test documentation should reflect the tests you actually implement.**

6.29 What we've achieved

Our application now has:



This is a solid backend foundation.

Step 6 Checklist ☒

Make sure:

- ☒ tests.py created
- ☒ TestCase used
- ☒ setUp() understood
- ☒ Product test created
- ☒ Box test created
- ☒ Oversized product tested
- ☒ Heavy product tested
- ☒ Exact dimensions tested
- ☒ Cheapest box tested
- ☒ Empty order tested
- ☒ Quantity tested
- ☒ Multiple products tested
- ☒ python manage.py test works

One thing I want you to remember for the interview

The strongest explanation is:

"I didn't only test the happy path. I also tested edge cases such as products that are too large, products that exceed the weight capacity, exact dimension matches, empty orders, quantities, multiple products, and cases where multiple boxes are suitable." That shows **developer thinking**, not just Django knowledge.

Next: Step 7 — Build the User Interface

We'll connect the backend to a simple professional Django UI where you can:



Then the project will start looking like a **real application rather than only an API/backend**.



step 7

Step 7 — Build the User Interface

Now we have the important backend parts:

```

Step 3 → Models / Database
Step 4 → Box Recommendation Logic
Step 5 → Django View + API
Step 6 → Automated Tests
  
```

Now we will make the project **easy for a warehouse user to use**.

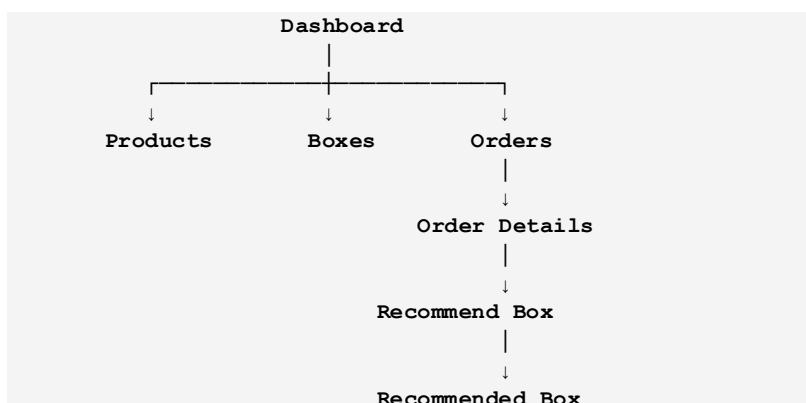
The assignment asks for a small Django-based system that recommends the suitable shipping box for an order.

Assignment - Python_Django Deve...

We will create a simple UI rather than making it overly complicated.

7.1 What should the user see?

Our final flow should look like:



The user should not need to understand Python or Django.

7.2 Pages we need

We'll create these pages:

1. Dashboard

BOX SELECTION SYSTEM

Products

Boxes

Orders

2. Product List

Products

Laptop	30 × 20 × 5	2 kg
Mouse	10 × 6 × 4	0.2 kg
Keyboard	40 × 15 × 5	1 kg

3. Box List

Boxes

Small Box

20 × 20 × 10

Max Weight: 2 kg

Cost: ₹10

Medium Box

35 × 25 × 10

Max Weight: 5 kg

Cost: ₹20

4. Order List

Orders

Order #1

Order #2

Order #3

5. Order Details

Order #1

Laptop × 1

Mouse × 2

[Recommend Box]

6. Recommendation Result

Recommended Box

Medium Box

35 × 25 × 10 cm

Maximum Weight: 5 kg

Cost: ₹20

7.3 First create the template folder

Our project currently looks approximately like:

```
box-selection-system/
├── box_selector/
├── config/
├── manage.py
├── db.sqlite3
└── requirements.txt
```

Create:

```
templates/
```

Then:

```
templates/ └──
box_selector/
```

So:

```
box-selection-system/
├── templates/
│   └── box_selector/
├── box_selector/
├── config/
└── manage.py
```

7.4 Create base.html

Inside:

```
templates/box_selector/
```

create:

```
base.html
```

Use:

```
HTML
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>{% block title %}Box Selection System{% endblock %}</title>
```

```

<style>
body {
    font-family: Arial, sans-serif;
    margin: 0;
    background: #f5f5f5;
}

nav {
    background: #222;
padding: 15px 30px;
}

nav a {
    color:
white;
    text-decoration:
none;
    margin-right:
20px;
}

.container {
width: 90%;
width: 1100px;
margin: 30px auto;
}

.card {
    background: white;
padding: 20px;
margin-
bottom: 20px;
border-
radius: 8px;
}

button,
.button {
    background: #222;
color: white;
padding:
10px 16px;
border:
none;
border-radius:
5px;
text-decoration:
none;
cursor: pointer;
}

table {
width: 100%;
border-collapse: collapse;
background: white;
}

th,
td {
    padding: 12px;
border-bottom: 1px solid #ddd;
text-align: left;
}

.success {
padding: 15px;
background: #e8f5e9;
border-radius: 5px;
}

.error {
padding: 15px;
background: #ffebee;
border-radius: 5px;
}
</style>
</head>

<body>

<nav>
    <a href="{% url 'home' %}">Box Selection</a>
    <a href="{% url 'product-list' %}">Products</a>
    <a href="{% url 'box-list' %}">Boxes</a>

```

```
<a href="{% url 'order-list' %}">Orders</a> </nav>

<div class="container">
{% block content %}
    {% endblock %}
</div>

</body>
</html>
```

7.5 Why use base.html?

Instead of copying:

```
HTML
<nav>...</nav>
```

into every page, we create one common template. Then other pages can use:

```
django
{% extends "box_selector/base.html" %}
```

This is called **template inheritance**. Interview answer:

"I use Django template inheritance so common elements like navigation and styling don't need to be duplicated across every page."

7.6 Configure Templates

Open:

```
config/settings.py
```

Find:

```
Python
TEMPLATES = [
```

You should see something similar to:

```
Python
'DIRS': [],
```

Change it to:

```
Python
'DIRS': [BASE_DIR / 'templates'],
```

So Django knows where our project-level templates are located.

7.7 Create Dashboard View

Open:

```
box_selector/views.py
```

Add:

```
Python from django.shortcuts import
render
```

Then:

```
Python def
home(request) :
    return render(request, "box_selector/home.html")
```

Now we have:

```
Browser
↓ home
URL      ↓
home()
↓
home.html
```

7.8 Create Dashboard Template

Create:

```
templates/box_selector/home.html
```

Add:

```
HTML
{% extends "box_selector/base.html" %}

{% block title %}
Dashboard
{% endblock %}

{% block content %}

<div class="card">
    <h1>Box Selection System</h1>

    <p>
        Select an order to find the most suitable shipping
        box.
    </p>
</div>

<div class="card">
    <h2>Quick Links</h2>
    <p>
        <a class="button" href="{% url 'product-list' %}">
            Products
        </a>

        <a class="button" href="{% url 'box-list' %}">
            Boxes
```

```

        </a>

        <a class="button" href="{% url 'order-list' %}">
            Orders
        </a>
    </p>
</div>

{% endblock %}

```

7.9 Add the Home URL

Open:

```
box_selector/urls.py
```

Change it to:

```

Python
from django.urls import path

from .views import (    home,
    recommend_box_view,
)

urlpatterns = [    path("", home, name="home"),

    path(
        "orders/<int:order_id>/recommend/",
        recommend_box_view,    name="recommend-box",
    ),
]

```

Now:

```
http://127.0.0.1:8000/
```

will display our dashboard.

7.10 Product List Page

Now let's display products.

In `views.py`:

```

Python
from .models import Box, Order, Product

```

Then add:

```
Python def product_list(request):
    products = Product.objects.all()

    return render(
        request,
        "box_selector/product_list.html",
        {"products": products},
    )
```

7.11 Create Product Template

Create:

```
templates/box_selector/product_list.html
```

Add:

```
HTML
{% extends "box_selector/base.html" %}

{% block title %}
Products
{% endblock %}

{% block content %}

<h1>Products</h1>

<table>
  <thead>
    <tr>
      <th>Name</th>
      <th>Dimensions</th>
      <th>Weight</th>
    </tr>
  </thead>
  <tbody>
    {% for product in products %}
    <tr>
      <td>{{ product.name }}</td>

      <td>
        {{ product.length }} x
        {{ product.width }} x
        {{ product.height }} cm
      </td>

      <td>
        {{ product.weight }} kg
      </td>
    </tr>

    {% empty %}
    <tr>
      <td colspan="3">No products found.
    </td>
    </tr>
    {% endfor %}
  </tbody>
</table>

{% endblock %}
```

7.12 Add Product URL

In `box_selector/urls.py`:

```
Python
path(
    "products/",
    product_list,
    name="product-list",
),
```

So:

```
/products/
```

calls:

```
Python
product_list()
```

7.13 Box List Page

Now do the same for boxes.

In `views.py`:

```
Python def box_list(request):    boxes =
Box.objects.all().order_by("cost")

    return render(
request,
    "box_selector/box_list.html",
    {"boxes": boxes},
    )
```

Notice:

```
Python
.order_by("cost")
```

This displays cheaper boxes first.

7.14 Create Box Template

Create:

```
templates/box_selector/box_list.html
```

Add:

```
HTML
{% extends "box_selector/base.html" %}

{% block title %}
Boxes
{% endblock %}

{% block content %}

<h1>Shipping Boxes</h1>

<table>
  <thead>
    <tr>
      <th>Name</th>
      <th>Dimensions</th>
      <th>Max Weight</th>
      <th>Cost</th>
    </tr>
  </thead>

  <tbody>
    {% for box in boxes %}
    <tr>
      <td>{{ box.name }}</td>

      <td>
        {{ box.length }} ×
        {{ box.width }} ×
        {{ box.height }} cm
      </td>

      <td>
        {{ box.max_weight }} kg
      </td>

      <td>
        ₹{{ box.cost }}
      </td>
    </tr>

    {% empty %}
    <tr>
      <td colspan="4">
No boxes found.
      </td>
    </tr>
    {% endfor %}
  </tbody>
</table>

{% endblock %}
```

7.15 Add Box URL

In `box_selector/urls.py`:

```
Python
path(
    "boxes/",
    box_list,
    name="box-list",
),
```

Now:

`http://127.0.0.1:8000/boxes/`
shows our boxes.

7.16 Order List

Now we need to show orders.

In `views.py`:

```
Python def order_list(request):    orders =
Order.objects.all().order_by("-created_at")

    return render(
request,
    "box_selector/order_list.html",
    {"orders": orders},
    )
```

The:

```
Python
.order_by("-created_at")
```

means newest orders appear first.

7.17 Create Order List Template

Create:

```
templates/box_selector/order_list.html
```

Add:

```
HTML
{% extends "box_selector/base.html" %}

{% block title %}
Orders
{% endblock %}

{% block content %}

<h1>Orders</h1>

<table>
  <thead>
    <tr>
      <th>Order</th>
      <th>Created</th>
      <th>Action</th>
    </tr>
  </thead>

  <tbody>
    {% for order in orders %}
    <tr>
      <td>
        Order #{{ order.id }}
      </td>
```

```

        <td>
            {{ order.created_at }}
        </td>

        <td>
            <a
                class="button"
                href="{% url 'order-detail' order.id %}"
            >
                View Order
            </a>
        </td>
    </tr>

    {% empty %}
    <tr>
        <td colspan="3">
            No orders found.
        </td>
    </tr>
    {% endfor %}
</tbody>
</table>

{% endblock %}

```

7.18 Order Detail View

Now we need to see what's inside an order.

In `views.py`:

```

Python def order_detail(request, order_id):
    order = get_object_or_404(Order, id=order_id)

    return render(
        request,
        "box_selector/order_detail.html",
        {"order": order},
    )

```

We already learned:

```
Python get_object_or_404()
```

in Step 5.

7.19 Order Detail Template

Create:

```
templates/box_selector/order_detail.html
```

Add:

```
HTML
```

```
{% extends "box_selector/base.html" %}

{% block title %}
Order #{{ order.id }}
{% endblock %}

{% block content %}

<div class="card">

    <h1>Order #{{ order.id }}</h1>

    <p>
        Created: {{ order.created_at }}
    </p>

</div>

<div class="card">

    <h2>Order Items</h2>

    <table>
        <thead>
            <tr>
                <th>Product</th>
                <th>Quantity</th>
                <th>Weight</th>
            </tr>
        </thead>
        <tbody>

            {% for item in order.items.all %}

            <tr>
                <td>
                    {{ item.product.name }}
                </td>

                <td>
                    {{ item.quantity }}
                </td>

                <td>
                    {{ item.product.weight }} kg
                </td>
            </tr>

            {% empty %}

            <tr>
                <td colspan="3">
                    No items in this order.
                </td>
            </tr>

            {% endfor %}

        </tbody>
    </table>

</div>

<div class="card">

    <a
        class="button"
        href="{% url 'recommend-box' order.id %}"
    >
        Recommend Box
    </a>

</div>
```



```
{% endblock %}
```

7.20 Add Order URLs

Our `box_selector/urls.py` should now contain:

```
Python
from django.urls import path

from .views import (
    home,
    product_list,
    box_list,
    order_list,
    order_detail,
    recommend_box_view,
)

urlpatterns = [
    path(
        "",
        home,
        name="home",
    ),

    path(
        "products/",
        product_list,
        name="product-list",
    ),

    path(
        "boxes/",
        box_list,
        name="box-list",
    ),

    path(
        "orders/",
        order_list,
        name="order-list",
    ),

    path(
        "orders/<int:order_id>/",
        order_detail,
        name="order-detail",
    ),

    path(
        "orders/<int:order_id>/recommend/",
        recommend_box_view,
        name="recommend-box",
    ),
]
```

7.21 But There Is One Problem

Our current recommendation view returns:

```
Python
JsonResponse(...)
```

That's useful for an API.

But when a normal warehouse user clicks:

[Recommend Box]

we don't want them to see raw JSON.

We want a nice HTML page.

So we'll create a **separate HTML recommendation view**.

7.22 Create HTML Recommendation View

In `views.py`, add:

```
Python def recommendation_page(request,
order_id):    order = get_object_or_404(Order,
id=order_id)    box = recommend_box(order)

    return render(
request,
    "box_selector/recommendation.html",
    {
        "order": order,
        "box": box,
    },
    )
```

Notice something important: We

are still using:

```
Python
recommend_box(order)
```

from our service.

So the architecture remains:

```
HTML Page
↓
View
↓
Service
↓
Algorithm
```

7.23 Recommendation Template

Create:

```
templates/box_selector/recommendation.html
```

Add:

```

HTML
{% extends "box_selector/base.html" %}

{% block title %}
Box Recommendation
{% endblock %}

{% block content %}

<h1>Box Recommendation</h1>

<div class="card">

    <h2>Order #{{ order.id }}</h2>

</div>

{% if box %}

<div class="card">

    <div class="success">

        <h2>Recommended Box</h2>

<h3>{{ box.name }}</h3>

        <p>
            <strong>Dimensions:</strong>
            {{ box.length }} x
            {{ box.width }} x
            {{ box.height }} cm
        </p>

        <p>
            <strong>Maximum Weight:</strong>
            {{ box.max_weight }} kg
        </p>

        <p>
            <strong>Cost:</strong>
            ₹{{ box.cost }}
        </p>

    </div>

</div>

{% else %}

<div class="card">

    <div class="error">

        <h2>No Suitable Box</h2>

        <p>
            No suitable box was found for this order.
        </p>

    </div>

</div>

{% endif %}

<a
    class="button"
    href="{% url 'order-detail' order.id %}"
>
    Back to Order
</a>

```

```
{% endblock %}
```

7.24 Change the Button

In `order_detail.html`, change:

```
HTML
href="{% url 'recommend-box'
order.id %}"
```

to:

```
HTML href="{% url 'recommendation-page'
order.id %}"
```

So the button opens our HTML page.

7.25 Add the New URL

In `box_selector/urls.py`, import:

```
Python
recommendation_page,
```

and add:

```
Python
path(
    "orders/<int:order_id>/recommendation/",
    recommendation_page,
    name="recommendation-page", ),
```

Now our user flow is:

```
/orders/1/
↓
[ Recommend Box ]
↓
/orders/1/recommendation/
    ↓ recommendation_page()    ↓ recommend_box()
↓
HTML result
```

7.26 Final views.py

At this point, your views should look approximately like:

```
Python
```

```

from django.http import JsonResponse
from django.shortcuts import get_object_or_404, render

from .models import Box, Order, Product from
.services import recommend_box

def home(request):
    return render(
        request,
        "box_selector/home.html",
    )

def product_list(request):
    products = Product.objects.all()

    return render(
        request,
        "box_selector/product_list.html",
        {"products": products},
    )

def box_list(request):
    boxes =
Box.objects.all().order_by("cost")

    return render(
        request,
        "box_selector/box_list.html",
        {"boxes": boxes},
    )

def order_list(request):
    orders =
Order.objects.all().order_by("-created_at")

    return render(
        request,
        "box_selector/order_list.html",
        {"orders": orders},
    )

def order_detail(request, order_id):
    order =
get_object_or_404(Order, id=order_id)

    return render(
        request,
        "box_selector/order_detail.html",
        {"order": order},
    )

def recommendation_page(request, order_id):
    order = get_object_or_404(Order, id=order_id)
    box = recommend_box(order)

    return render(
        request,
        "box_selector/recommendation.html",
        {
            "order": order,
            "box": box,
        },
    )

def recommend_box_view(request, order_id):
    order = get_object_or_404(Order, id=order_id)
    box = recommend_box(order)    if box is None:

```

```
return JsonResponse({
    "success": False,
    "message": "No suitable box found for this order."
})

return JsonResponse({
    "success": True,
    "order_id": order.id,
    "recommended_box": {
        "id": box.id,
        "name": box.name,
        "length": str(box.length),
        "width": str(box.width),
        "height": str(box.height),
        "max_weight": str(box.max_weight),
        "cost": str(box.cost),
    }
})
```

7.27 Why do we have two recommendation views?

This is actually useful.

API

```
/orders/1/recommend/
```

returns:

```
JSON
{
    "success": true,
    "recommended_box": {
        "name": "Medium Box"
    }
}
```

Web page

```
/orders/1/recommendation/
```

returns a user-friendly HTML page. So:

```
API
↓
Other applications / frontend
HTML View
↓
Human user
```

You can tell the interviewer:

"I separated the API response from the HTML presentation so the recommendation logic can support both programmatic and browser-based usage."

7.28 Test the UI

Start the server:

```
Bash python manage.py
runserver
```

Open:

```
http://127.0.0.1:8000/
```

You should see:

```
BOX SELECTION SYSTEM

Products
Boxes
Orders
```

Click:

```
Products
```

You should see your products. Then:

```
Boxes
```

You should see your boxes. Then:

```
Orders
```

You should see your orders. Click:

```
View Order
```

Then:

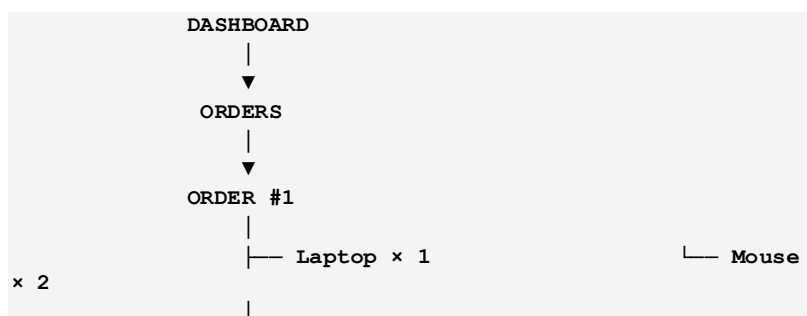
```
Recommend Box
```

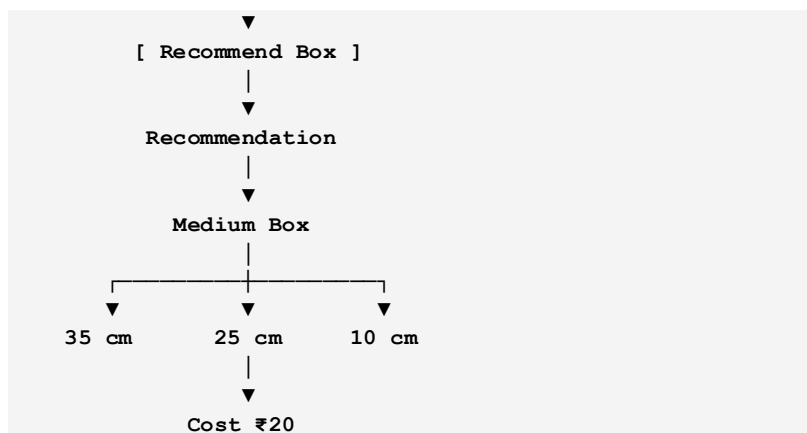
Finally:

```
Recommended Box
Medium Box
```

7.29 Your Complete User Flow

This is what you should demonstrate during your interview:





7.30 One important improvement

We currently create orders through Django Admin.

That's okay for now.

But a better final application should allow the warehouse user to create an order directly from the website:

```

Create Order

Product: [ Laptop ▼ ]
Quantity: [ 2 ]

Product: [ Mouse ▼ ]
Quantity: [ 1 ]

[ Create Order ]
  
```

Then:

```

Order Created
↓
Recommend Box
  
```

We'll add that functionality in the next stage.

7.31 Interview Questions From Step 7

Q1. What is Django Template?

"A Django template is an HTML file that can display dynamic data received from a Django view."

Q2. What is template inheritance?

"It allows multiple templates to share a common base template, which reduces duplicated HTML."

Q3. How does a view send data to a template?

For example:


```
Python return
render(
    request,
    "box_selector/product_list.html",
    {"products": products},
)
```

The dictionary:

```
Python
{"products": products}
```

passes data to the template.

Q4. Why separate service and view?

"The view handles the HTTP request and presentation, while the service handles the boxselection business logic."

Q5. Why have an API and HTML view?

"The API provides machine-readable JSON, while the HTML view provides a user-friendly interface for warehouse staff."

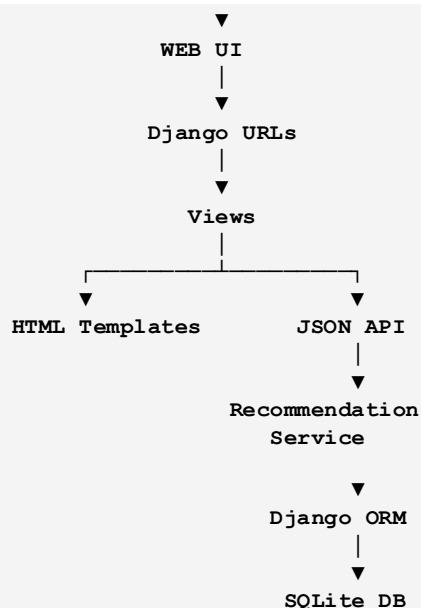
Step 7 Checklist ☒

You should now have:

- ☒ templates/ directory
- ☒ base.html
- ☒ Dashboard
- ☒ Product list
- ☒ Box list
- ☒ Order list
- ☒ Order detail
- ☒ Recommendation page
- ☒ Navigation
- ☒ HTML recommendation result
- ☒ API recommendation endpoint

Your project is now becoming a **complete Django application**:

```
USER
|
```



Next: Step 8 — Create Orders From the Website

We'll remove the need to manually create orders through Django Admin and build a proper **Create Order form** with product selection and quantity. Then we'll add validation so users can't enter invalid quantities or invalid product data.



step 8

Step 8 — Create Orders From the Website

Now we have:

```

Step 3 → Database
Step 4 → Recommendation algorithm
Step 5 → API / Views
Step 6 → Tests
Step 7 → User interface
  
```

But there is one major problem:

The user currently has to create orders through Django Admin.

A real warehouse user should be able to create an order directly from our website. So in Step 8 we'll build:

```

Create Order
↓
Select Product
↓
Enter Quantity
↓
Add More Products
↓
Create Order
↓
Order Details
↓
  
```

Recommend Box

The assignment requires the system to recommend a box **when a customer places an order**, so having a proper order-creation flow makes our implementation much closer to that requirement.

Assignment - Python_Django Deve...

8.1 What will our page look like?

We'll make a simple form:

CREATE NEW ORDER		
Product:	[Laptop	▼]
Quantity:	[1	]
Product:	[Mouse	▼]
Quantity:	[2	]
[Create Order]		

For the first version, we'll keep the form simple and allow the user to add **one or more order items** using multiple product/quantity rows.

8.2 Why use Django Forms?

Instead of manually checking everything in `views.py`, Django provides **Forms**. A Django

form helps us: receive user input validate input display errors clean data

- make form handling easier For
- example:
- `Quantity = -5`
- should not be accepted.

We can validate it.

8.3 Create `forms.py`

Inside:

```
box_selector/
```

create:

```
forms.py
```

We'll start with one form for an individual order item.

```
Python
from django import forms

from .models import Product

class OrderItemForm(forms.Form):
    product = forms.ModelChoiceField(
        queryset=Product.objects.all()
    )

    quantity = forms.IntegerField(
        min_value=1
    )
```

8.4 Understand ModelChoiceField

This:

```
Python
product =
forms.ModelChoiceField(
    queryset=Product.objects.
all() )
```

means:

Show the available products from our database as choices.

So if our database has:

```
Laptop
Mouse
Keyboard
```

the form will display something like:

```
Product:
[ Laptop ▼ ]
```

8.5 Why IntegerField?

Quantity is a number.

So:

```
Python
quantity = forms.IntegerField(
    min_value=1 )
```

allows:

```
1
2
3
10
```

but rejects:

```
0
-1
```

This is good validation.

8.6 Create the Order Form

Now we need a form that can contain multiple order items.

A simple approach for our project is to use Django's **formsets**. Create:

```
Python from django.forms import
formset_factory
```

Then after `OrderItemForm`:

```
Python
OrderItemFormSet = formset_factory(
    OrderItemForm,
    extra=1
)
```

This means:

Create a collection of `OrderItemForm` forms.

8.7 What is a Formset?

A formset lets us handle multiple forms together.

For example:

```
Form 1:
Laptop x 1

Form 2:
Mouse x 2

Form 3:
Keyboard x 1
```

All three can be submitted together.

This is useful for our order because one order can contain multiple products.

8.8 Our `forms.py`

It should now look like:

```
Python
from django import forms
from django.forms import formset_factory from

.models import Product

class OrderItemForm(forms.Form):
    product = forms.ModelChoiceField(
        queryset=Product.objects.all()
    )

    quantity = forms.IntegerField(
        min_value=1
    )

OrderItemFormSet = formset_factory(
    OrderItemForm,
    extra=1
)
```

8.9 Create the Order View

Open:

```
box_selector/views.py
```

Add:

```
Python
from django.db import transaction

from .forms import OrderItemFormSet from
.models import Order, OrderItem
```

Now create:

```
Python def
create_order(request):
    if request.method == "POST":
        formset = OrderItemFormSet(request.POST)

        if formset.is_valid():
            with transaction.atomic():
                order = Order.objects.create()

                for form in formset:
                    product = form.cleaned_data["product"]
                    quantity = form.cleaned_data["quantity"]

                    OrderItem.objects.create(
                        order=order,
```

```

        product=product,
        quantity=quantity,
    )

    return redirect(
        "order-detail",
        order_id=order.id,
    )

    else:
        formset =
        OrderItemFormSet()

    return render(
        request,
        "box_selector/create_order.html",
        {"formset": formset},
    )

```

We'll understand this carefully.

8.10 Why do we check `request.method`?

We have:

```

Python if request.method ==
"POST":

```

There are two situations.

User opens the page

```
GET
```

We show an empty form.

User submits the form

```
POST
```

We process the submitted data. So:

```

GET
↓
Show form

POST
↓
Validate form
↓
Create order

```

8.11 What does `formset.is_valid()` do?

```

Python if
formset.is_valid():

```

Django checks the submitted values. For

example:

```
Product = Laptop
Quantity = 2
```

Valid:

✓

But:

```
Quantity = -5
```

Invalid:

X

Django will automatically make the form invalid because we set:

```
Python
min_value=1
```

8.12 What is `cleaned_data`?

After a form is validated, Django gives us cleaned values. For

example:

```
Python
product =
form.cleaned_data["product"]
quantity =
form.cleaned_data["quantity"]
```

This means:

Get the validated product and quantity from the form.

8.13 Create the Order

This:

```
Python
order = Order.objects.create()
```

creates:

Order #4

Then we add each product to that order.

8.14 Create OrderItems

We loop through:

```
Python for form in
formset:
```

For each form:

```
Python
OrderItem.objects.create(
order=order,
product=product,
quantity=quantity,
)
```

For example:

```
Order #10

OrderItem 1
Laptop x 1

OrderItem 2
Mouse x 2
```

8.15 Why use `transaction.atomic()`?

This is a good interview-level concept. We

wrote:

```
Python with
transaction.atomic():
```

This means the database operations inside the block are treated as **one transaction**. Imagine:

```
Create Order
↓
Create Item 1 ✓
↓
Create Item 2 ✓
↓
Create Item 3 ✗
```

Without a transaction, you might end up with a partially created order.

With `transaction.atomic()`:

```
Everything succeeds → COMMIT
```

or:

Something fails → ROLLBACK
So we don't leave incomplete data.

Interview answer

"`transaction.atomic()` makes the database operations behave as one transaction. If something fails, the changes can be rolled back instead of leaving partial data." That's a strong answer.

8.16 Redirect After Creating Order

After successfully creating the order:

```
Python return redirect(    "order-
detail",    order_id=order.id,
)
```

The user is sent to:

```
/orders/4/
```

So they can immediately see their new order.

8.17 Important: Import `redirect`

At the top of `views.py`, make sure you have:

```
Python
from django.shortcuts import (
    get_object_or_404,
    redirect,    render,
)
```

8.18 Create the HTML Form

Create:

```
templates/box_selector/create_order.html
```

Add:

```
HTML
{% extends "box_selector/base.html" %}

{% block title %}
Create Order
{% endblock %}

{% block content %}
<div class="card">

    <h1>Create New Order</h1>
```

```
<form method="post">

    {% csrf_token %}

    {{ formset.management_form }}

    {% for form in formset %}

        <div class="card">

            <h3>Product {{ forloop.counter }}</h3>

            {{ form.product.label_tag }}
            {{ form.product }}

            <br><br>

            {{ form.quantity.label_tag }}
            {{ form.quantity }}

            {% if form.errors %}
                <div class="error">
                    {{ form.errors }}
                </div>
            {% endif %}

        </div>

    {% endfor %}

    <button type="submit">
        Create Order
    </button>

</form>

</div>

{% endblock %}
```

8.19 What is {% csrf_token %}?

You'll see:

```
django
{% csrf_token %}
```

inside the form.

This is a Django security feature that helps protect POST forms against **Cross-Site Request Forgery (CSRF)** attacks.

For a POST form in Django, you should normally include it. Interview

answer:

"CSRF protection helps prevent unauthorized websites from submitting requests using a user's authenticated session." For a fresher, that's enough.

8.20 What is management_form?

This:

```
django
```

```
{{ formset.management_form }}
```

is required for Django formsets.

It contains hidden information that tells Django how many forms belong to the formset.

Without it, formset processing won't work correctly. Remember:

```
Formset
↓
management_form
↓
Django knows how to process the forms
```

8.21 Add Create Order URL

Open:

```
box_selector/urls.py
```

Import:

```
Python
create_order,
```

Then add:

```
Python
path(
    "orders/create/",
    create_order,      name="create-
order",
),
```

Now:

```
http://127.0.0.1:8000/orders/create/
```

opens our form.

8.22 Add Create Order Button

Open:

```
order_list.html
```

At the top, add:

```
HTML
<a
  class="button"
  href="{% url 'create-order' %}"
>
  Create New Order
</a>
```

So the page becomes:

```
Orders [ Create New Order ]
```

```
Order #1
Order #2
Order #3
```

8.23 Problem: We only have one form

Currently:

```
Python
extra=1
```

means we initially display one product row.

We want multiple products.

We could build JavaScript to dynamically add rows, but for an interview assignment, let's keep the first implementation simple. We can set:

```
Python
OrderItemFormSet = formset_factory(
    OrderItemForm,
    extra=3
)
```

Then the page initially displays:

```
Product 1
Product 2
Product 3
```

The user can leave unused rows empty **only if we configure the formset accordingly**. A cleaner approach is to make the form optional.

Let's modify our formset.

8.24 Allow Empty Extra Rows

Change:

```
Python
OrderItemFormSet = formset_factory(
    OrderItemForm,
    extra=3
)
```

to:

```
Python
OrderItemFormSet = formset_factory(
    OrderItemForm,
    extra=3,
    can_delete=True,
)
```

But that introduces delete fields.

For our first version, an even simpler approach is:

```
Python
OrderItemFormSet = formset_factory(
    OrderItemForm,
    extra=1,
)
```

and later we can add JavaScript/dynamic forms.

For now, keep one product per form submission.

Then we can improve it in a later iteration.

But our actual assignment needs multiple products, so we need a practical solution.

8.25 Better Approach for This Assignment

Let's use a simple **three-row formset** and allow empty rows.

Change `OrderItemForm`:

```
Python class
OrderItemForm(forms.Form):
    product = forms.ModelChoiceField(
        queryset=Product.objects.all(),
        required=False,
    )

    quantity = forms.IntegerField(
        min_value=1, required=False,
    )
```

Now empty rows are allowed. Then:

```
Python
OrderItemFormSet = formset_factory(
    OrderItemForm,
    extra=3,
)
```

Now users can enter:

```
Product 1 → Laptop × 1
Product 2 → Mouse × 2
Product 3 → Keyboard × 1
```

Or:

```
Product 1 → Laptop × 1
Product 2 → [empty]
Product 3 → [empty]
```

8.26 We Need to Skip Empty Rows

Our view currently assumes every form has a product.

We need to change it.

Use:

```

Python def
create_order(request):
    if request.method == "POST":
        formset = OrderItemFormSet(request.POST)

        if formset.is_valid():
            valid_items = []

            for form in formset:
                product = form.cleaned_data.get("product")
                quantity = form.cleaned_data.get("quantity")

                if product is None and quantity is None:
                    continue

                if product is None or quantity is None:
                    form.add_error(
                        None,
                        "Product and quantity are required."
                    )
                continue

            valid_items.append(
                (product, quantity)
            )

            if not valid_items:
                formset.non_form_errors = lambda: [
                    "Add at least one product."
                ]
            else:
                with transaction.atomic():
                    order = Order.objects.create()

                    for product, quantity in valid_items:
                        OrderItem.objects.create(
                            order=order,
                            product=product,
                            quantity=quantity,
                        )
                    return redirect(
                        "order-detail",
                        order_id=order.id,
                    )

            else:
                formset =
                OrderItemFormSet()

            return render(
                request,
                "box_selector/create_order.html",
                {"formset": formset},
            )

```

However, I **don't recommend keeping this exact error-handling pattern** because assigning a lambda to `non_form_errors` is not clean Django practice. Let's make it cleaner.

8.27 Cleaner Formset Validation

We can create a custom formset later.

For this assignment, let's keep the view straightforward and use:

```
Python def
create_order(request):
    if request.method == "POST":
        formset = OrderItemFormSet(request.POST)

        if formset.is_valid():
            valid_items = []

            for form in formset:
                product = form.cleaned_data.get("product")
                quantity = form.cleaned_data.get("quantity")

                if not product and not quantity:
                    continue

                if not product or not quantity:
                    form.add_error(
                        None,
                        "Please provide both product and quantity."
                    )
                    continue

            valid_items.append((product, quantity))

            if valid_items:
                with transaction.atomic():
                    order = Order.objects.create()

                    for product, quantity in valid_items:
                        OrderItem.objects.create(
                            order=order,
                            product=product,
                            quantity=quantity,
                        )

            return redirect(
                "order-detail",
                order_id=order.id,
            )

        else:
            formset = OrderItemFormSet()
            return render(
                request,
                "box_selector/create_order.html",
                {"formset": formset},
            )
```

This is easier to understand.

8.28 One Important Validation Issue

There is one problem with:

```
Python
quantity = forms.IntegerField(
    min_value=1,
    required=False,
)
```

If someone enters:

```
Product = Laptop
Quantity = empty
```


the form itself is technically valid because quantity is optional. Our view catches that:

```
Python if not product or not
quantity:
```

and adds an error. So

we handle:

```
Product only → ✗
Quantity only → ✗
Both → ☒
Neither → skip
```

That's what we want.

8.29 Add a Better Create Order Button

In `home.html`, we can also add:

```
HTML
<a class="button"
href="{% url 'create-order' %}"
>
  Create New Order
</a>
```

So the dashboard becomes more useful.

8.30 Test the Complete Flow

Start Django:

```
Bash python manage.py
runserver
```

Open:

```
http://127.0.0.1:8000/orders/create/
```

You should see:

```
Create New Order

Product 1
Product: [ Laptop ▼ ]
Quantity: [ 1 ]

Product 2
Product: [ Mouse ▼ ]
Quantity: [ 2 ]

Product 3
Product: [ Keyboard ▼ ]
```

```
Quantity: [ 1 ]  
[ Create Order ]
```

Submit it.

Django should create:

```
Order #4  
Laptop × 1  
Mouse × 2  
Keyboard × 1
```

Then redirect you to:

```
/orders/4/
```

8.31 Then Recommend the Box

From the order page:

```
Order #4  
Laptop × 1  
Mouse × 2  
Keyboard × 1  
[ Recommend Box ]
```

Click:

```
Recommend Box
```

And our Step 4 algorithm runs.

```
Order ↓  
recommend_box()  
↓  
Calculate requirements  
↓  
Find suitable boxes  
↓  
Lowest cost  
↓  
Recommendation
```

8.32 Important Issue With Our Simplified Algorithm

Remember what we discussed in Step 4. If an

order contains:

```
Laptop × 1  
Keyboard × 1  
Mouse × 2
```

our current algorithm uses the maximum dimensions rather than performing true 3D packing.

So the UI can correctly create the order, but the **packing algorithm remains simplified**.

That's okay as long as we clearly document the assumption.

Printed using [ChatGPT to PDF](#), powered by PDFCrowd [HTML to PDF API](#).

The assignment itself does not specify a detailed multi-product packing algorithm, so we should not claim that we've solved general 3D warehouse packing.

Assignment - Python_Django Deve...

8.33 What We Added in Step 8

Before:

```
Admin
↓
Create Order
```

Now:

```
Website
↓
Create Order
↓
Select Products
↓
Enter Quantity
↓
Save Order
↓
Order Details
↓
Recommend Box
```

That's a major improvement.

8.34 Interview Questions

Q1. Why did you use Django Forms?

"Django Forms help me handle user input and validation in a structured way." **Q2. Why**

use a formset?

"One order can contain multiple products, so a formset allows me to submit multiple orderitem forms together."

Q3. Why validate quantity?

"Quantity must be a positive value because zero or negative quantities don't make sense for an order."

Q4. Why use `transaction.atomic()`?

"Creating an order and its order items should happen together. If something fails, I don't want a partially created order."

Q5. What happens after creating an order?

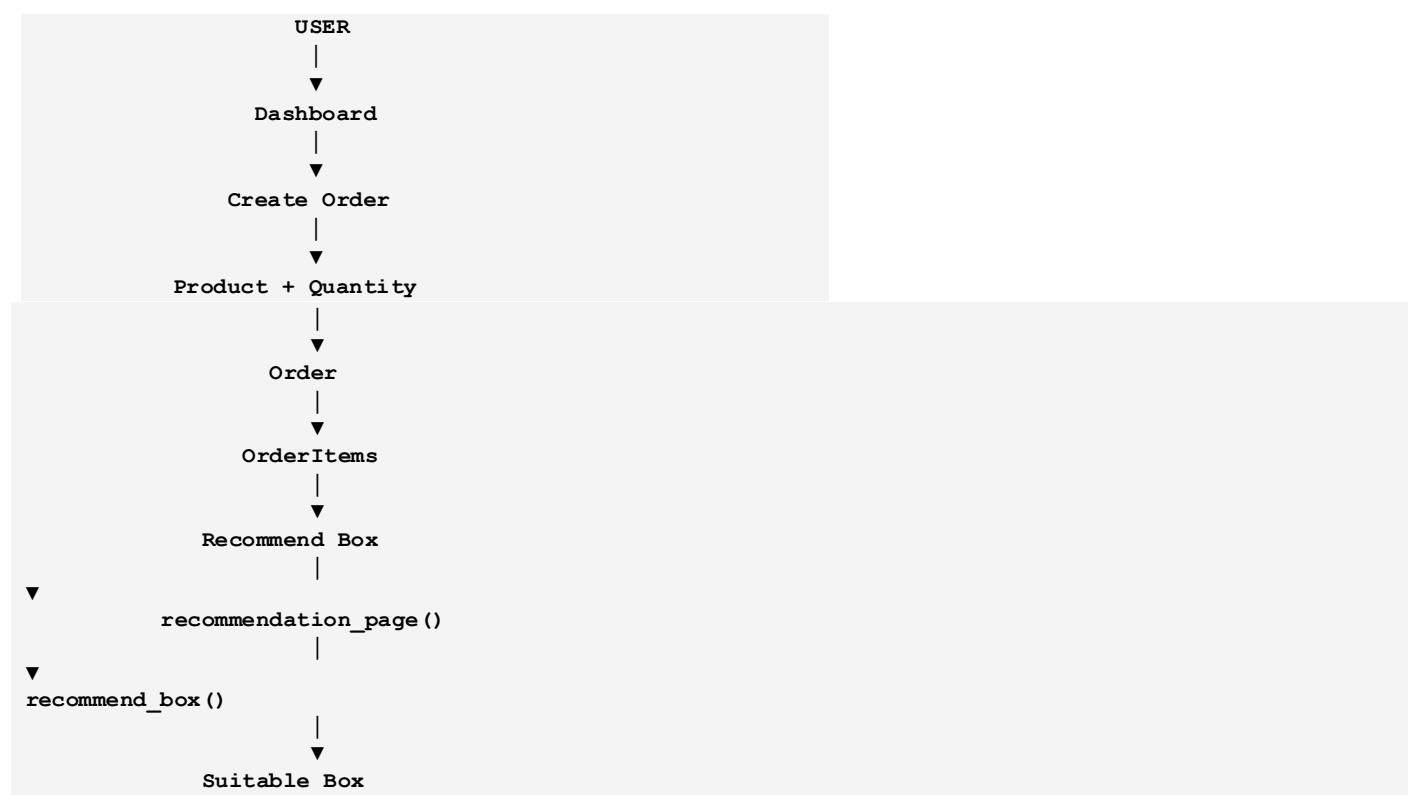
"The user is redirected to the order detail page, where they can request the box recommendation."

8.35 Step 8 Checklist

You should now have:

- ☒ forms.py created
- ☒ OrderItemForm created
- ☒ Form validation
- ☒ Multiple order-item forms
- ☒ Create Order view
- ☒ POST handling
- ☒ Transaction handling
- ☒ Create Order template
- ☒ Create Order URL
- ☒ Create Order button
- ☒ Order created from website
- ☒ Redirect to order detail
- ☒ Recommendation still works

Your application is now:



Next: Step 9 — Improve Validation & Error Handling

We'll make the application more **production-like**: prevent duplicate products in the same order, validate dimensions/weight/cost, handle invalid quantities cleanly, improve error messages, and make the API responses more consistent.

This is the step that will help you answer interview questions like **"How did you handle edge cases and validation?"** very confidently.



give me list of products, orders, boxes, order items for add by admin , also give me models.py

Yes. For your interview project, you can add **realistic sample data through Django Admin**. I recommend using the following data because it also gives you good cases for testing the box recommendation logic.

1. Products

Go to **Admin** → **Products** → **Add Product**.

Laptop	30	20	5	2.00
Wireless Mouse	10	6	4	0.20
Keyboard	40	15	5	1.00
Headphones	20	18	10	0.50
Smartphone	16	8	1	0.20
Tablet	25	17	1	0.60
Monitor	55	35	10	4.00
Printer	45	40	20	6.00
Webcam	12	8	6	0.30
External Hard Drive	15	10	3	0.40

Units: Use **cm** for dimensions and **kg** for weight.

2. Boxes

Go to **Admin** → **Boxes** → **Add Box**.

Small Box	20	20	10	2	10
Medium Box	35	25	10	5	20
Large Box	50	40	20	10	40
Extra Large Box	70	50	30	20	70
Monitor Box	60	45	20	15	60
Heavy Duty Box	60	50	40	25	90

This gives you different box sizes and prices, which is useful for testing the **cheapest suitable box** logic.

3. Orders

You don't need to manually enter much information for an order.

Your current `order` model only has:

```
Python
created_at
```

So in Admin:

Admin → **Orders** → **Add Order**

Just click **Save**. For example:

Order #1	Automatically created
Order #2	Automatically created
Order #3	Automatically created
Order #4	Automatically created
Order #5	Automatically created

The `created_at` value will be automatically generated.

4. Order Items

After creating orders, go to:

Admin → **Order Items** → **Add Order Item**

Use these examples.

Order #1

Laptop	1

Expected box: **Medium Box** Why?

Laptop:

30 × 20 × 5 cm Weight:

2 kg

Small Box is only 20 × 20 × 10, so the laptop is too long. Medium Box:

35 × 25 × 10

Weight capacity = 5 kg

So Medium Box is suitable.

Order #2

Wireless Mouse	2
Headphones	1

Expected box: **Medium Box**

Order #3

Smartphone	2
Wireless Mouse	1

Expected box: **Small Box**

Order #4

Keyboard	1
Wireless Mouse	1

Expected box: **Large Box** Keyboard is:

40 × 15 × 5

So Medium Box's length of 35 cm is not enough.

Order #5

Monitor	1

Expected box: **Monitor Box**

Order #6

Printer	1

Expected box: **Extra Large Box** Printer:

45 × 40 × 20 Weight:

6 kg

Extra Large Box:

70 × 50 × 30 Weight

capacity:

20 kg

5. Important test order

You should also create one order that **cannot fit into any box**.

Order #7

Create a new product:

Very Large Machine	100	100	100	30

Then create:

Very Large Machine	1

Expected result:

No suitable box found.

This is an important interview test case because you can show that your application handles failure properly.

6. Your final `models.py`

For your current project, I recommend using this clean version:

Python

```
from django.db import models
```

```
class Product(models.Model):
    name = models.CharField(max_length=200)
```

```
    length = models.DecimalField(
max_digits=10,          decimal_places=2
    )
```

```
    width = models.DecimalField(
max_digits=10,          decimal_places=2
    )
```

```
    height = models.DecimalField(
max_digits=10,          decimal_places=2
    )
```

```
    weight = models.DecimalField(
max_digits=10,          decimal_places=2
    )
```

```
    def __str__(self):
return self.name
```

```
class Box(models.Model):
    name = models.CharField(max_length=200)
```

```
    length = models.DecimalField(
max_digits=10,          decimal_places=2
    )
```

```
    width = models.DecimalField(
max_digits=10,          decimal_places=2
    )
```

```
    height = models.DecimalField(
max_digits=10,          decimal_places=2
    )
```

```
    max_weight = models.DecimalField(
max_digits=10,          decimal_places=2
    )
```

```

    )

    cost = models.DecimalField(
max_digits=10,          decimal_places=2
    )

    def __str__(self):
return self.name

class Order(models.Model):      created_at =
models.DateTimeField(auto_now_add=True)

    def __str__(self):
return f"Order #{self.id}"

class OrderItem(models.Model):
order = models.ForeignKey(
Order,
    on_delete=models.CASCADE,
related_name="items"
    )

    product = models.ForeignKey(
Product,          on_delete=models.CASCADE
    )
    quantity =

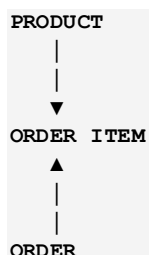
models.PositiveIntegerField()

    def __str__(self):
return f"{self.product.name} x {self.quantity}"

```

7. Simple understanding of the relationships

Think about it like this:



For example:

```

Order #1
├── Laptop × 1
├── Wireless Mouse × 2
└── Headphones × 1

```

The **OrderItem** is the connection between an Order and a Product. For example:

```

Order #1
+
Laptop
+
Quantity = 1
↓
OrderItem

```

Why OrderItem is necessary?

Because one order can contain many products, and the same product can appear in many orders. For example:

```
Order #1 → Laptop × 1
Order #2 → Laptop × 2
Order #3 → Mouse × 5
```

So we don't put products directly inside `Order`. We use `OrderItem`.

8. After adding this data

Run:

```
Bash
python manage.py
makemigrations python
manage.py migrate
python manage.py
runserver
```

Then open:

```
http://127.0.0.1:8000/admin/
```

Add the data in this order:

1. Products → 2. Boxes → 3. Orders → 4. Order Items Then you

can test:

```
/orders/1/recommend/
```

or your HTML recommendation page:

```
/orders/1/recommendation/
```

One important point for your interview: **the current algorithm uses the largest length/width/height among the products plus total weight. It is a simplified packing model, not a true 3D packing algorithm.** Don't claim that it performs exact physical packing.



now start step 9

STEP 9 — Improve Validation & Error Handling

Now we will make your project **more professional and interview-ready**.

In Step 8, we created orders from the website. But a good application should also stop **wrong data** from entering the database.

For example:

- Product dimension cannot be negative.
- Product weight cannot be negative.
- Box cost cannot be negative.
- Quantity cannot be 0.
- Same product should not be added twice to one order.
- An incomplete order item should show an error.
- If no box can handle an order, the user should get a clear message.

9.1 What we will improve

Our Step 9 will have these parts:

```
Step 9
|
├─ 9.1 Product validation
├─ 9.2 Box validation
├─ 9.3 Order quantity validation
├─ 9.4 Duplicate product validation
├─ 9.5 Empty order validation ── 9.6 Recommendation
error handling
├─ 9.7 API error response
└─ 9.8 Tests for all validation cases
```

9.2 Validate Product and Box data

Currently, Django allows values such as:

```
Length = -10
Weight = -5
Cost = -100
```

That doesn't make business sense.

We can use Django validators.

Change your `models.py` to:

```
Python
from django.core.validators import MinValueValidator from
django.db import models

class Product(models.Model):
    name = models.CharField(max_length=200)

    length = models.DecimalField(
        max_digits=10,
```

```

        decimal_places=2,
        validators=[MinValueValidator(0.01)]
    )

    width = models.DecimalField(
        max_digits=10, decimal_places=2,
        validators=[MinValueValidator(0.01)]
    )

    height = models.DecimalField(
        max_digits=10, decimal_places=2,
        validators=[MinValueValidator(0.01)]
    )

    weight = models.DecimalField(
        max_digits=10, decimal_places=2,
        validators=[MinValueValidator(0.01)]
    )

    def __str__(self):
    return self.name

class Box(models.Model):
    name =
models.CharField(max_length=200)

    length = models.DecimalField(
        max_digits=10, decimal_places=2,
        validators=[MinValueValidator(0.01)]
    )

    width = models.DecimalField(
        max_digits=10, decimal_places=2,
        validators=[MinValueValidator(0.01)]
    )

    height = models.DecimalField(
        max_digits=10, decimal_places=2,
        validators=[MinValueValidator(0.01)]
    )

    max_weight = models.DecimalField(
        max_digits=10, decimal_places=2,
        validators=[MinValueValidator(0.01)]
    )

    cost = models.DecimalField(
        max_digits=10, decimal_places=2,
        validators=[MinValueValidator(0.01)]
    )

    def __str__(self):
    return self.name

class Order(models.Model):
    created_at =
models.DateTimeField(auto_now_add=True)

    def __str__(self):
    return f"Order #{self.id}"

class OrderItem(models.Model):
    order = models.ForeignKey(
        Order,
        on_delete=models.CASCADE,
        related_name="items"
    )

```

```

    )

    product = models.ForeignKey(
        Product,
        on_delete=models.CASCADE
    )

    quantity = models.PositiveIntegerField(
        validators=[MinValueValidator(1)]
    )

    def __str__(self):
        return f"{self.product.name} x {self.quantity}"

```

What changed?

We added:

```

Python from django.core.validators import
MinValueValidator

```

And:

```

Python validators=[MinValueValidator(0.01)]

```

This means:

```

0.01 or greater → allowed
0 → rejected
negative → rejected

```

For quantity:

```

Python validators=[MinValueValidator(1)]

```

So:

```

1 → allowed
2 → allowed
0 → rejected
-1 → rejected

```

9.3 Run migrations

Because we changed the model definitions, run:

```

Bash
python manage.py
makemigrations python
manage.py migrate

```

Then check your project:

```

Bash
python manage.py check

```

You should get:

```
System check identified no issues.
```

9.4 Improve Order Form validation

Now let's improve `forms.py`.

Use:

```
Python
from django import forms
from django.forms import formset_factory from

.models import Product

class OrderItemForm(forms.Form):
    product = forms.ModelChoiceField(
        queryset=Product.objects.all(),
        required=False,
    )

    quantity = forms.IntegerField(
        min_value=1,          required=False,
    )

class BaseOrderItemFormSet(forms.BaseFormSet):

    def clean(self):
        super().clean()

        if any(self.errors):
            return

        products = []
        has_items = False

        for form in self.forms:
            product = form.cleaned_data.get("product")
            quantity = form.cleaned_data.get("quantity")

            # Completely empty row
            if not product and not quantity:
                continue
            has_items = True

            # Product missing
            if not product:
                form.add_error(
                    "product",
                    "Please select a product."
                )

            # Quantity missing
            if not quantity:
                form.add_error(
                    "quantity",
                    "Please enter a quantity."
                )

            # Duplicate product
            if product:
```

```

        if product in products:
            form.add_error(
                "product",
                "This product is already added to the order."
            )
        else:
            products.append(product)

    if not has_items:
        raise forms.ValidationError(
            "Please add at least one product to the order."
        )

    OrderItemFormSet = formset_factory(
        OrderItemForm,
        formset=BaseOrderItemFormSet,
        extra=3,
    )

```

9.5 Understand this code

This part:

```

Python class
BaseOrderItemFormSet(forms.BaseFormSet):

```

creates our own formset validation.

The normal Django form validates individual rows.

Our custom formset validates the **whole order**. For

example:

```

Row 1 → Laptop × 1
Row 2 → Mouse × 2
Row 3 → Keyboard × 1

```

The form validates each row.

The formset validates the relationship between rows.

Duplicate product example

Suppose the user enters:

```

Row 1 → Laptop × 1
Row 2 → Laptop × 2

```

We don't want this.

Instead, the user should enter:

```

Laptop × 3

```

Our code checks:

```

Python if product in
products:

```


If the product already exists, it shows:

```
This product is already added to the order.
```

9.6 Empty order validation

Suppose the user clicks:

Create Order without

selecting anything. We don't

want to create:

```
Order #10
└─ Nothing
```

Instead, we show:

```
Please add at least one product to the order.
```

This is handled by:

```
Python if not
has_items:
    raise forms.ValidationError(
        "Please add at least one product to the order."
    )
```

This is an important validation for your interview.

9.7 Improve create_order view

Now update your view.

```
Python
from django.db import transaction
from django.shortcuts import redirect, render

from .forms import OrderItemFormSet from
.models import Order, OrderItem

def create_order(request):
    if request.method == "POST":
        formset = OrderItemFormSet(request.POST)
    if formset.is_valid():
        valid_items =
        []
        for form in formset:
```

```

        product = form.cleaned_data.get("product")
        quantity = form.cleaned_data.get("quantity")

        if not product and not quantity:
            continue

        if product and quantity:
            valid_items.append(
                (product, quantity)
            )

        transaction.atomic():

            order = Order.objects.create()

        for product, quantity in valid_items:

            OrderItem.objects.create(
                order=order,
                quantity=quantity,
                product=product,
            )

        return redirect(
            "order-detail",
            order_id=order.id
        )

    else:
        formset =
        OrderItemFormSet()

        return render(
            request,
            "box_selector/create_order.html",
            {
                "formset": formset
            }
        )

```

9.8 Why `transaction.atomic()`?

This is very important for an interview. We

have:

```

Python with
transaction.atomic():

```

Inside it, we create:

```

Order
OrderItem
OrderItem
OrderItem

```

Imagine this happens:

```

Order created      ✓
OrderItem 1 created ✓
OrderItem 2 created ✓
OrderItem 3 failed  X

```

Without a transaction, we could have a partially created order.

With:

```
Python
transaction.atomic()
```

Django can roll back the database changes if an error occurs. Simple

interview answer:

"I use transaction.atomic so that order creation and its order items are handled as one database transaction. If something fails, the changes can be rolled back instead of leaving an incomplete order."

9.9 Improve recommendation error handling

Our service currently returns:

```
Python
None
```

when no box is available.

That's okay.

Keep the service simple:

```
Python def
recommend_box(order):
    items = order.items.all()

    if not items.exists():
return None

    total_weight = sum(
        item.product.weight * item.quantity
        for item in items
    )

    required_length = max(
item.product.length
for item in items
    )

    required_width = max(
item.product.width
for item in items
    )

    required_height = max(
item.product.height
for item in items
    )

    suitable_boxes = Box.objects.filter(
length__gte=required_length,          width__gte=required_width,
height__gte=required_height,
max_weight__gte=total_weight,
    ).order_by("cost")    return
suitable_boxes.first()
```

The service doesn't need to know anything about HTML or JSON. That's good architecture.

9.10 Improve API response

Your API view can remain:

```
Python
from django.http import JsonResponse from
django.shortcuts import get_object_or_404

from .models import Order from
.services import recommend_box

def recommend_box_view(request, order_id):

    order = get_object_or_404(
        Order, id=order_id
    )
    box =
    recommend_box(order)

    if box is None:
        return JsonResponse(
            {
                "success": False,
                "message": "No suitable box found for this order."
            },
            status=404
        )

    return JsonResponse(
        {
            "success": True,
            "order_id": order.id,
            "recommended_box": {
                "id": box.id,
                "name": box.name,
                "length": str(box.length),
                "width": str(box.width),
                "height": str(box.height),
                "max_weight": str(box.max_weight),
                "cost": str(box.cost),
            }
        }
    )
```

Notice this:

```
Python
status=404
```

If no box is found, the API tells the client that the requested recommendation is unavailable.

9.11 Test cases for Step 9

Now add validation tests.

At the bottom of `tests.py`, you can add tests such as:

```
Python
from django.core.exceptions import ValidationError
from django.test import TestCase from .models
import Box, Product

class ModelValidationTests(TestCase):

    def test_product_cannot_have_negative_length(self):

        product = Product(
name="Invalid Product",
length=-10,          width=10,
height=10,          weight=1,
        )

        with self.assertRaises(ValidationError):
product.full_clean()    def
test_product_cannot_have_zero_weight(self):

        product = Product(
name="Invalid Product",
        length=10,
width=10,
height=10,
weight=0,
        )

        with self.assertRaises(ValidationError):
product.full_clean()    def
test_box_cannot_have_negative_cost(self):

        box = Box(
        name="Invalid Box",
length=20,
width=20,
height=10,
max_weight=5,
cost=-10,
        )

        with self.assertRaises(ValidationError):
box.full_clean()
```

9.12 Run all tests

Run:

```
Bash python manage.py
test
```

You should now have your original recommendation tests plus the new validation tests. For example:

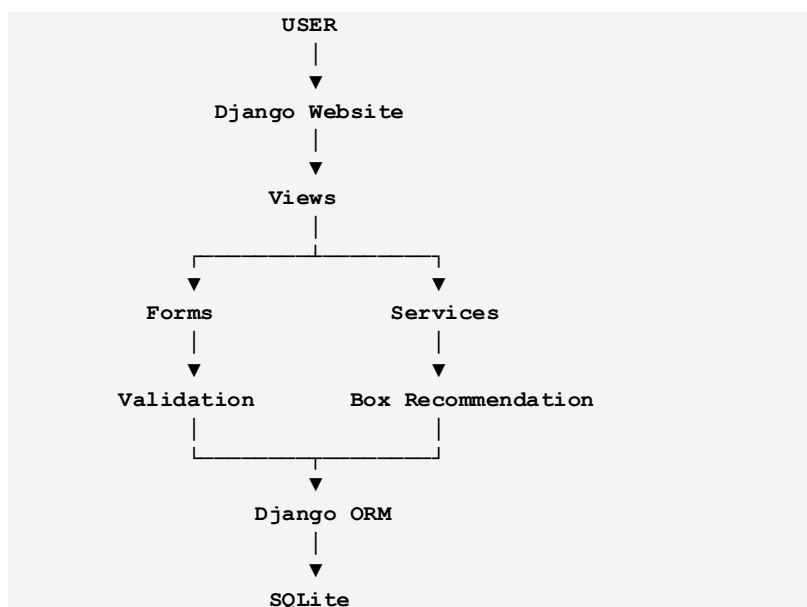
```
Found 11 test(s).
.....
-----Ran 11
tests in 0.xxxs

OK
```

Your exact number may be different depending on how many tests you currently have.

9.13 Step 9 final architecture

After Step 9, your project becomes:



This is much better for an interview because your responsibilities are separated:

<code>models.py</code>	Database structure + basic validation
<code>forms.py</code>	User input validation
<code>views.py</code>	Handle HTTP requests
<code>services.py</code>	Box recommendation/business logic
<code>urls.py</code>	URL routing
<code>tests.py</code>	Automated testing
<code>templates/</code>	User interface

Interview answer for Step 9

If the interviewer asks:

"How did you handle validation and errors?" Say:

"I added validation at different levels. In the models, I validate that dimensions, weight and box cost cannot be zero or negative. In the order form, I validate that quantity is at least one, the order contains at least one product, and the same product is not added twice. I also use `transaction.atomic` when creating an order so the order and order items are saved safely. If no suitable box is available, the system shows a clear message and the API returns an appropriate error response." **Step 9 is now complete.**

Next Step 10 should be: improve the database/admin setup and make the Django Admin professional — including `list_display`, search, filters, ordering, inline OrderItems, and easier management of Products, Boxes, and Orders.



step 10

STEP 10 — Make Django Admin Professional

Now we will improve the **Django Admin Panel**.

At the moment, your admin probably looks very basic:

```
Products
Boxes
Orders
Order items
```

For an interview project, we can make it much better and easier to use.

10.1 What we will improve

In Step 10 we will add:

```
Step 10
├── Product Admin
│   ├── Show useful columns
│   ├── Search products
│   ├── Filter products
│   └── Add ordering
├── Box Admin
│   ├── Show dimensions
│   ├── Search boxes
│   ├── Filter boxes
│   └── Order by cost
├── Order Admin
│   ├── Show order ID
│   ├── Show created date
│   └── Add OrderItems inside Order
└── OrderItem Admin
    ├── Search
    └── Filter
```

The biggest improvement will be **OrderItem Inline**.

Instead of doing this:

```
Order
↓
Save Order

Then separately:
Order Item
↓
Select Order
↓
Select Product
```

↓
Quantity

we will be able to do:

Order #1	
Laptop	1
Mouse	2
Headphones	1
SAVE	

This is much easier.

10.2 Update admin.py

Open:

box_selector/admin.py

Replace the existing code with:

```
Python
from django.contrib import admin from .models

import Box, Order, OrderItem, Product

@admin.register(Product) class
ProductAdmin(admin.ModelAdmin):
    list_display = (
        "id",
        "name",
        "length",
        "width",
        "height",
        "weight",
    )

    search_fields = (
        "name",
    )

    ordering = (
        "name",
    )

@admin.register(Box) class
BoxAdmin(admin.ModelAdmin):
```



```
list_display = (
    "id",
    "name",
    "length",
    "width",
    "height",
    "max_weight",
    "cost",
)

search_fields = (
    "name",
)

ordering = (
    "cost",
)

class OrderItemInline(admin.TabularInline):
    model = OrderItem
    extra = 1

@admin.register(Order) class
OrderAdmin(admin.ModelAdmin):
    list_display = (
        "id",
        "created_at",
    )

    ordering = (
        "-created_at",
    )

    inlines = [
        OrderItemInline,
    ]

@admin.register(OrderItem) class
OrderItemAdmin(admin.ModelAdmin):
    list_display = (
        "id",
        "order",
        "product",
        "quantity",
    )

    search_fields = (
        "product__name",
    )

    list_filter = (
        "product",
    )
```

10.3 Product Admin

This:

```
Python list_display = (
    "id",
    "name",
    "length",
    "width",
    "height",
    "weight",
)
```

controls which columns appear in Django Admin. Instead of:

```
Product
-----
```

Laptop
Mouse
Keyboard

you will see:

ID	Name	Length	Width	Height	Weight
1	Laptop	30.00	20.00	5.00	2.00
2	Wireless Mouse	10.00	6.00	4.00	0.20
3	Keyboard	40.00	15.00	5.00	1.00

This makes it much easier to check your data.

10.4 Product search

We added:

```
Python search_fields = (
    "name",
)
```

Now you can search:

Laptop

and Django Admin will find:

Laptop

You can also search:

Mouse

and get:

Wireless Mouse

10.5 Product ordering

We added:

```
Python ordering = (
    "name",
)
```

Products will be displayed alphabetically. For

example:

External Hard Drive
Headphones
Keyboard
Laptop Monitor

```
Printer
Smartphone
Tablet
Wireless Mouse
```

10.6 Box Admin

For boxes we use:

```
Python list_display = (
    "id",
    "name",
    "length",
    "width",
    "height",
    "max_weight",
    "cost",
)
```

Now you can easily compare boxes:

ID	Name	L	W	H	Max Weight	Cost
1	Small Box	20	20	10	2	10
2	Medium Box	35	25	10	5	20
3	Large Box	50	40	20	10	40

This is useful because the box recommendation algorithm depends on exactly these values.

10.7 Order boxes by cost

We use:

```
Python ordering = (
    "cost",
)
```

So the boxes appear from cheapest to most expensive:

```
Small Box
Medium Box
Large Box
Monitor Box
Extra Large Box
Heavy Duty Box
```

This also makes sense from a warehouse perspective.

10.8 The most important improvement — OrderItem Inline

This is the main feature of Step 10. We create:

```
Python class
OrderItemInline(admin.TabularInline):
    model = OrderItem
    extra = 1
```

What is Inline?

Inline means:

"Show related records inside the parent object's admin page." Our

relationship is:

```
Order
|
|— OrderItem  |— OrderItem
|— OrderItem
```

So when you open an Order, you can manage its OrderItems on the same page.

10.9 Order Admin

We add:

```
Python
@admin.register(Order)
class
OrderAdmin(admin.ModelAdmin):
```

Then:

```
Python list_display = (    "id",
    "created_at",
)
```

The order list becomes:

```
ID | Created At
-----
7  | Sep 3, 2026
6  | Sep 3, 2026
5  | Sep 3, 2026
4  | Sep 3, 2026
```

And:

```
Python ordering =
(
    "-
created_at", )
```

means newest orders appear first.

The - means descending order.

10.10 Add products directly to an Order

Because we added:

```
Python
inline = [
    OrderItemInline,
]
```

when you open:

Order #1

you will see something like:

```
Order #1

Created at:
September 3, 2026

Order items
-----

Product          Quantity
[ Laptop          ▼] [ 1 ]
[ Wireless Mouse  ▼] [ 2 ]
[ Headphones      ▼] [ 1 ] -----
-----

[ SAVE ]
```

Now you don't have to separately visit **Order Items**.

10.11 How to create your test orders now

Go to:

```
Admin
↓
Orders
↓
Add Order
```

Click **Save**.

Then inside the same page, add: **Order #1**

```
Product          Quantity
-----
Laptop            1
```

Save.

Order #2

Product	Quantity
-----Wireless Mouse	2
Headphones	1

Save.

Order #3

Product	Quantity

Smartphone	2
Wireless Mouse	1

Save.

Order #4

Product	Quantity

Keyboard	1
Wireless Mouse	1

Save.

Order #5

Product	Quantity

Monitor	1

Save.

10.12 Why this is better

Before:

```

Create Order
↓
Save
↓
Go to OrderItem
↓
Select Order
↓
Select Product
↓
Enter Quantity
↓
Save
↓
Repeat

```

After Step 10:

```
Create Order
  ↓
Add products
  ↓
Add quantities
  ↓
Save
```

Much cleaner.

10.13 OrderItem Admin

We also keep a separate OrderItem Admin:

```
Python
@admin.register(OrderItem)
class
OrderItemAdmin(admin.ModelAdmin):
```

with:

```
Python list_display = (    "id",
    "order",
    "product",
    "quantity",
)
```

You will see:

ID	Order	Product	Quantity
1	1	Laptop	1
2	2	Mouse	2
3	2	Headphones	1

This is useful for debugging.

10.14 Search OrderItems by product

We added:

```
Python search_fields = (
    "product__name",
)
```

Notice:

```
product__name
```

There are **two underscores** between `product` and `name`. It means:

```
OrderItem
  ↓
```

Product ↓ name

So if you search:

Laptop

Django can find OrderItems containing the Laptop product.

10.15 Filter OrderItems

We added:

```
Python list_filter = (
    "product",
)
```

Django will provide a filter on the right side. For example:

```
FILTER

Product
☐ Laptop
☐ Keyboard
☐ Mouse
☐ Monitor
☐ Printer
```

This makes admin management easier.

10.16 Check the Admin

Start your server:

```
Bash
python manage.py runserver
```

Open:

```
http://127.0.0.1:8000/admin/
```

You should now have:

```
Django administration

AUTHENTICATION AND AUTHORIZATION
  Users
  Groups

BOX SELECTOR
Products
  Boxes
  Orders
  Order items
```


Click **Products**.

You should see your product table.

Click **Boxes**.

You should see your box table.

Click **Orders**.

You should see your orders.

Open an order and you should see its **Order Items directly inside the Order page**.

10.17 Important interview concepts from Step 10

What is `ModelAdmin`?

Simple answer:

"`ModelAdmin` allows us to customize how a Django model is displayed and managed inside the Django Admin."

What is `list_display`?

"`list_display` defines which model fields are shown as columns in the admin list page."

What is `search_fields`?

"`search_fields` allows the admin user to search records using selected fields."

What is `list_filter`?

"`list_filter` adds filters to the Django Admin so users can quickly filter records."

What is an Inline?

"An inline allows related objects to be created or edited directly from the parent object's admin page."

10.18 Interview answer

If the interviewer asks:

"How did you improve the Django Admin?" You can say:

"I customized the Django Admin using ModelAdmin. I added useful columns with list_display, search using search_fields, filtering using list_filter, and ordering for products, boxes and orders. I also used TabularInline to manage OrderItems directly inside an Order. This makes the admin panel easier for warehouse users to manage."

10.19 Step 10 final structure

Your project is now becoming:

```
box-selection-system/
├── box_selector/
│   ├── migrations/
│   ├── __init__.py
│   └── admin.py
├── Step 10
│   ├── apps.py
│   ├── forms.py
│   ├── models.py
│   ├── services.py
│   ├── tests.py
│   ├── urls.py
│   └── views.py
├── config/
│   ├── settings.py
│   ├── urls.py
│   ├── asgi.py
│   └── wsgi.py
├── templates/
│   ├── box_selector/
│   │   ├── base.html
│   │   ├── home.html
│   │   ├── product_list.html
│   │   ├── box_list.html
│   │   ├── order_list.html
│   │   ├── order_detail.html
│   │   └── create_order.html
│   └── recommendation.html
├── db.sqlite3
├── manage.py
├── requirements.txt
└── .gitignore
```

☒ Step 10 checklist

Before moving to Step 11, make sure:

- ☐ Products appear correctly in Admin
- ☐ Product search works
- ☐ Boxes show dimensions, weight and cost
- ☐ Boxes are sorted by cost
- ☐ Orders show newest first
- ☐ OrderItems appear inside Orders
- ☐ Products can be added directly to Orders
- ☐ OrderItem search works OrderItem
- ☐ filtering works
- ☐ `python manage.py check` gives no errors `python`
- ☐ `manage.py test` passes

Next Step 11 should be API improvement: we will make the recommendation API more professional, add proper HTTP methods, validation, JSON request/response structure, and API tests.



Step 11 and improve all html page with css

Absolutely. **Step 11** will focus on two things:

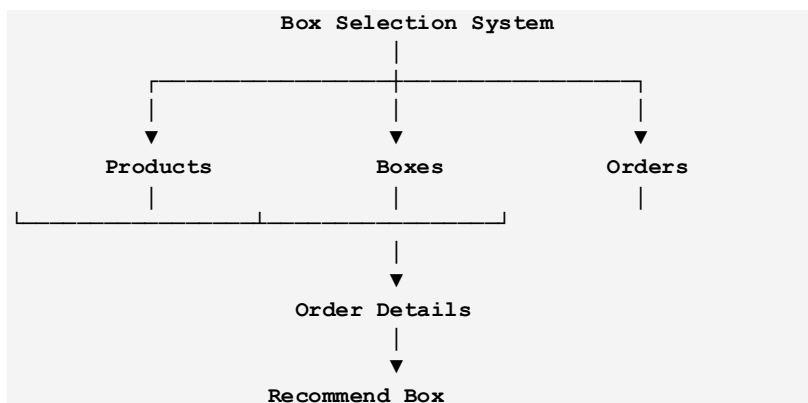
1. Make the recommendation API more professional.
2. Give **all HTML pages a clean, consistent CSS design** so the project looks like a real interview project rather than a basic Django demo.

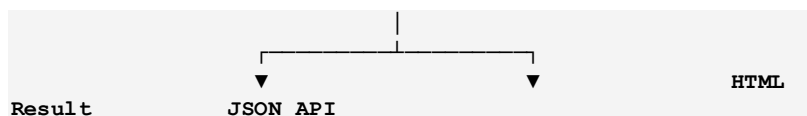
We will keep the code simple enough for you to explain in an interview.

STEP 11 — Professional API + Complete HTML/CSS

11.1 What we will build

After Step 11, your application flow will look like:





PART A — Improve the API

11.2 Current API

Currently we have:

```
GET /orders/1/recommend/
```

which returns:

```
JSON
{
  "success": true,
  "order_id": 1,
  "recommended_box": {
    "id": 2,
    "name": "Medium Box",
    "length": "35.00",
    "width": "25.00",
    "height": "10.00",
    "max_weight": "5.00",
    "cost": "20.00"
  }
}
```

This is already good for a small project.

We will now improve error handling and make the response consistent.

11.3 Create API helper function

Open:

```
box_selector/views.py
```

We can keep the recommendation logic in `services.py` and only handle HTTP-related work inside the view.

Use:

```
Python
from django.http import JsonResponse from
django.shortcuts import get_object_or_404

from .models import Order from
.services import recommend_box

def recommend_box_view(request, order_id):
```

```

    if request.method != "GET":
return JsonResponse(
    {
        "success": False,
        "message": "Only GET method is allowed."
    },
    status=405
)

    order = get_object_or_404(
Order,      id=order_id
    )      box =

recommend_box(order)

    if box is None:
return JsonResponse(
    {
        "success": False,
        "order_id": order.id,
        "message": "No suitable box found for this order."
    },
    status=404
)

return JsonResponse(
    {
        "success": True,
        "order_id": order.id,
        "message": "Suitable box found.",
        "recommended_box": {
            "id": box.id,
            "name": box.name,
            "length": str(box.length),
            "width": str(box.width),
            "height": str(box.height),
            "max_weight": str(box.max_weight),
            "cost": str(box.cost),
        }
    }
)

```

11.4 Why check the HTTP method?

We added:

```

Python if request.method !=
"GET":

```

Our endpoint is only meant to **retrieve a recommendation**. Therefore:

```

GET      → allowed
POST     → not allowed
PUT      → not allowed
DELETE  → not allowed

```

If someone sends POST:

```

JSON
{
    "success": false,
    "message": "Only GET method is allowed."
}

```

with HTTP status:

```

405 Method Not Allowed

```

11.5 API status codes

You should understand these for your interview:

200	Request successful
400	Bad request
404	Resource not found
405	HTTP method not allowed
500	Server error

For our API:

Successful recommendation

```
200 OK
```

Order doesn't exist

```
404 Not Found
```

No suitable box

```
404 Not Found
```

Wrong HTTP method

```
405 Method Not Allowed
```

PART B — Improve HTML

Now let's make the website look professional. We will create a common CSS file:

```
static/css/style.css
```

Our structure becomes:

```
templates/
├── box_selector/
│   ├── base.html
│   │   ├── home.html
│   │   ├── product_list.html
│   │   ├── box_list.html
│   │   └── order_list.html
```

```

├── order_detail.html
├── create_order.html
├── recommendation.html

static/
├── css/
├── style.css

```

11.6 Configure static files

Open:

```
config/settings.py
```

Make sure you have:

```

Python
STATIC_URL =
"static/"

```

For development, we can create:

```
static/
```

in the project root.

So your project becomes:

```

box-selection-system/
├── static/
│   ├── css/
│   └── style.css
├── templates/
├── box_selector/
├── config/
├── manage.py
└── db.sqlite3

```

11.7 Create style.css

Create:

```
static/css/style.css
```

Put this code inside:

```

CSS
* {      box-sizing: border-
box;
}

body {
margin: 0;
font-family: Arial, Helvetica, sans-serif;
background: #f4f6f8;      color: #1f2937;
}

```

```

a {
  text-decoration: none;
color: inherit; }

.navbar {
  background: #1f2937;
color: white;
padding: 16px 0; }

.navbar-container {
width: 90%;      max-
width: 1200px;
margin: auto;
display: flex;
  justify-content: space-between;
align-items: center; }

.logo {
  font-size: 22px;
font-weight: bold; }

.nav-links {
display: flex;
gap: 20px; }

.nav-links a {
color: #d1d5db;
font-size: 15px; }

.nav-links a:hover {
color: white; }

.container {
width: 90%;      max-
width: 1200px;
margin: 30px auto;
}

.page-header {
display: flex;
  justify-content: space-between;
align-items: center;      margin-
bottom: 25px; }

.page-title {
margin: 0;      font-
size: 30px; }

.page-description {
color: #6b7280;
margin-top: 8px; }

.card-grid {
display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
gap: 20px; }

.card {
  background: white;
border-radius: 10px;
padding: 24px;
box-shadow: 0 2px 8px rgba(0, 0, 0, 0.08); }

```



```
.card h3 {
  margin-top: 0;
}

.stat-number {
  font-size: 32px;
  font-weight: bold;
  margin: 10px 0; }

.stat-label {
  color: #6b7280; }

.btn {
  display: inline-block;
  padding: 10px 18px;
  border-radius: 6px;
  border: none;      cursor:
  pointer;          font-size:
  14px;
}

.btn-primary {
  background: #2563eb;
  color: white; }

.btn-primary:hover {
  background: #1d4ed8;
}

.btn-success {
  background: #16a34a;
  color: white; }

.btn-success:hover {
  background: #15803d;
}

.btn-secondary {
  background: #6b7280;
  color: white; }

.btn-secondary:hover {
  background: #4b5563;
}

.table-container {
  background: white;
  border-radius: 10px;
  overflow-x: auto;
  box-shadow: 0 2px 8px rgba(0, 0, 0, 0.08); }

.data-table {      width: 100%;
border-collapse: collapse; }

.data-table th, .data-
table td {      padding:
14px 16px;      text-
align: left;
  border-bottom: 1px solid #e5e7eb; }

.data-table th {
  background: #f9fafb;
  font-size: 14px;
}
```



```
.data-table tr:hover {
background: #f9fafb; }

.badge {
    display: inline-block;
padding: 5px 10px;    border-
radius: 20px;    font-size:
12px;    font-weight: bold;
}

.badge-success {
background: #dcfce7;
color: #166534;
}

.badge-warning {
background: #fff3cd;
color: #85c1e9; }

.form-card {    max-width: 800px;    margin:
auto;    background: white;    padding: 30px;
border-radius: 10px;    box-shadow: 0 2px 8px
rgba(0, 0, 0, 0.08); }

.form-row {
display: grid;
    grid-template-columns: 1fr 1fr;
gap: 20px;
    margin-bottom: 20px;
}

.form-group {
    margin-bottom: 20px;
}

.form-group label {
display: block;
font-weight: bold;
margin-bottom: 8px; }

.form-group select,
.form-group input {
width: 100%;
padding: 11px;
    border: 1px solid #d1d5db;
border-radius: 6px;    font-size:
14px;
}

.error-message {
color: #dc2626;
font-size: 13px;
margin-top: 5px; }

.alert {    padding:
15px;    border-radius:
8px;    margin-bottom:
20px;
}

.alert-error {
background: #fee2e2;
color: #991b1b;
}

.alert-success {
```

```

background: #dcfce7;
color: #166534; }

.recommendation-card {
max-width: 700px;
margin: 30px auto; background: white;
padding: 35px; border-radius: 12px;
text-align: center; box-shadow: 0 4px 12px
rgba(0, 0, 0, 0.08); }

.recommendation-box {
margin: 25px 0;
padding: 25px;
border: 2px solid #16a34a;
border-radius: 10px; }

.recommendation-box h2 {
margin-top: 0; }

.footer {
margin-top: 50px;
padding: 20px;
text-align: center;
color: #6b7280; }

@media (max-width: 700px) {

.navbar-container {
flex-direction: column; gap:
12px;
}

.nav-links {
flex-wrap: wrap;
justify-content: center;
}

.page-header {
flex-direction: column;
align-items: flex-start; gap:
15px;
}

.form-row {
grid-template-
columns: 1fr;
}
}

```

11.8 Why use base.html?

Instead of writing the navbar on every page, we'll write it once. Create/update:

```
templates/box_selector/base.html
```

```

HTML
{% load static %}

<!DOCTYPE html>
<html lang="en">
<head>

```

```

<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>
    {% block title %}
    Box Selection System
    {% endblock %}
</title>

<link rel="stylesheet" href="{% static 'css/style.css' %}">
</head>

<body>

<nav class="navbar">

    <div class="navbar-container">

        <div class="logo">
             Box Selection System
        </div>

        <div class="nav-links">

            <a href="{% url 'home' %}">
                Dashboard
            </a>

            <a href="{% url 'product-list' %}">
                Products
            </a>

            <a href="{% url 'box-list' %}">
                Boxes
            </a>

            <a href="{% url 'order-list' %}">
                Orders
            </a>

            <a href="{% url 'create-order' %}">
                Create Order
            </a>

            <a href="/admin/">
                Admin
            </a>

        </div>

    </div>

</nav>

<main class="container">

    {% block content %}
    {% endblock %}

</main>

<footer class="footer">
    <p>
        Box Selection System © 2026
    </p>
</footer>

</body>
</html>

```

11.9 Dashboard — home.html

Update:

Printed using [ChatGPT to PDF](#), powered by PDFCrowd [HTML to PDF API](#).

templates/box_selector/home.html

HTML

```
{% extends "box_selector/base.html" %}

{% block title %} Dashboard |
Box Selection System
{% endblock %}

{% block content %}

<div class="page-header">

    <div>
        <h1 class="page-title">
            Dashboard
        </h1>

        <p class="page-description">
            Manage products, boxes and shipping orders.
        </p>
    </div>

    <a href="{% url 'create-order' %}"
class="btn btn-primary">
        + Create Order
    </a>

</div>

<div class="card-grid">

    <div class="card">

        <div class="stat-label">
            Products
        </div>

        <div class="stat-number">
            {{ product_count }}
        </div>

        <a href="{% url 'product-list' %}"
class="btn btn-secondary">
            View Products
        </a>

    </div>

    <div class="card">

        <div class="stat-label">
            Boxes
        </div>

        <div class="stat-number">
            {{ box_count }}
        </div>

        <a href="{% url 'box-list' %}"
class="btn btn-secondary">
            View Boxes
        </a>

    </div>

    <div class="card">
        <div class="stat-label">
            Orders
        </div>
    </div>

</div>
```

```

<div class="stat-number">
    {{ order_count }}
</div>

<a href="{% url 'order-list' %}" class="btn btn-secondary">
    View Orders
</a>

</div>

</div>

{% endblock %}

```

11.10 Update Dashboard View

Your `home` view needs to send these values.

In `views.py`:

```

Python
from django.shortcuts import render from
.models import Box, Order, Product

def home(request):

    context = {
        "product_count": Product.objects.count(),
        "box_count": Box.objects.count(),
        "order_count": Order.objects.count(),
    }

    return render(
request,
    "box_selector/home.html",
    context
    )

```

Now your dashboard will dynamically show:

Products	10	View Products
Boxes	6	View Boxes
Orders	7	View Orders

11.11 Product List

Update:

```

templates/box_selector/product_list.html

```

HTML

```
{% extends "box_selector/base.html" %}

{% block title %} Products |
Box Selection System
{% endblock %}

{% block content %}

<div class="page-header">

    <div>
        <h1 class="page-title">
            Products
        </h1>

        <p class="page-description">
            All products available for shipping.
        </p>
    </div>

</div>

<div class="table-container">

<table class="data-table">

    <thead>
        <tr>
            <th>ID</th>
            <th>Product</th>
            <th>Length</th>
            <th>Width</th>
            <th>Height</th>
            <th>Weight</th>
        </tr>
    </thead>

    <tbody>

        {% for product in products %}

        <tr>

            <td>
                {{ product.id }}
            </td>

            <td>
                <strong>{{ product.name }}</strong>
            </td>

            <td>
                {{ product.length }} cm
            </td>

            <td>
                {{ product.width }} cm
            </td>

            <td>
                {{ product.height }} cm
            </td>

            <td>
                {{ product.weight }} kg
            </td>

        </tr>

        {% empty %}

        <tr>
```



```

        <td colspan="6">No products found.
    </td>
</tr>

{% endfor %}

</tbody>

</table>

</div>

{% endblock %}

```

11.12 Box List

Update:

templates/box_selector/box_list.html

```

HTML
{% extends "box_selector/base.html" %}

{% block title %} Boxes |
Box Selection System
{% endblock %}

{% block content %}

<div class="page-header">

    <div>
        <h1 class="page-title">
            Shipping Boxes
        </h1>

        <p class="page-description">
            Available boxes and their shipping costs.
        </p>
    </div>

</div>

<div class="table-container">

<table class="data-table">

    <thead>

        <tr>
            <th>ID</th>
            <th>Box</th>
            <th>Dimensions</th>
            <th>Max Weight</th>
            <th>Cost</th>
        </tr>
    </thead>

```

```

</thead>

<tbody>

    {% for box in boxes %}

        <tr>

            <td>
                {{ box.id }}
            </td>

            <td>
                <strong>{{ box.name }}</strong>
            </td>

            <td>
                {{ box.length }} x
                {{ box.width }} x
                {{ box.height }} cm
            </td>

            <td>
                {{ box.max_weight }} kg
            </td>

            <td>
                ₹{{ box.cost }}
            </td>

        </tr>

        {% empty %}

            <tr>
                <td colspan="5">
                    No boxes available.
                </td>
            </tr>

        {% endfor %}

    </tbody>

</table>

</div>

{% endblock %}

```

11.13 Order List

Update:

templates/box_selector/order_list.html

```

HTML
{% extends "box_selector/base.html" %}

{% block title %} Orders |
Box Selection System
{% endblock %}

{% block content %}

<div class="page-header">

    <div>
        <h1 class="page-title">

```

```

    Orders
</h1>

<p class="page-description">
    View and manage customer orders.
</p>
</div>

<a href="{% url 'create-order' %}"
class="btn btn-primary">
    + Create Order
</a>

</div>

<div class="table-container">
<table class="data-table">

    <thead>

        <tr>
            <th>Order ID</th>
            <th>Created At</th>
            <th>Action</th>
        </tr>

    </thead>

    <tbody>

        {% for order in orders %}

        <tr>

            <td>
                <strong>
                    Order #{{ order.id }}
                </strong>
            </td>

            <td>
                {{ order.created_at|date:"d M Y, h:i A" }}
            </td>

            <td>
                <a
                    href="{% url 'order-detail' order.id %}"
                    class="btn btn-secondary"
                >
                    View Order
                </a>
            </td>

        </tr>

        {% empty %}

        <tr>
            <td colspan="3">
                No orders found.
            </td>
        </tr>

        {% endfor %}

    </tbody>

</table>

</div>

```

```
{% endblock %}
```

11.14 Create Order Page

Update:

```
templates/box_selector/create_order.html
```

HTML

```
{% extends "box_selector/base.html" %}

{% block title %} Create Order | Box
Selection System
{% endblock %}

{% block content %}

<div class="page-header">

    <div>
        <h1 class="page-title">
            Create New Order
        </h1>

        <p class="page-description">
            Select products and enter their quantities.
        </p>
    </div>

</div>

<div class="form-card">

    <form method="POST">

        {% csrf_token %}

        {{ formset.management_form }}

        {% if formset.non_form_errors %}

            <div class="alert alert-error">

                {% for error in formset.non_form_errors %}

                    {{ error }}

                {% endfor %}

            </div>

        {% endif %}

        {% for form in formset %}

            <div class="form-row">

                <div class="form-group">

                    <label>
                        Product
                    </label>

                    {{ form.product }}

                    {% for error in form.product.errors %}
```

```

        <div class="error-message">
            {{ error }}
        </div>

    {% endfor %}

</div>

<div class="form-group">

    <label>
        Quantity
    </label>

    {{ form.quantity }}

    {% for error in form.quantity.errors %}

        <div class="error-message">
            {{ error }}
        </div>

    {% endfor %}

</div>

</div>

{% endfor %}

<button
type="submit"
class="btn btn-success"
>
    Create Order
</button>

<a
href="{% url 'order-list' %}"
class="btn btn-secondary"
>
    Cancel
</a>

</form>

</div>

{% endblock %}

```

11.15 Order Detail Page

Update:

```

templates/box_selector/order_detail.html

HTML
{% extends "box_selector/base.html" %}

{% block title %} Order #{{ order.id }} | Box Selection
System
{% endblock %}

{% block content %}

<div class="page-header">

```

```

<div>

  <h1 class="page-title">
    Order #{{ order.id }}
  </h1>

  <p class="page-description">
    Created on
    {{ order.created_at|date:"d M Y, h:i A" }}
  </p>

</div>

<a
  href="{% url 'recommendation-page' order.id %}"
  class="btn btn-success"
>
  Recommend Box
</a>

</div>

<div class="card">

  <h2>
    Order Items
  </h2>

  <div class="table-container">
<table class="data-table">

  <thead>

    <tr>
      <th>Product</th>
      <th>Quantity</th>
      <th>Weight</th>
    </tr>

  </thead>

  <tbody>

    {% for item in order.items.all %}

    <tr>

      <td>
        <strong>
          {{ item.product.name }}
        </strong>
      </td>

      <td>
        {{ item.quantity }}
      </td>

      <td>
        {{ item.product.weight }} kg each
      </td>

    </tr>

    {% empty %}

    <tr>
      <td colspan="3">
        No items in this order.
      </td>
    </tr>

    {% endfor %}

  </tbody>
</table>

</div>

```

```

        </tbody>

    </table>

</div>

</div>

{% endblock %}

```

11.16 Recommendation Page

This is the most important page. Update:

templates/box_selector/recommendation.html

```

HTML
{% extends "box_selector/base.html" %}

{% block title %} Box Recommendation |
Order #{{ order.id }}
{% endblock %}

{% block content %}

<div class="recommendation-card">

    <h1>
        📦 Box Recommendation
    </h1>

    <p class="page-description">
        Recommended shipping box for Order #{{ order.id }}
    </p>

    {% if box %}

        <div class="alert alert-success">

            Suitable box found successfully.

        </div>

        <div class="recommendation-box">

            <h2>
                {{ box.name }}
            </h2>

            <p>

                <strong>
                    Dimensions:
                </strong>

                {{ box.length }}
                x
                {{ box.width }}
                x
                {{ box.height }}
                cm
            </p>

            <p>

                <strong>

```

```

        Maximum Weight:
    </strong>

    {{ box.max_weight }} kg

</p>

<p>

    <strong>                Box Cost:
    </strong>

    ₹{{ box.cost }}

</p>
</div>

<a
    href="{% url 'order-detail' order.id %}"
    class="btn btn-secondary"
>
    Back to Order
</a>

{% else %}

<div class="alert alert-error">

    No suitable box was found for this order.

</div>

<a
    href="{% url 'order-detail' order.id %}"
    class="btn btn-secondary"
>
    Back to Order
</a>

{% endif %}

</div>

{% endblock %}

```

11.17 Make sure your URL names exist

Your `box_selector/urls.py` should have all these URLs:

```

Python
from django.urls import path

from .views import (
    create_order,      home,
    order_detail,
    order_list,
    product_list,
    box_list,
    recommend_box_view,
    recommendation_page,

```



```

)

urlpatterns = [

    path(
        "", home,
        name="home"
    ),

    path(
        "products/",
        product_list,
        name="product-list"
    ),

    path(
        "boxes/",
        box_list,
        name="box-list"
    ),

    path(
        "orders/",
        order_list,
        name="order-list"
    ),

    path(
        "orders/create/",
        create_order, name="create-
order"
    ),

    path(
        "orders/<int:order_id>/",
        order_detail, name="order-detail"
    ),

    path(
        "orders/<int:order_id>/recommend/",
        recommend_box_view, name="recommend-box"
    ),

    path(
        "orders/<int:order_id>/recommendation/",
        recommendation_page, name="recommendation-page"
    ),
]

```

11.18 Important: Check your views

Your `views.py` should contain the corresponding HTML views.

A clean version is:

```

Python
from django.db import transaction from
django.http import JsonResponse
from django.shortcuts import get_object_or_404, redirect, render

from .forms import OrderItemFormSet
from .models import Box, Order, OrderItem, Product from
.services import recommend_box

```

```
def home(request):

    context = {
        "product_count": Product.objects.count(),
        "box_count": Box.objects.count(),
        "order_count": Order.objects.count(),
    }

    return render(
request,
        "box_selector/home.html",
context
    )

def product_list(request):

    products = Product.objects.all().order_by("name")

    return render(
request,
        "box_selector/product_list.html",
{
            "products": products
        }
    )

def box_list(request):

    boxes = Box.objects.all().order_by("cost")

    return render(
request,
        "box_selector/box_list.html",
{
            "boxes": boxes
        }
    )

def order_list(request):

    orders = Order.objects.all().order_by("-created_at")

    return render(
request,
        "box_selector/order_list.html",
{
            "orders": orders
        }
    )

def order_detail(request, order_id):

    order = get_object_or_404(
Order,
        id=order_id
    )

    return render(
request,
        "box_selector/order_detail.html",
{
            "order": order
        }
    )

def create_order(request):

    if request.method == "POST":

        formset = OrderItemFormSet(request.POST)
```



```

        if formset.is_valid():
valid_items = []
        for
form in formset:

            product = form.cleaned_data.get("product")
quantity = form.cleaned_data.get("quantity")

            if not product and not quantity:
continue

            if product and quantity:
valid_items.append(
(product, quantity)
)

transaction.atomic():

            order = Order.objects.create()

for product, quantity in valid_items:

            OrderItem.objects.create(
order=order,
quantity=quantity,
product=product,
)

            return redirect(
"order-detail",
order_id=order.id
)

else:

            formset = OrderItemFormSet()

            return render(
request,
"box_selector/create_order.html",
{
    "formset": formset
}
)

def recommendation_page(request, order_id):

    order = get_object_or_404(
Order,
id=order_id
)
    box =

recommend_box(order)

    return render(
request,
"box_selector/recommendation.html",
{
    "order": order,
    "box": box
}
)

def recommend_box_view(request, order_id):

    if request.method != "GET":

        return JsonResponse(
{
    "success": False,
    "message": "Only GET method is allowed."
}
)

```

```

        },
        status=405
    )

    order = get_object_or_404(
        Order,
        id=order_id
    )
    box =

recommend_box(order)    if

box is None:

    return JsonResponse(
        {
            "success": False,
            "order_id": order.id,
            "message": "No suitable box found for this order."
        },
        status=404
    )

    return JsonResponse(
        {
            "success": True,
            "order_id": order.id,
            "message": "Suitable box found.",
            "recommended_box": {
                "id": box.id,
                "name": box.name,
                "length": str(box.length),
                "width": str(box.width),
                "height": str(box.height),
                "max_weight": str(box.max_weight),
                "cost": str(box.cost),
            }
        }
    )

```

11.19 Test the complete website

Start:

```

Bash python manage.py
runserver

```

Open:

```

http://127.0.0.1:8000/

```

You should now have a dashboard.

Test the complete flow:

Step 1 Open:

```

/

```

Dashboard.

Step 2

Open:

```
/products/
```

Check products.

Step 3 Open:

```
/boxes/
```

Check boxes.

Step 4 Open:

```
/orders/
```

Check orders.

Step 5

Click:

```
+ Create Order
```

Add:

```
Laptop x 1
```

Step 6

After creating the order, click:

```
Recommend Box
```

You should see:

```

📦 Box Recommendation

Suitable box found successfully.

Medium Box

Dimensions:
35 x 25 x 10 cm

Maximum Weight:
5 kg

Box Cost:
₹20
```

11.20 Test the API

Open:

```
http://127.0.0.1:8000/orders/1/recommend/
```

You should receive JSON:

```
JSON
{
  "success": true,
  "order_id": 1,
  "message": "Suitable box found.",
  "recommended_box": {
    "id": 2,
    "name": "Medium Box",
    "length": "35.00",
    "width": "25.00",
    "height": "10.00",
    "max_weight": "5.00",
    "cost": "20.00"
  }
}
```

11.21 Test "No suitable box"

Use your large product:

```
Very Large Machine
100 x 100 x 100 cm
30 kg
```

Create an order with:

```
Very Large Machine x 1
```

Click:

Recommend Box

You should see:

```
No suitable box was found for this order.
```

API:

```
JSON
{
  "success": false,
  "order_id": 7,
  "message": "No suitable box found for this order."
}
```

11.22 Test invalid URL

Try:

```
http://127.0.0.1:8000/orders/9999/recommend/
```

If Order 9999 doesn't exist, Django should return:

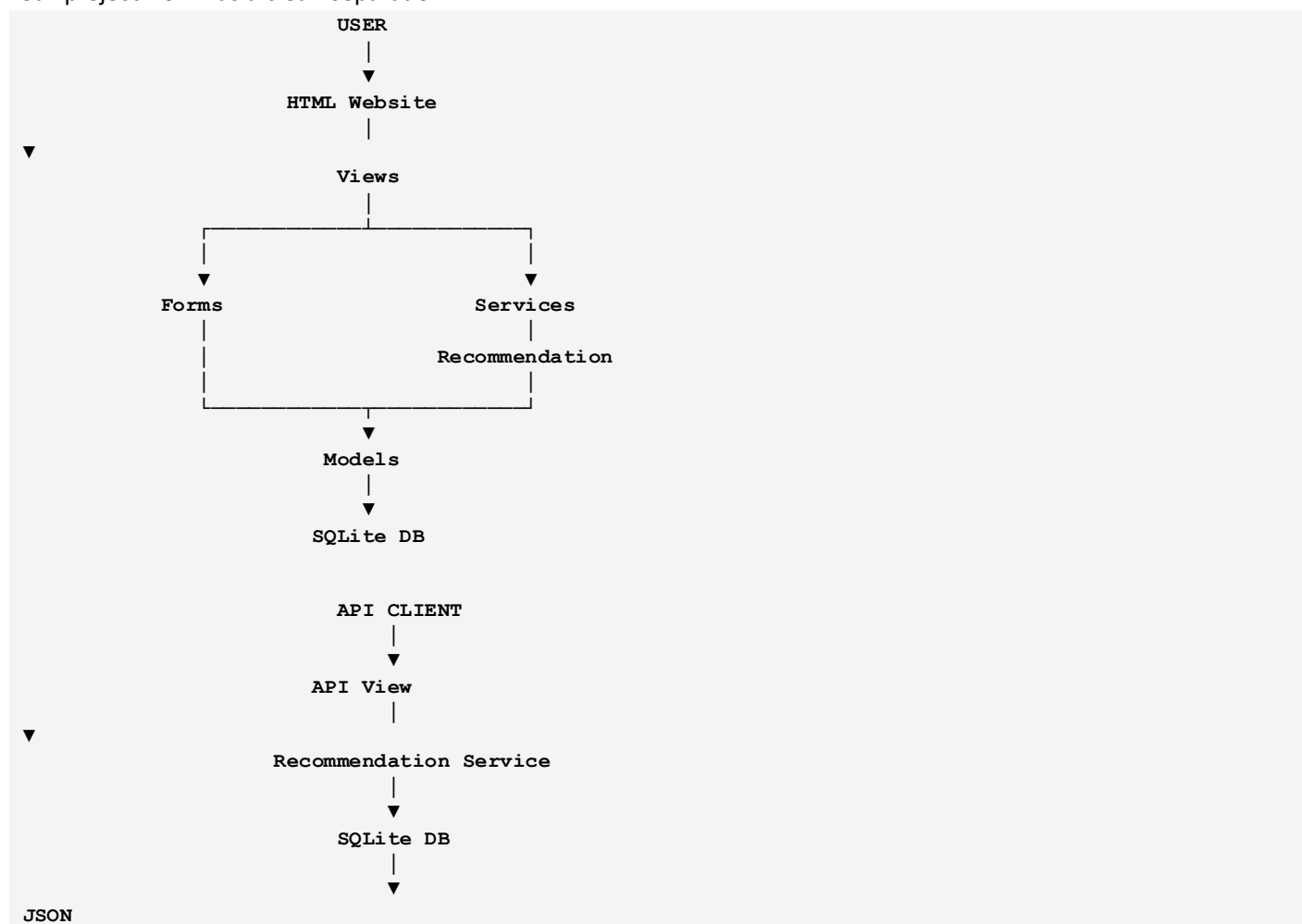
```
404 Not Found
```

because we use:

Python `get_object_or_404()`

11.23 Step 11 architecture

Your project now has a clean separation:



This is a good architecture for a fresher/interview project.

Step 11 Interview Questions

Q1. Why did you use a separate `services.py`?

Answer:

"I keep the main business logic in `services.py` instead of putting it directly inside the view. This makes the code easier to test, maintain and reuse."

Q2. Why use templates?

"I used Django templates to create the web interface. Templates allow me to display database data dynamically while keeping the presentation separate from the business logic."

Q3. Why use static CSS?

"I separated CSS into a static file instead of writing CSS inside every HTML page. This allows all pages to use the same design and makes the project easier to maintain."

Q4. Why use `base.html`?

"I use template inheritance. The common navbar, CSS and footer are kept in `base.html`, and other pages extend it. This avoids repeating the same HTML code."

Q5. Why use GET for recommendation?

"The recommendation operation only reads the order and calculates the suitable box. It doesn't modify database data, so GET is appropriate." **Q6. What happens when no box is suitable?**

"The service returns None. The HTML page displays a clear message, and the API returns a JSON response indicating that no suitable box was found."

Step 11 Checklist

Before moving forward, verify:

- ☐ API only accepts GET
- ☐ API returns JSON
- ☐ API handles missing orders API
- ☐ handles no suitable box
- ☐ `base.html` created
- ☐ Common CSS created
- ☐ Dashboard redesigned
- ☐ Product page redesigned
- ☐ Box page redesigned
- ☐ Order page redesigned
- ☐ Create Order page redesigned Order
- ☐ Detail page redesigned
- ☐ Recommendation page redesigned
- ☐ Navigation works Admin link
- ☐ works Mobile CSS added `python`
- ☐ `manage.py` check passes `python`
- ☐ `manage.py` test passes

One important project-quality point

Your current recommendation algorithm is intentionally a **simplified packing model**: it checks the largest required dimensions and total weight. It does **not** perform true 3D packing or product rotation. For your interview, describe this honestly as a documented business assumption rather than claiming it solves general warehouse bin-packing.



now give me settings.py

Yes. For your current project, use this **complete** `config/settings.py`. It includes the important settings we have used in Steps 1–11: Django apps, SQLite, templates, static CSS, timezone, and basic security settings.

Replace your current:

`config/settings.py`

with this:

```
Python from pathlib
import Path

# Build paths inside the project like this:
# BASE_DIR / 'subdir'.
BASE_DIR = Path(__file__).resolve().parent.parent

# SECURITY WARNING: keep the secret key used in production secret!
SECRET_KEY = "django-insecure-change-this-in-production"

# SECURITY WARNING: don't run with debug turned on in production! DEBUG
= True

ALLOWED_HOSTS = []

# Application definition

INSTALLED_APPS = [
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
```

```

"django.contrib.staticfiles",

# Our application
"box_selector",
]

MIDDLEWARE = [
    "django.middleware.security.SecurityMiddleware",

    "django.contrib.sessions.middleware.SessionMiddleware",

    "django.middleware.common.CommonMiddleware",

    "django.middleware.csrf.CsrfViewMiddleware",

    "django.contrib.auth.middleware.AuthenticationMiddleware",

    "django.contrib.messages.middleware.MessageMiddleware",

    "django.middleware.clickjacking.XFrameOptionsMiddleware", ]

ROOT_URLCONF = "config.urls"

TEMPLATES = [
    {
        "BACKEND": "django.template.backends.django.DjangoTemplates",
        # Project-level templates folder
        "DIRS": [
            BASE_DIR / "templates",
        ],
        "APP_DIRS": True,
        "OPTIONS": {
            "context_processors": [
                "django.template.context_processors.request",

                "django.contrib.auth.context_processors.auth",

                "django.contrib.messages.context_processors.messages",
            ],
        },
    ],
]

WSGI_APPLICATION = "config.wsgi.application"

# Database
# https://docs.djangoproject.com/en/stable/ref/settings/#databases

DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.sqlite3",
        "NAME": BASE_DIR / "db.sqlite3",
    }
}

# Password validation

AUTH_PASSWORD_VALIDATORS = [
    {
        "NAME": (
            "django.contrib.auth.password_validation."
            "UserAttributeSimilarityValidator"
        ),
    },
    {

```

```

        "NAME": (
            "django.contrib.auth.password_validation."
            "MinimumLengthValidator"
        ),
    },
    {
        "NAME": (
            "django.contrib.auth.password_validation."
            "CommonPasswordValidator"
        ),
    },
    {
        "NAME": (
            "django.contrib.auth.password_validation."
            "NumericPasswordValidator"
        ),
    },
]

# Internationalization

LANGUAGE_CODE = "en-us"

TIME_ZONE = "Asia/Kolkata"

USE_I18N = True

USE_TZ = True

# Static files (CSS, JavaScript, Images)

STATIC_URL = "static/"

STATICFILES_DIRS = [
    BASE_DIR / "static",
]

# Default primary key field type

DEFAULT_AUTO_FIELD = "django.db.models.BigAutoField"

```

Your important project settings

1. Register your app

This is very important:

```

Python
"box_selector",

```

Without this, Django won't properly recognize your application.

2. Templates

created:

```

templates/
box_selector/

```

So this setting is required:

```
Python
"DIRS": [
    BASE_DIR / "templates",
],
```

It tells Django:

"Look for HTML templates inside the project's `templates` folder."

3. Static CSS

created:

```
static/
├── css/
│   └── style.css
```

Therefore we use:

```
Python
STATIC_URL = "static/"

STATICFILES_DIRS = [
    BASE_DIR / "static",
]
```

Then in `base.html`:

```
HTML
{% load static %}

<link rel="stylesheet" href="{% static 'css/style.css' %}">
```

Django will load:

```
static/css/style.css
```

4. SQLite database

For your interview assignment, SQLite is perfectly suitable for this small project:

```
Python
DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.sqlite3",
        "NAME": BASE_DIR / "db.sqlite3",
    }
}
```

Your database file will be:

```
db.sqlite3
```

5. Time zone

Because you're developing this project in India, I've set:

```
Python
TIME_ZONE = "Asia/Kolkata"
```

So Django will use Indian Standard Time.

Final project structure

After all these steps, your project should approximately look like:

```
box-selection-system/
├── box_selector/
│   ├── migrations/
│   ├── __init__.py
│   ├── admin.py
│   ├── apps.py
│   ├── forms.py
│   ├── models.py
│   ├── services.py
│   ├── tests.py
│   ├── urls.py
│   └── views.py
├── config/
│   ├── __init__.py
│   ├── asgi.py
│   └── settings.py
├── THIS FILE
│   ├── urls.py
│   └── wsgi.py
├── static/
│   └── css/
│       └── style.css
├── templates/
│   └── box_selector/
│       ├── base.html
│       ├── home.html
│       ├── product_list.html
│       ├── box_list.html
│       ├── order_list.html
│       ├── order_detail.html
│       ├── create_order.html
│       └── recommendation.html
├── db.sqlite3
├── manage.py
├── requirements.txt
└── .gitignore
```

Test it

After saving `settings.py`, run:

```
Bash python manage.py
check
```

Expected:

```
System check identified no issues (0 silenced).
```

Then:

```
Bash python manage.py
runserver
```

Open:

```
http://127.0.0.1:8000/
```

If the CSS is not showing, make sure this exact structure exists:

```
static/
├── css/
│   └── style.css
```

and that `base.html` contains:

```
HTML
{% load static %}

<link rel="stylesheet" href="{% static 'css/style.css'
%}">
```

One security note: for GitHub, don't keep a real production `SECRET_KEY` in the repository. For this interview project, the development key above is okay, but later we can move it into a `.env` file.



create-order page update with when first show one product add then user click on plus button then user add second product in one box

Yes. This is a good improvement for the **Create Order** page. We will change the behavior from:

```
Product 1
Product 2
Product 3
```

always being visible, to:

```
Product
Quantity [+ Add Product]
```

Initially, only **one product row** appears.

When the user clicks **+ Add Product**, a second row appears. Example:

Create New Order

```
Product      Quantity
[ Laptop ▼ ] [ 1 ]

[+ Add Product]
```

After clicking + Add Product:

Product	Quantity	
▼ [1]		[Laptop
[Mouse ▼]	[2]	[Remove]
[+ Add Product]		

We'll keep Django's **formset**, but change it to start with one row and use JavaScript to dynamically add rows.

1. Update forms.py

Change:

```
Python
extra=3
```

to:

```
Python
extra=1
```

Your forms.py should be:

```
Python
from django import forms
from django.forms import formset_factory from
.models import Product

class OrderItemForm(forms.Form):
    product = forms.ModelChoiceField(
        queryset=Product.objects.all(),
        required=False,
    )
    quantity = forms.IntegerField(
        min_value=1, required=False,
    )

class BaseOrderItemFormSet(forms.BaseFormSet):

def clean(self):
    super().clean()

    if any(self.errors):
return
```



```

        products = []
has_items = False
for
form in self.forms:

    product = form.cleaned_data.get("product")
    quantity = form.cleaned_data.get("quantity")

    # Empty row
    if not
product and not quantity:
continue
has_items = True

    # Product missing
if not product:
    form.add_error(
        "product",
        "Please select a product."
    )

    # Quantity missing
if not quantity:
    form.add_error(
        "quantity",
        "Please enter a quantity."
    )

    # Duplicate product
if product:
    if product
in products:

        form.add_error(
            "product",
            "This product is already added to the order."
        )

else:

    products.append(product)

if not has_items:

    raise forms.ValidationError(
        "Please add at least one product to the order."
    )

OrderItemFormSet = formset_factory(
    OrderItemForm,
    formset=BaseOrderItemFormSet,
    extra=1,
)

```

Important

This:

```

Python
extra=1

```

means:

Show one empty product row when the page first opens.

2. Update create_order.html

Now replace your current:

```
templates/box_selector/create_order.html
```

with this:

```
HTML
{% extends "box_selector/base.html" %}

{% block title %} Create Order | Box
Selection System
{% endblock %}

{% block content %}

<div class="page-header">

    <div>
        <h1 class="page-title">
            Create New Order
        </h1>

        <p class="page-description">
            Add one or more products to create a shipping order.
        </p>
    </div>

</div>

<div class="form-card">

    <form method="POST" id="order-form">

        {% csrf_token %}

        {{ formset.management_form }}

        {% if formset.non_form_errors %}

            <div class="alert alert-error">

                {% for error in formset.non_form_errors %}

                    {{ error }}

                {% endfor %}

            </div>

        {% endif %}

        <div id="order-items">

            {% for form in formset %}

                <div class="order-item-row">

<div class="form-group">

                <label>
                    Product
                </label>

                {{ form.product }}
```

```

        {% for error in form.product.errors %}

            <div class="error-message">
                {{ error }}
            </div>

        {% endfor %}

    </div>

    <div class="form-group">

        <label>
            Quantity
        </label>

        {{ form.quantity }}

        {% for error in form.quantity.errors %}

            <div class="error-message">
                {{ error }}
            </div>

        {% endfor %}

    </div>

    <div class="remove-button-container">

        <button
type="button"
            class="btn btn-danger remove-item"
>
            Remove
        </button>

    </div>

</div>

{% endfor %}

</div>

<div class="order-actions">

    <button
type="button"
id="add-product"
        class="btn btn-secondary"
    >
        + Add Product
    </button>

</div>

<div class="submit-actions">

    <button
type="submit"
        class="btn btn-success"
    >
        Create Order
    </button>

    <a
        href="{% url 'order-list' %}"
class="btn btn-secondary"
    >

```



```

        Cancel
    </a>

</div>

</form>

</div>

<script> document.addEventListener("DOMContentLoaded",
function () {    const container =
document.getElementById("order-items");    const addButton =
document.getElementById("add-product");

    const totalForms = document.getElementById(
        "id_form-TOTAL_FORMS"
    );

    function addProductRow() {

        const formIndex = parseInt(
totalForms.value
        );

        const firstRow = container.querySelector(
            ".order-item-row"
        );

        const newRow = firstRow.cloneNode(true);

        newRow.innerHTML = newRow.innerHTML.replace(
            /form-\d+/g,
            `form-${formIndex}`
        );

        newRow.querySelectorAll("input").forEach(
            function (input) {
input.value = "";
            }
        );

        newRow.querySelectorAll("select").forEach(
            function (select) {
                select.selectedIndex = 0;
            }
        );

        newRow.querySelectorAll(".error-message").forEach(
            function (error) {
error.remove();
            }
        );

        container.appendChild(newRow);

        totalForms.value = formIndex + 1;

        updateRemoveButtons();

    }

```

```
function updateRemoveButtons() {

  const rows = container.querySelectorAll(
    ".order-item-row"
  );

  rows.forEach(function (row) {

    const removeButton =
      row.querySelector(".remove-item");

    removeButton.addEventListener(
      "click",
      function () {

        if (container.querySelectorAll(
          ".order-item-row"
        ).length === 1) {

          return;

        }

        row.remove();

      }
    );

  });

  addButton.addEventListener(
    "click",
    addProductRow
  );

  updateRemoveButtons();

});

</script>

{% endblock %}
```

3. Add CSS for the new rows

Open:

```
static/css/style.css
```

Add this at the bottom:

```
CSS
.order-item-row {
  display: grid;
  grid-template-columns: 2fr 1fr auto;
  gap: 15px; align-items: end;

  padding: 20px; margin-
  bottom: 15px; background:
  #f9fafb;
```

```

        border: 1px solid #e5e7eb;        border-
radius: 8px;
    }

.remove-button-container {
padding-bottom: 20px; }

.btn-danger {
    background: #dc2626;
color: white; }

.btn-danger:hover {
background: #b91c1c;
}

.order-actions {
margin-top: 10px;
margin-bottom: 25px;
}

.submit-actions {
display: flex;
gap: 10px;
padding-top: 20px;
border-top: 1px solid #e5e7eb; }

@media (max-width: 700px) {

    .order-item-row {                grid-
template-columns: 1fr;            }

    .remove-button-container {
padding-bottom: 0;            }

    .submit-actions {
        flex-direction: column;
    }
}

```

4. How the JavaScript works

This is the important part. When
the page opens:

```
TOTAL_FORMS = 1
```

So Django knows:

```
There is currently 1 form.
```

The user sees:

```
Product 1
```

User clicks + Add Product

JavaScript reads:

```
JavaScript const formIndex = parseInt(
totalForms.value
);
```

Initially:

```
formIndex = 1
```

Then it clones the first row:

```
JavaScript const newRow =
firstRow.cloneNode(true);
```

So:

```
Product 1
```

becomes:

```
Product 1
Product 2
```

Then we change:

```
form-0
```

to:

```
form-1
```

using:

```
JavaScript newRow.innerHTML =
newRow.innerHTML.replace(
    /form-\d+/g,
    `form-${formIndex}`
);
```

This is important because Django formsets expect unique indexes.

5. Example

Initially:

```
form-0-product form-0-quantity
```

After clicking:

```
form-0-product form-0-quantity
```



```
form-1-product form-1-quantity
```

After clicking again:

```
form-0-product form-0-quantity
form-1-product form-1-quantity
form-2-product form-2-quantity
```

And the management form changes:

```
TOTAL_FORMS = 1
```

to:

```
TOTAL_FORMS = 2
```

and then:

```
TOTAL_FORMS = 3
```

This allows Django to understand how many forms were submitted.

6. Remove button

We also added:

```
Remove
```

for every additional row. For

example:

Laptop	1	
Mouse	2	[Remove]
Keyboard	1	[Remove]

If the user removes Mouse:

Laptop	1
Keyboard	1

One important limitation

For this first implementation, when a row is removed, we remove it from the browser.

For a production-quality dynamic Django formset, we should handle form indexes/deletion more carefully using Django's `can_delete` mechanism or rebuild indexes before submission.

For your interview project, I recommend we improve this in the **next step** rather than hiding the limitation.

7. Your final user experience Page opens

Create New Order

Add one or more products...

Product

[Select Product ▼]

Quantity

[1]

[+ Add Product]

[Create Order]

[Cancel]

Click + Add Product

Product

[Laptop ▼]

Quantity

[1]

Product

[Select Product ▼]

Quantity

[1]

[Remove]

[+ Add Product]

[Create Order]

[Cancel]

Click again

Laptop

x 1

Mouse

x 2

Keyboard

x 1

[+ Add Product]

This gives you the **one-by-one product addition** behavior you want.

8. Important: Test it

Run:

Bash

python manage.py check

Then:

Bash

python manage.py

runserver

Open:

```
http://127.0.0.1:8000/orders/create/
```

Test:

Test 1

Page opens → **only one product row.**

Test 2

Click:

```
+ Add Product
```

→ second row appears.

Test 3

Click again → third row appears.

Test 4

Try:

```
Laptop x 1  
Laptop x 2
```

→ validation should say:

```
This product is already added to the order.
```

Test 5

Try submitting without selecting a product.

→

```
Please add at least one product to the order.
```

Test 6

Select:

```
Laptop x 1  
Mouse x 2
```

→ Order should be created successfully.

Interview explanation

If the interviewer asks:

"How did you allow users to add multiple products dynamically?" Say:

"I used a Django formset for order items. Initially, I display one form. When the user clicks Add Product, JavaScript dynamically creates another form row and updates the formset management form's TOTAL_FORMS value. This allows the user to add multiple products without refreshing the page. On the backend, Django validates the submitted formset and prevents invalid or duplicate products."

This is a **much better UI** than showing three fixed rows from the beginning.